Microsoft Azure: Zero to Hero - The Complete Guide

Section 1: Welcome

Course Introduction

1. Why Cloud?
    1. 90% of software companies use the cloud
    2. Global cloud market expected to reach 623.3$ bn by 2023
    3. Cloud infrastructure spending surpassed 80$ bn in 2018
    4. About a third of companies' IT budget goes for cloud services
    5. The world is going to the cloud
        1. If someone is not there yet, he, very likely, will be on the cloud
2. Why Microsoft Azure?
    1. The 2nd largest public cloud (could be no 1)
    2. The highest growth rate
    3. The most regions (most data centers in most geographies)
    4. The most popular in enterprise
    5. Many Azure jobs
3. Note
    1. You don't have to know anything about cloud
    2. You don't have to know anything about Azure
    3. We're going to learn everything from the start
    4. We're going to have a lot of Azure hands-on
4. Example: ReadIt
    1. Bookshop with many features
    2. Starting architecture:
        1. Two VMs
            1. Weather API
            2. Catalog API
    3. End of the course
        1. A lot of services
5. By the end of the course:
    1. You'll know what is the cloud
    2. You'll know what is Azure
    3. You'll know what you can do with it
    4. You'll learn how to use its services
    5. You'll learn how to design modern, secure, reliable apps
    6. You'll become a Cloud Architect
6. The course is different
    1. Very practical
    2. Extremely hands-on
    3. Emphasis on what you'll actually use
7. Downlaod the Azure Architecture Summary!
    1. The ultimate, practical, guide to help you design great cloud architecture!
        1. Condensed version

Join the Cloud and Software Architects Community

Who Is This Course For

1. Who is this course for?
    1. Architects and Architects to be (knowledge about cloud is important)
    2. Developers who want to learn about the cloud
    3. Anyone who is interested in the most important computing platform in the world

How This Course Is Organized

1. Three main parts:
    1. Overview of the cloud and Azure
        1. Basic concepts
    2. Overview of main services (demos + hands on)
        1. Learn and use
        2. Build ReadIt app
    3. Architecting modern apps on Azure
        1. Using services

Agenda

1. Welcome section
2. Introduction to the Cloud
3. Introduction to Azure
    1. What is it?
    2. Regions
    3. Why is it special?
4. First Look at Azure
    1. Portal
5. Basic Concepts
6. Introducing Our App
    1. Features
    2. Functionalities
7. Compute
8. Networking

9. Data
10. Messaging
11. Azure Active Directory
12. Monitoring
13. Security
14. DR
    1. Disaster Recovery
15. Cost Management
16. Azure Policy
    1. Deep dive
17. Architecting Apps for Azure
18. Migrating to the Cloud
19. Advanced Services
20. Conclusion

Section 2: Introduction to the Cloud

Current Status in Computing

1. Before the cloud...
    1. If you needed a server, you had to:
        1. Buy it
        2. Install (OS, Softwares)
        3. Maintain it
        4. Replace it
        5. Have an IT team
    2. The same goes with:
        1. Networking
        2. Databases
        3. User Management
        4. ...
    3. Example:
        1. November - Black Friday sales
            1. CPU utilization to 120% (crashes website)
        2. If we increase resources:
            1. Utilization is low throughout the year except for during the Black Friday
                1. Waste of Money for unused power
        3. Cloud solves this problem

What is the Cloud?

1. The Cloud:
    1. Compute, Networking, Storage and other services managed by someone else
2. Cloud Providers:
    1. Companies who build huge data centers
    2. Fill it with servers, networking, cooling, electricity etc.
    3. Design and install various services
    4. Make it publicly accessible
3. Cloud Services
    1. Clouds are huge and the competition is fierce
    2. Offer a lot of additional services:
        1. AI
        2. IoT
        3. Kubernetes
        4. ...
    3. 100s of services
4. In the cloud era
    1. If you need a server, you can:
        1. Construct it in the cloud within minutes
        2. Use it as you wish
        3. Pay for what you use
        4. Shut it down when not needed
        5. Automatically maintained, patched, secured, monitored by the cloud provider

Characteristics of the Cloud

1. 5 Characteristics of Cloud Computing
    1. On-Demand Self Service
    2. Broad Network Access
    3. Resource Pooling
    4. Rapid Elasticity
    5. Measured Service
2. On-Demand Self Service
    1. No human interaction is needed for resource provisioning
        1. If I need a VM, I don't have to send an email or talk to someone
            1. Resource can be provisioned (created) with a click a button
    2. Provisioning is available 24/7
3. Broad Network Access
    1. Resources can be accessed from anywhere using the network
    2. Ideally high broadband
    3. No physical access is required at any time
        1. No access to physical server in the datacenter is required
            1. Forbidden as well
4. Resource Pooling
    1. Physical resources are shared between customers

        1. If there is a host machine and VMs are created on it, other customers might use the same host machine for their VMs
            1. We do not have any access to the other customers
            2. We do not have any access to the physical machines (no knowledge of exact machine used)
        2. Some advanced cloud services allow for physical resource separation (for security reasons)
            1. We can request the creation of VM in a separate physical host
                1. They do not allow us to specific which host to use
                2. More expensive
    5. Rapid Elasticity
        1. Resources can be scaled up and down as needed, automatically
        2. No need to purchase resources for a one-time peak scenario
            1. Motivation for on-premise orgs to move to the cloud
    6. Measured Service
        1. Payment is done only for resources actually used
            1. If I use two VMs, I pay only for the 2 VMs
            2. If I shut them down, I stop paying for them
        2. Server time/ DB storage / Function calls etc.
        3. Measured usually done in high-resolution
            1. Server time by the second
        4. No need to invest money in non-used resources

CapEx vs OpEx

1. To understand financial aspects of using cloud
    1. CapEx: Capital Expense
        1. Making upfront investment for future use/ profit
        2. Non optimal
        3. Not flexible
    2. OpEx: Operating Expense
        1. Pay for what you actually use
        2. **Extremely flexible**
        3. **Most optimal**
2. Traditional IT - CapEx Oriented
    1. Major investment for:
        1. Building data center
        2. Purchasing servers
        3. Purchasing air conditioning
        4. Purchasing network devices
        5. Purchase software license (DB etc)
    2. Only after doing all the above, we can use it
    3. There's also OpEx involved:
        1. Electricity
        2. Salaries
        3. Maintenance
        4. ...
3. **Goal is make make IT OpEx**
    1. This makes it financially efficient

IaaS, PaaS, SaaS

1. IaaS
    1. Infrastructure as a Service
    2. The cloud provides the underlying platform
        1. Compute
        2. Networking
        3. Storage
    3. The client handles and is responsible for the rest
    4. Most common example:
        1. Virtual Machines
            1. The cloud provides the host machine, networking and disks
            2. The client instantiates the virtual (guest) machine, installs software on it, patches it, maintains it etc...
                1. **Client must ensure that the VM is up and running**
2. PaaS
    1. Platform as a Service
    2. The cloud provides platform for running apps
    3. Including: Compute, networking, storage, runtime environment, scaling, redundancy, security, updates, patching, maintenance etc...
    4. The client just needs to bring the code to run
        1. Cloud takes care of the rest
    5. Examples:
        1. Web Apps
            1. The cloud provides the runtime for running web apps
            2. The client uploads the code, and it just runs
            3. The client has no access to the underlying VMs
                1. **Cloud is responsible to ensure that the app is up and running**
3. SaaS
    1. Software as a Service
    2. A software running completely in the cloud
    3. The user doesn't need to install anything on-premises or on his machine
    4. The provider of the software takes care of updates, patches, redundancy, scalability etc...
        1. **We can only use the software**
        2. We cannot mess with the underlying infrastructure of it
    5. Examples:
        1. Office 365
        2. Salesforce
4. Level of Control:
    1. On-premises
        1. Applications

        2. Data
        3. Runtime
        4. Middleware
        5. OS
        6. Virtualization
        7. Servers
        8. Storage
        9. Networking
    2. Infrastructure (IaaS)
        1. Applications
        2. Data
        3. Runtime
        4. Middleware
        5. OS
        6. Virtualization (cloud)
        7. Servers (cloud)
        8. Storage (cloud)
        9. Networking (cloud)
    3. Platform (PaaS)
        1. Applications
        2. Data
        3. Runtime (cloud)
        4. Middleware (cloud)
        5. OS (cloud)
        6. Virtualization (cloud)
        7. Servers (cloud)
        8. Storage (cloud)
        9. Networking (cloud)
    4. Software (SaaS)
        1. Applications (cloud)
        2. Data (cloud)
        3. Runtime (cloud)
        4. Middleware (cloud)
        5. OS (cloud)
        6. Virtualization (cloud)
        7. Servers (cloud)
        8. Storage (cloud)
        9. Networking (cloud)
5. Additional Service Types
    1. FaaS - Functions as a Service
    2. DBaaS - Database as a Service
    3. DaaS - Desktop as a Service
    4. IOTaaS - IOT as a Service
    5. AIaaS - AI as a Service

Types of Clouds

1. Types of clouds:
    1. Public
    2. Private
    3. Hybrid
2. Public Cloud
    1. The cloud is set up in the public network
    2. Managed by large companies
        1. Microsoft
        2. Amazon
        3. Google
        4. ...
    3. Accessible through the internet
        1. No need for specialized internal network
    4. Available to all clients and users
        1. Of any organization
    5. Clients have no access to underlying infrastructure
        1. We can use resources built on the public cloud
    6. Examples:
        1. AWS
        2. Azure
        3. Google Cloud
        4. IBM Cloud
        5. Oracle cloud
        6. Ali Cloud
        7. ...
3. Private Cloud
    1. A cloud set up in an organization's premisis
    2. Managed by the organization's IT team
    3. Accessible only in the organization's network
    4. Available to users from the organization
        1. External users who are not members of the organization do not have access to this cloud
    5. Users private cloud infrastructure and engines
    6. Contains a subset of the public cloud's capabilities
    7. Mainly for security reasons
        1. The organization wants resources and data to be stored inside the organization's premises
    8. Examples:
        1. VMWare Cloud
        2. Red Hat Openshift Container Platform
        3. **Azure Stack**

4. Hybrid Cloud
    1. A cloud set up in an organization's premises...
    2. ...but also connected to the pulic cloud
    3. Workload can be separated between the two clouds
    4. i.e. Sensitive data in the organization's premises, public data in the public cloud
        1. Example: Usernames, passwords and credit card information can be stored in the organization's premisis but professional profiles, example, linkedin
    5. Usually managed by the public cloud, but not always
        1. It could be the other-way around
            1. The private cloud is used to manage resources of a public cloud
    6. Examples:
        1. **Azure Arc**
        2. **AWS Outposts**
            1. The above extend public cloud into the organization's premisis and manage and control the private cloud from the public cloud

Main Cloud Providers

1. Cloud Providers:
    1. Companies which build datacenters and provide public cloud services
        1. IaaS, PaaS, SaaS
        2. Other services
2. Main Cloud Providers:
    1. Amazon Web Services
    2. Microsoft Azure
    3. Google Cloud Platform (GCP)
3. Cloud Providers Growth
    1. AWS - 29%
    2. Azure - 47% (fastest)
    3. GCP - 43%

Section 3: Introduction to Azure

Introduction

1. Introduction:
    1. Microsoft's public cloud
    2. Announded in October 2008
    3. Released in February 2010
    4. The 2nd largest public cloud
    5. Closing the gap...
2. First focused on PaaS services
    1. To counter AWS's IaaS focus
        1. Focused on VMs and weak on the PaaS front
    2. Later added IaaS
    3. Currently offers the largest variety of cloud services
    4. Major clients of Azure
        1. Boeing
        2. Ebay
        3. Samsung
        4. BMW

Regions and Zones

1. Regions:
    1. Microsoft built a lot of datacenters for Azure
    2. Each datacenters' location is called **Region**
    3. There are ~60 Azure Regions (more than any other cloud)
    4. **Almost every new resource in the cloud should be allocated to a region**
    5. Look at the regions by accessing the azure link
2. Zones:
    1. Some of the regions have more than one physical datacenter
        1. Great for availability in case one datacenter fails
    2. Each datacenter is called Zone
    3. **When there are more than one datacenter in a region, the region is said to have Availability Zones**
    4. Some cloud services benefit from Availability Zones
        1. They can skip from one zone to another and can do it transparently
3. Paired Regions
    1. Some regions have designated pair region
        1. For increased availability
            1. When a full region fails (along with it's availability zones) - the other one can fill its place
    2. Relevant for some of the cloud services (not many)
    3. **Pairs are set by Azure and cannot be changed**
        1. Replicated data goes to paired region

Azure Services

1. Everything that can be done in the cloud is called Cloud Service
    1. ie. Instantiating VMs, building databases, set up networks, use AI algorithms, using central user management etc...
2. Go to: https://azure.microsoft.com/en-us/services/
    1. List of azure services
        1. Long list (100s)

Defining Account

1. Free account:

1. Credit of $200
    1. We can setup notification when we reach the limit
2. azure.microsoft.com
    1. Free Account
    2. Start Free
    3. A microsoft account
        1. Enter code
    4. Enter phone and credit card
    5. Credit card details (no charge)
    6. Agree to terms and sign up

Section 4: First Look at Azure

The Azure Portal

1. Click **Build in the portal**
2. Portal:
    1. One-stop-shop
        1. Instatiating and removing resources
        2. See status of resources
        3. ...
3. burger button:
    1. List of resource categories
4. Click on top right corner account
5. Settings button: (top right)
    1. Theme
    2. Contrast
6. Notifications button (top right)
    1. We can remove notifications
7. Directry & Subscription button
8. Azure Cloud Shell
    1. CLI
9. Search bar:
    1. Virtual
        1. Services
            1. Available services that match the search term
        2. Marketplace
        3. Documentation
        4. Resources
            1. What resources were created that confirmed to the search term
10. Close Quickstart
    1. Real homepage

Accessing the Portal in the Future

1. portal.azure.com

Account and Subscription

1. Differences:
    1. Subscription
        1. Logical Container
            1. Contains the various resources you provision in the cloud (VMs, DBs, networks etc...)
        2. We can have multiple subscriptions
            1. One got created when we signed up
        3. Container for resources
        4. Can be attached to a lot of accounts
            1. Team can independently do CRUD on resources
    2. Account
        1. Identity
            1. An identity with access to resources in the subscription (ie. you)
                1. The created account is automatically attached to the subscription created automatically
        2. Can be attached to a lot of subscriptions
            1. Example: A cloud architect can have multiple clients with each client having its own subscription
                1. Client can attach my account to their subscription
                    1. **We need to login to a subscription that we want to work with at the moment**

Changes in Azure Portal

Creation of Resource

1. Navigate > Subscriptions
    1. List of subscriptions we have access to
        1. Default subscription exists
2. First resource:
    1. Resource group:
        1. Steps:
            1. Search bar: resource groups
            2. Click on **Resource Groups** (free)
            3. Click **Add**
            4. **Basics**
                1. Select **Subscription** (need to attach resource to a subscription)
                2. Resource group name
                3. Resource details:
                    1. Region: West Europe (resources need to be created in a specific region)

5. Review & Create
    1. Summary
6. Click **Create**

2. Notification:
    1. Resource group was created
        1. Can click on **Go to resource group**
        2. Important to see the notification area especially for long time taking resources

Finding the Resource

1. Search bar:
    1. Click
        1. Recent resources:
            1. my-rg
        2. type **my-rg**
            1. Under **Resource Groups**

The Resource Page

1. Resource screen:
    1. name
    2. Type
    3. Menu
        1. Overview
            1. Location
            2. Subscription Id (unique)
            3. Subscription name
    4. Groups:
        1. Settings
        2. Cost Management
        3. Automation
        4. Support - Troubleshooting
        5. Monitoring
    5. Activity Log:
        1. See what happened to the resource
        2. Access control (IAM)
            1. Who can access this resource

Removing a Resource

Azure CLI & PowerShell

TASK: Remove the Resource Group

Section 5: Azure Basic Concepts

Regions

1. Selecting Regions:
    1. Almost every resource in Azure should be placed in a Region
        1. VMs
        2. Resource groups
    2. How to select region?
        1. Factors that affect:
            1. Geographical proximity to system's audience
                1. Closest to the audience we target with the system
                    1. If audience is in US, pick one in US
            2. Service's availability
                1. Not all services are available in all regions
                    1. Check: https://azure.microsoft.com/en-us/global-infrastructure/services/
                        1. Premium plan of azure functions is not available in CANADA east
                        2. Consumption plan is not available in CANADA east
            3. Availability Zones:
                1. A region might have more than **one physical data center**
                    1. If one fails, another will take over
            4. Pricing:
                1. There is some difference in pricing based on region
                    1. Example: Norway West: $ 192 /month
                    2. Example: West US: $ 152.57 /month

Resource Groups

1. What is a resource group?
    1. A logical container for resources
    2. Used for grouping resources by a logical boundary
    3. Free
    4. Examples:
        1. Development/Test/Production resources
        2. Team A resources
2. How to identify?
    1. **resource group: production-rg**
        1. Every resource in this resource group belongs to production environment
3. What is the difference between resource group and subscription?
    1. Hierarchy:

1. Management groups (It is a place to manage subscriptions - if multiple subscriptions exist in an organization)
    1. Subscriptions (They have associated accounts & cost center)
        1. Resource groups (logical resources container)
            1. Resources
2. Account is attached to a Subscription (account level container)
    1. Cost is per Subscription
3. Resource groups naming conventions:
    1. It's best practice to have an "rg" or "RG" as part of the resource group name
    2. Could be a prefix or suffix:
        1. RG-Project-Dev
        2. Finance-Resources-rg
4. Resource groups:
    1. Almost every resource in Azure is placed in a Resource Group
        1. Example: VM is created in a Resource group
5. Search for virtual machines:
    1. Add
    2. Virtual machine
    3. subscription:
        1. Resource group:
            1. One can be created
6. Resource Groups help us in logically grouping resources.

Storage Accounts

1. What is it?
    1. Used to store almost anything in Azure
    2. Used transparently by various services (to store things that user is not even aware of)
        1. Example:
            1. Database backups
            2. VM disks
            3. Diagnostic data
    3. Used also for explicit data storage
        1. Later
    4. Quite cheap
        1. **1000s of GBs storage for only few dollars**

SLA

1. Service Level Agreement
    1. The uptime % of a cloud service
        1. 95% - 18d 6h 17m 27s
        2. 99.99% - 52m 35s
2. When provisioning new services, always check the SLA of the service used
    1. Example: App Service - 99.95% of the time or No SLA (dependening on the tier)
    2. Example: Azure SQL Database
        1. Premium Tier with Zone Redundant Deployments - 99.995% (second best in Azure)
        2. Not Premium Tier or Business Critical - 99.99%
        3. Hyperscale tier with one replica - 99.95% and 99.9% for zero replicas
3. **The actual SLA of the system needs to be calculated**
4. **SLA Calculation**
    1. To get the actual system SLA, multiply the SLAs of the participating services
        1. Example:
            1. App Service SLA = 99.95%
            2. Azure SQL SLA = 99.99%
            3. Actual SLA = 99.95 x 99.99 = 99.94% = 5h 15m 34s annual downtime

SLA Calculators

1. https://uptime.is
2. Azure Composite SLA Estimator: https://slaestimator.aztoso.com/

Cost

1. Almost everything in the cloud costs money
    1. Exception: Resource groups, ...
2. Few pricing models:
    1. Per resource (i.e. VM)
    2. Per consumption (i.e. Function Apps)
        1. We are not paying for creation of function app but for using the function app
            1. When a call is made to a function app, we pay a very small amount
    3. Reservations
        1. We pay upfront for the resource with a commitment of using it for a certain amount of time
        2. Example:
            1. We might commit to use a VM for 3 years
                1. We pay upfront for 3 years and get a **hefty discount for the price**
3. **Rule of Thumb:**
    1. **Always check resource's cost before provisioning**
    2. **If it is expensive, check for more cost-effective alternatives**
        1. Example: **Look for reservations when available and relevant** (depending on needs)
4. To find out how much the resource might cost:
    1. https://azure.microsoft.com/en-us/pricing/calculator
        1. List of services:
            1. Click resources
                1. Virtual machines
                2. Click the view
                3. Select Region (West Europe)

4. Operating System (Windows)
5. Type (OS Only (no other pre-installed software))
6. Tier (Standard)
7. Instance type (D2v3)
8. Number of hours/month (730)

2. Cost ($154.76)
    1. Storage - assumed to be used extensively
3. Selecting reservation ($105.63)
    1. Linux - $38.47

2. Parameters to tune:
    1. OS (Linux vs Windows)
    2. Instance (smallest to largest)
    3. Reservation (Pay as you go, 1 year reserved (~ 36% discount), 3 years reserved (~ 56% discount))
3. Example:
    1. Azure SQL
        1. Pick Region
        2. Single DB
        3. Instance: 2 vCores (go to DTU to decrease even further)
        4. Reservation

Setting Budget

1. Search: cost Management
    1. Select: Cost Management + Billing (Central hub for managing anything that is cost related in Azure)
    2. Click on: Cost Management
    3. Select: Budgets
        1. Helps us define budget (if we are getting close to it, we get a notification)
            1. Name: close-to-200
            2. Annual
            3. Amount: 200
            4. Next
                1. % of budget: 90 (180)
                2. Alert recipient (email)
                    1. email-id
                3. Select language

Architects and the Cloud

1. Software Architects designing regular system need to know:
    1. Non-Functional Requirements
    2. Technology Stack
    3. Component's architecture
    4. Communication Patterns
2. Cloud-based systems require, in addition:
    1. Infrastructure knowledge
        1. Networking design
        2. Firewall design
        3. Application gateway design
        4. Load Balancer design
        5. ...
    2. Security (aspects relevant to cloud & Infrastructure)
        1. A great architect must be fluent in securing application as an integral part of its role
    3. Hands-on (with cloud)
        1. **Architect must be the most experienced and knowledgeable in the cloud**
3. We will learn all the above

Section 6: Introducing Our App

Introduction

1. ReadIt!
    1. Your bookshop on the cloud (simulates real-life bookshop)
    2. Contains 4 main services:
        1. Books Catalog
        2. Shopping Cart
        3. Inventory Management
        4. Order Engine
    3. Technical details:
        1. Developed using .NET Core & NodeJS
            1. You don't have to be familiar with these platforms!
        2. Uses VS Code
        3. Uses various database (relational, nosql, object-store)
    4. What we'll do?
        1. Begin with the basic app, run it locally
            1. Ensure everything is working fine
        2. Move it to the cloud, demonstrating various deployment (compute) options
        3. Add networking and Security
        4. Add database support
        5. Add messaging
        6. Discuss various alternatives for deployments etc.
        7. Keep an eye on the cost...

ReadIt Demo

1. Add books to shopping cart

2. Click on place Order
3. Sign-in
    1. Inventory

Installing .NET SDK

1. .Net SDK
    1. open: https://dotnet.microsoft.com/en-us/download/dotnet
        1. Download .Net 6.0 (recommended)
        2. Download SDK (not runtime)
        3. Install
    2. Click Windows button:

        1. cmd

```
dotnet [enter]
dotnet --version
```

Installing VS Code

1. Search: download vs code
    1. Windows
    2. Install with default options
2. Open VS Code
    1. Right click and pin to taskbar

Installing Extensions

1. Search for:
    1. c#
        1. By Microsoft
    2. Azure Account
        1. Handles all Authentication aspects with Azure
            1. By Microsoft
    3. Azure App Service
        1. Handles app-services
2. New icon on the left:
    1. Click the icon

Troubleshooting the Local Catalog

1. If the following error occurs:

```
Unable to find fallback package folder 'C:\Program Files (x86)\Microsoft\Xamarin\NuGet\
```

    1. Fix:
        1. Remove `nuget.config` from `%appdata%\NuGet` folder
        2. Restart VS Code
        3. Run the project

Running the Catalog Locally

1. Download from https://www.udemy.com/course/microsoft-azure-from-zero-to-hero-the-complete-guide/learn/lecture/24314170#overview
2. New folder:
    1. readit
    2. Paste `catalog` folder under
    3. Go to folder bar: Type cmd
    4. `code .`
3. Click F5
4. We are using in-memory database.

Going Through the Code

1. Index.cshtml.cs
    1. OnGet(): Automatically run when page is first displayed
        1. Gets books and puts them in a list
            1. It connects to a database
                1. Later DB Connection
                2. Later Redis
    2. OnPostAddToShoppingCart()
    3. appsettings.json
        1. Config data for website
            1. "BooksDB" - contains connection string to DB
            2. "ConnectionString" - contains connection string to Redis
        2. Startup.cs
            1. useInMemory = true;

Section 7: Azure Compute

Introduction

1. Azure Compute
    1. Used the most probably
2. Compute:

1. It is a set of cloud services for hosting and running applications
2. It allows us to upload code and then run it
3. Offers various levels of control and flexibility
    1. On underlying infrastructure
3. Types of cloud services:
    1. IaaS: Cloud manages the underlying infrastructure but user controls OS, Middleware, Runtime, Data, and Applications
    2. PaaS: Cloud manages OS, Middleware & Runtime as well. User controls Application & Data
    3. SaaS: Cloud manages all
4. Compute: IaaS & PaaS
    1. Someimes IaaS is best fit and sometimes PaaS
    2. 4 Types of Compute services:
        1. Virtual Machines
        2. App Services
        3. AKS
        4. Azure functions

Virtual Machines

1. VMs:
    1. A virtual (=not real) server running on a physical (=real) Server
    2. Allows creation of new servers extremely quick
    3. It is based on existing resources of the physical Server
        1. We are asking a physical server to construct a VM that will use its resources
    4. From the user's point of view - a regular server, nothing new
        1. **We work with it as though it is a physical server**
            1. Connect using RDP or SSL protocols
            2. We can install software on it
            3. We can upgrade it.
            4. ... (whatever is possible with regular server)
    5. It is called an **Unmanaged Service**
        1. **Azure doesn't manage what is happening inside the VM**
            1. **It is our responsibility**
                1. If we cause the machine to go down due to abuse, then Azure doesn't care
2. Architecture of a VM:
    1. Host OS: OS of the physical Server
    2. Hypervisor: Software that manages VMs on the physical server.
        1. Server installed in the racks in physical servers
    3. Guest OS: Any OS (virtual)
    4. Bins/Libs: Installed in Guest OS (virtual)
    5. Code: Application code (virtual)
        1. Virtual: based on physical resources.
    6. We can run multiples VMs on a single physical machine
        1. **VM density**: number of VMs per host
3. IaaS: Host OS & Hypervisor are completely managed by Azure.
    1. We don't have any control or access to those layers
        1. Azure knows how to allocate resources in the most effective way to save power and cost
    2. Internals of VM: Managed by us, full control & full access
        1. We can do whatever we want
            1. **We have to take responsibility and ensure that everything is working fine in the VM**
4. Steps for instantiating a VM in Azure:
    1. Select a location
    2. Select an image (OS + Pre-Installed software)
        1. Usually we install software later
    3. Select the size (CPU, Memory)
    4. That's it (basically)
        1. **Don't forget to check the price!**
            1. Select the most efficient and cost effective VM

Creation of Virtual Machine

1. Recently Azure has put restriction on spending limit
    1. It might prevent new VMs from being created.
        1. To get rid of the eror, follow the steps given below:
            1. https://learn.microsoft.com/en-us/azure/cost-management-billing/manage/spending-limit#remove-the-spending-limit-in-azure-portal
2. Check the price of a VM in calculator
    1. Steps:
        1. Click Virtual Machines
        2. Click on View
            1. Location: West Europe (pick the one closest)
            2. OS: Windows
            3. Type: OS only
            4. Tier: Standard (adequate performance)
            5. Instance: D2 v3: 2 CPUs, 8 GB RAM, 50 GB Temporary Storage
                1. Upto 64 Cores, 1792 GB RAM, 2048 GB Temporary storage
            6. Pay as you go: We are going to delete in few minutes
            7. OS: License included
                1. Azure Hybrid Benefit: Allows moving Windows license from on-premises to the cloud (don't choose this now)
        3. Search VMs
            1. click: Create VMs
                1. Subscription: Choose one
                2. Resource group: New
                    1. first-vm-rg
                3. VM Name: first-vm
                4. Region: West Europe
                5. OS: Windows Server 2019
                6. Size:

                1. Search for D2s (cheaper than D2 V3)
            7. Username: pick one
            8. Password: pick one
                1. Look at restrictions:
                    1. 12-123 chars long
                2. Confirm the Password
            9. Public inbound ports: Allow selected ports
            10. Select inbound ports: RDP (3389)
                1. RDP: Used to connect to VM usig RDP
            11. Click: Review+Create
            12. Click: Create
        2. Azure begins a **deployment task** (for deploying new resource)
            1. Sends it to Azure Control
                1. It accepts the task and begins the deployment process.
            2. Resource created:
                1. 5 created:
                    1. VM
                    2. Network interface
                    3. Network security group
                    4. IP Address (IP used to connect to VM)
                    5. Virtual Network (network the machine resides in)
                2. Other resources not shown:
                    1. Disk
                3. Click: Go to resource
                    1. Overview: ...
        3. Click **Connect**
            1. RDP
                1. Leave defaults
                2. Click: **Download RDP File**
                3. Click: connect
                4. Enter credentials
                    1. Username: .<username>
                    2. Password:
        4. Server manager:
            1. Used to manage Microsoft servers (part of Windows Server OS)
            2. Click on Windows button:
                1. List of applications
            3. Close it.
            4. Stop it
                1. Payments might incur for storage and ip address
                    1. **Remove the whole resource-group** (to delete all resources even those we are not aware of)
                        1. **first-vm-rg**

The Real Cost of VM

1. Includes:
    1. Cost of VM includes:
        1. VM
        2. Disk
        3. IP (depends on the type)
            1. Not all IPs have Cost
        4. Storage (involved even if we are unaware of them)
            1. Extremely low
    2. Example:
        1. VM - D2v3 - 154.76
        2. Disk - P10 - 21.68
        3. Public IP - Dynamic - 2.92
        4. Storage - LRS (same zone backup) - < 1
            1. ~ $180

Reducing the Cost of VM

1. Techniques:

    1. Most effective techniques to reduce costs of VM:
        1. Auto Shutdown
        2. Reserved Instances
        3. Spot Instances
        4. Disk Optimization

2. Auto Shutdown:

    1. As simple as it sounds...
    2. Automatically shuts down the machine when not needed
        1. Relevant mainly for test / dev machines
    3. Storage and IP (if static) costs still incurred
    4. **Can save > 50% of VM cost**
    5. How to implement:

        1. During creation:

            ```
            Auto-shutdown
            Enable auto-shutdown   - On/Off
            ```

3. Reserved Instances

    1. Allows upfront payment with substantial discount

        1. We are paying up-front and not each month
    2. Usually offered for 1 or 3 years
        1. Great for production machine which run continuously
    3. Offers great discounts (upto 62% of the list price)
    4. Can be divided into monthly payments
    5. **Cannot be stopped / refunded**
        1. **Unless..** - roumer
    6. Azure calculator:
        1. 1 year reserved (~36% discount)
        2. 3 year reserved (~56% discount)

4. Spot Instances:

    1. Machines that run on unused capacity in Azure
        1. Not all servers in the data center are in use all the time
            1. We can use the un-used servers in the data center for a reduced price
                1. **Can be evicted any moment when needed by Azure**
                    1. VM can be shutdown or even deleted (a short notification is sent)
    2. Offers up to 90% discount, price fluctuates according to demand
        1. If demand goes up, price goes up
    3. Great for non-critical, non-continous tasks
        1. i.e. Batch processes, long running calculations (which can stop and continue later)
    4. Portal:
        1. Azure Spot Instance:
            1. Eviction type: (when will the Azure evict or shutdown)
                1. Capacity only
                    1. When Azure needs the machine, it evicts or deletes
                2. Price or capacity
                    1. We can set a maximum limit for the price. If spot instance's price (based on demand) reaches that limit, Azure shuts the machine down.
            2. Eviction policy:
                1. Stop/Deallocte
                2. Delete

5. Disk Optimization

    1. Make sure to select the right disk for the machine
    2. Default is Premium SSD - the most expensive option
        1. Premium SSD is very expensive
            1. It is fastest
            2. It offers better SLA
    3. **Non IO-intensive machines can do with Standard SSD**
        1. Little slower
        2. If I/O is not very important
        3. Difference in SLA is not very substantial
    4. Example: App servers, in-memory cache can do with standard SSD
    5. Note: Disk type affects the SLA
    6. Portal:
        1. Disks:
            1. OS disk type: Premium SSD

6. More Cost Saving Techniques:

    1. Select the right size for your machine
        1. CPU shouldn't rest, you pay for it
            1. It must not be at 10% utilization. It must be at 70% - 80% utilization
    2. Select Linux over Windows when possible
        1. Linux is cheaper becase there is no Windows license involved in pricing
    3. Check price in nearby regions

Availability of VMs

1. How to make sure that the VM that I am working with doesn't go down suddenly and leave me with nothing to work with?
    1. This requires creation of other VMs that will work together
        1. **How to effectively distribute VM instances across Azure**
    2. SLA for VMs:
        1. https://azure.microsoft.com/en-us/support/legal/sla/virtual-machines/v1_9/
            1. If 2 or more VMs are deployed in 2 or more Availability Zones in the same region, the SLA is at-least 99.99% of the time
                1. ~52m 35s / year
            2. If 2 or more VMs are deployed in same Availability Set or in same Dedicated Host Group then SLA is at least 99.95% of the time
                1. ~4h 22m 44s
            3. Single instance of VM:
                1. Premium SSD: 99.9% (~8h 45m 56s)
                2. Standard SSD: 99.5% (~1d 19h 49m 44s)
                3. Standard HDD: 95% (18d 6h 17m 27s) (**not recommended**)
2. Availability Concepts in Azure:
    1. Fault Domain
    2. Update Domain
    3. Availability Set
    4. Availability Zone
3. Fault Domain:
    1. Logical group of physical hardware that share a common power source and network switch
        1. Similar to rack in a traditional data center
            1. A cabinet with single power source and network source
                1. It can host multiple servers
                2. If there's a problem with the power or networking in the domain (=rack) - all servers in it shut down
                    1. Solution: **You want to make sure your servers are spread across more than one fault domain (=rack)**

    2. A rack is a fault domain.
        1. If a rack goes down, all the servers in it go down.
            1. **Ensure that VMs are not located in the same fault domain**
4. Update Domain:
    1. Logical group of physical hardware that **can undergo maintenance and be rebooted at the same time**
        1. Azure is in control of updating and rebooting the underlying hardware of the VMs
            1. Ensures that the host machines are always upto date with hot fixes needed.
                1. Host machine can get rebooted
                    1. => **VMs also reboot**
        2. **Maintenance is done by Azure at its own discretion**
            1. **We don't have control over when and where this is going to happen**
                1. If all servers are in the same update domain - they'll reboot at the same time during maintenance
                    1. **You want to make sure your servers are spread across more than one update domain** (?)
                        1. Hosts across update domain might be spread across multiple fault domains
5. Availability Set:

    1. It is a collection of fault domains and update domains your VMs will be spread across

    2. Can contain up to 3 Fault Domains and up to 20 Update Domains

    3. All domains (Fault & Update) are in the same Zone (=datacenter)

    4. Example:

        1. New availability set:

            1. Fault domains: 2
            2. Update domains: 3

        Fault Domain #1 Fault Domain #2
        [ ] [ ] - Update Domain #1
        [ ] [ ] - Update Domain #2
        [ ] [ ] - Update Domain #3

    5. VM deployments:

        1. VM #1 - Fault Domain #1 & Update Domain #2
        2. VM #2 - Fault Domain #2 & Update Domain #1
            1. Say Update Domain #2 reboots
                1. VM #1 reboots but VM #2 still exists
            2. Say Fault Domain #1 fails
                1. VM #1 shuts down but VM #1 still exists
        3. **If we did not define any availability set, then Azure could have decided to place both VMs on same Fault Domain**
            1. **Azure has no idea that both VMs belong to the same application or same system**
                1. We will loose both if the Fault Domain fails
        4. **Solution**
            1. Deploy identical VMs into the **same Availability Set**
                1. Ensures they won't be shut down simultaneously when a single fault domain shuts down or an update domain reboots
            2. If needed - deploy load balancer to route between the VMs
            3. **Availability Set is free**, you pay only for the additional VMs
6. Availability Zone:
    1. **If we want to use instead of Availability Set**
    2. **It is a physically separate zone within an Azure region**
        1. Technically - a building containing an autonomous data center (A zone is basically a building or data-center)
    3. Each zone functions as a fault & update domain
        1. Fault & Update domain level resolution is not required because we deploy VMs in separate buildings/data-centers
    4. **Provides protection against a complete zone shutdown**
        1. **Availability set is in the same zone/building/data-center**
        2. Hence the better SLA (as compared to Availability Set)
    5. Implementation:
        1. Deploy identical VMs into separate Availability Zones in the same region
            1. Ensures they won't be shut down simultaneously when the zone shuts down
                1. **If needed - deploy load balancer to route between VMs**
                    1. **Load balancer will know if any VM is not active and hence will not route the traffic to that VM**
    6. **Availability Zone is free, you pay only for the additional VMs**
    7. **Rule of thumb: Use Availability Zone if you can or else use Availability Set**

Creation of Available & Cost Effective VM

1. Search for Virtual Machines:
    1. Add > Virtual Machine
        1. New RG: optimized-vm-rg
        2. Name: optimized-vm
        3. Region: West Europe
        4. Availability options:
            1. Availability zone
            2. Availability set (select this)
        5. Click: Create new:
            1. Name: my-av-set
            2. Fault domains: 2
            3. Update domains: 3
        6. If Availability Zone: West Europe
            1. Pick number: 1, 2, or 3
                1. We can pick 2 or 3 for another VM
            2. Note: Some regions do not have availability zones
        7. OS: Windows
        8. Enter credentials

9. Next: disks
    1. OS disk type: Premium SSD (we can change later)
10. Next: Management
    1. Auto-shutdown:
        1. Enable
        2. Shutdown time: 7:00:00 PM UTC
        3. Notification before shutdown: uncheck
11. Review + create
12. Click: Download a template for automation
    1. ARM template

ARM Template

1. Azure Resource Manager Template
    1. A JSON file describing the resource(s) to be created
    2. Used by Azure in (almost) all deployments
        1. Azure first generates ARM template and then passes it on to the deployment engine
    3. It can be exported, modified, uploaded, deployed
    4. Can also be created from scratch
2. ARM Template is a declarative way of deploying resources
    1. Declarative:
        1. Describes the end result
            1. What are the resources & how do they look like
        2. Allow "What-If" operation
            1. We can simulate the deployment process and figure out what the actual resources are going to get created when we actually run the template
        3. Can deploy multiple resources at once
            1. A different file is not required for each resource
        4. Can be integrated in CI/CD processes
            1. ARM template can be used in CI/CD process
        5. Can be source controlled
            1. ARM template file can be stored in versioned in source control
        6. Used by: ARM template
    2. Imperative:
        1. Sends instructions to run (tells what to do)
        2. Error prone
            1. Easy to make mistakes
        3. Can't be verified
            1. Difficult to look at instructions and stop mistakes
                1. We come to know only after sending
        4. Can't be source controlled
            1. It is not a file
        5. Suited for quick and dirty operations
            1. Okay to quickly provision a VM
            2. Not suitable for full Azure environment
        6. Used by: Azure CLI, PowerShell (Although they can run ARM template too)
            1. **Preferred**

Using ARM Template

1. Template:
    1. Left-side has Hierarchy
        1. Parameters
        2. Variables
        3. Resources
            1. Actual resources that are going to be created
                1. Schedules - Used for auto-shutdown
        4. Click on VM resource:
            1. Type: Microsoft.Compute/virtualMachine
            2. Parameter: parameters{'virtualMachineName'} (key)
            3. location: paramters{'location'}
            4. vmsize: parameters{'virtualMachineSize'}
            5. storageAccountType: parameters{'osDiskType'}
        5. Parameters:
            1. osDiskType: "Premium_LRS"
    2. Click Download
        1. parameters.json
        2. template.json
    3. Copy the files to **template** folder
        1. `code .`
            1. parameters.json
                1. Update Premium_LRS to StandardSSD_LRS
                2. Update admin password
        2. Close vs-code
    4. Open portal:
        1. Search for resource groups
            1. cloudshell-storage-westeurope (could be slightly different)
                1. Automatically created when we domoed cloud Shell (storage account for cloud shell)
                2. Open it:
                    1. Click the storage account
                    2. Click File shares
                    3. Click the file share
                    4. Click **Add directory**
                        1. Name: **templates**
                        2. Open the directory

1. Click **Upload**
2. Select both files
3. Click **Open**
4. Click **Upload**

2. Click cloud shell button:
    1. Click **Bash**
    2. Run Azure CLI
3. Search for resource groups:
    1. Add:
        1. name: optimized-vm-rg
        2. Region: West Europe
        3. Go to resource
4. In cloud shell:

```
cd cloudDrive
cd templates
dir
az deployment group create --resource-group optimized-vm-rg --template-file template.json --parameters parameters.json
```

    1. Hit refresh in portal
    2. Go to disks tab:
        1. Standard SSD

Deleting Virtual Machines

1. Click on Auto-shutdown (on the left side panel)
2. Go to Overview:
    1. Click **Delete**
        1. Why option is given to retain resources such as OS disk, Network interfaces, and Public IP addresses?
            1. OS disk can be connected to a different VM
            2. Network interfaces & Public IP addresses can be re-used
    2. Virtual Network/Subnet - optimized-vm-rg-vnet/default (created automatically)
        1. It contains more resources (not just VM) so Azure wants to make sure deleting it by mistake will make other resources in-accessible
    3. Check all the resources & delete
3. Open the resource group name:
    1. It contains virtual network & network security group
        1. Delete the resource group

Virtual Machine Scale Set

1. A group of separate VMs sharing the same image
    1. VMs are managed as a group
        1. Can be scaled out or in manually or according to predefined conditions
            1. Great for handling unpredictable load
    2. We tell the scale-set to monitor the machines
        1. When scale-set detects a heavy load applied on the machines, it can add additional VMs so that load can be distributed more effectively across VMs
        2. When scale-set detects lower load, it can scale the VMs down by dropping the machines not needed anymore
    3. Once a scale-set is setup, the machines should NOT be modified
        1. Should not change files, install apps etc...
            1. Since new machines created by the scale set will be based on the original image (changes will be lost if we install in the VM
                1. solution: New image and modify the base image of the scale set
    4. For web-apps, a load balancer should be put in front of the scale set
        1. Load-balancer connects to scale-set
        2. It knows how many instances exist in the scale-set
        3. It performs actual routing to the actual instances
        4. It exposes a single address which the end-user is exposed to
2. Scale Set Pricing:
    1. Scale Set is free (mechanism is free)
    2. You pay for the VMs deployed in it

Using Scale Set

1. Search for scale-set
    1. Add:
        1. Resource group: New one
            1. vmss-rg
        2. Name: my-vmss
        3. Region: West Europe
        4. OS: Windows
        5. credentials
        6. click: Create
    2. Image:
        1. We might need an image with whatever software we need
2. Status:
    1. 1 out of 2 succeeded
        1. 2nd one did not start yet
3. Click: instances
    1. Actual VMs
4. Scaling:
    1. How we will be scaling:
        1. Manual scale

2. Custom Autoscale
    1. We can define a policy: To define when will scale-set add new instances
    2. Conditions:
        1. Instance limits:
            1. Minimum: 1
            2. Maximum: 2
            3. Default: 1
        2. Click **Add rule**
            1. Scale rule window:
                1. Average
                2. CPU
                3. Greater than: 80%
                4. For 10 minutes (average)
                5. Add 1 more VM
                6. Cool down:
                    1. Scale-set will wait 5 minutes before checking if another scaling operation is required (avoids too quickly adding or removing instances)
                7. Other rules:
                    1. Network In Total
                    2. Network Out Total
                    3. Disk Activity
                    4. ...
            2. Add
        3. Adds plain English like rule
        4. Click **Add rule**
            1. Scale out window:
                1. Average
                2. CPU
                3. Less than: 30%
                4. For 10 minutes (average)
                5. Remove 1 VM (Decrease count by)
                    1. Decrease count by: 1 (5 -> 4)
                    2. Decrease count to: 1 (5 -> 1)
    3. Scale-in-accessible Policy tab:
        1. Policy:
            1. Default: Balance the instances across availability zones and fault domain & delete the instance with the highest instance id
                1. Higher the id, later the instance was created
                    1. Youngest is removed first
                        1. However, if we remove the latest one results in all instances being in the same availability Zone, then scale-set will not remove the specific instance
            2. Newest VM: Delete the newest of the created VMs
            3. Oldest VM: Delete the oldest of the created VMs
2. Click Save:

    1. Error:

        1. The capability is not registered in subscription

            1. Solution:
                1. Open portal in another page:
                2. Search subscription & select it
                3. Go to: **Resource providers**
                    1. Type: insight
                    2. Select **microsoft.insights**
                    3. Click **Register**

        2. Open cloud shell:

            ```
            az provider show --namespace microsoft.insights -o table
            ```

    2. Hit Save in previous window

        1. Azure deletes the second instance (because of the rule)
5. Delete resource group

Azure Instance Metadata Services

1. A little known feature of Azure VMs (not used much)
    1. A REST API accessible from the VM
        1. Providing a lot of info about the machine
            1. Info includes:
                1. SKU
                2. Storage
                3. Networking
                4. Scheduled events
                5. ...
        2. Accessible ONLY from the VM
            1. We cannot go to the address and access this info from outside the VM
        3. **VM can query Azure for various capabilities of the VM itself**
2. With Scaleset:
    1. Get notification about upcoming eviction
        1. If we are listening to the metadata service, we can know that an eviction is in place and in few seconds the VM is going to be deleted
            1. Helps in running some final actions as part of the eviction
        2. **Can be polled every ~1 min to get enough time to close things up**
3. New VM:
    1. Click Create
        1. Resource group: mds-rg

2. VM name: mds-demo-vm
3. Region: West Europe
4. Disable Availability Zone
5. Windows VM
6. Give credentials
7. Click **Create**
2. Go to the VM page:
3. Connect
    1. Download RDP file
    2. Connect
    3. Give credentials
4. Go to **Local Server**
    1. Click **IE Enhanced Security Configuration** on
        1. Select **off** on both
            1. IE will not block when we access various sites
5. Open Explorer
    1. Download postman
    2. Click **Donwload the App**
    3. Run
    4. Open postman
        1. Login with account
        2. Click +
            1. Headers:
                1. key: Metadata
                2. value: true
            2. URL: http://169.254.169.254/metadata/instance?api-version=2020-06-01
                1. 169.254.169.254 - address of metadata service of Azure
            3. Data:
                1. SKU - 2019 datacenter
                2. OS Disk - 127 GB
                3. Private IP
                4. Public IP
                5. ...
            4. URL: http://169.254.169.254/metadata/scheduledevents?api-version=2019-08-01
                1. It might take time for the creation of **event-container** for the VM
                    1. Subsequent calls will be quicker
                2. **When the VM is going to be deleted either because it is part of a scale-set or as part of maintenance or due to failure of the rack, then the above API will return a list of events that will inform us that the VM is going to be rebooted (we have enough time to take action)**
6. Delete the resource group to delete all resources created

ReadIt Resources Helper List

1. Compute services used for the Readit App:
    1. VM
    2. App Services
    3. Azure Kubernetes
    4. Functions
2. Networking services used for the Readit App:
    1. VNets
    2. NSGs
    3. App Gateway
3. Data services used for the Readit App:
    1. Azure SQL
    2. Cosmos DB
    3. Storage Account
    4. Redis Cache
4. ...
5. Resource List:
    1. Addresses and connections strings
        1. https://www.udemy.com/course/microsoft-azure-from-zero-to-hero-the-complete-guide/learn/lecture/24411614#overview

Setting Up the Catalog App

1. Open VS Code and open catalog folder:
    1. Prepare the code for publishing in a VM:
        1. Open terminal in VS Code
            1. Run the following:

```
.net publish -o publish # prepares the .net code and puts the result in a folder named publish
```

            2. Open publish folder:
                1. Many files exist
2. Open portal:
    1. Search for Virtual Machine
    2. Add VM:
        1. Resource group: readit-app-rg
        2. Name: catalog-vm
        3. Region: West Europe
        4. Windows Server 2019
        5. Click on See all sizes:
            1. Select B2ms
                1. Weaker machines with a capability of bursting (powerful for a limited amount of time)
                2. Price is substantially lower

3. Similar config as D2v3 (default offered by Azure)
    1. 2 CPUs, 8 GB RAM
6. Give credentials
7. Go to Disks:
    1. Standard SSD
8. Networking:
    1. By default Azure offers a network based on the name of the resource group to which `-vnet` is appended.
    2. Click: Create new:
        1. Remove -rg
            1. readit-app-vnet
    3. Public IP:
        1. Dynamic by default: Everytime we restart the VM, we get a new IP
            1. Make it Static (Public IP will stay the same even if we restart the VM)
                1. Click: **Create new**
                    1. Assignment: Static
9. Management:
    1. Auto-shutdown:
        1. Enable auto-shutdown
    2. Notification before shutdown: Uncheck
10. Review+Create
11. Click Create
12. Go to the resource Page
    1. Connect:
        1. Give credentials
    2. Server Manager:
        1. Local Server
            1. IE Enhanced Security Configuration
                1. Click on On
                2. Click on Off for both
        2. Dashboard (Add capabilities to Server to add web-apps)
            1. Add roles and features
            2. Next
            3. Next
            4. Next
            5. Web Server
                1. Add features
            6. Next
            7. Features:
                1. Everything we need pre-selected
            8. Next
            9. Next
            10. Under Role Services:
                1. Check Customer Logging
                2. Check Logging Tools
            11. Next
            12. Install
    3. Open IE:
        1. Open: localhost
            1. Shows a home-page that shows that a web-server is installed on the machine
    4. Install Google Chrome or any modern browser
    5. Install .Net core
        1. download .Net
        2. Click first link
        3. Download .NET:
            1. .NET 6.0 hosting bundle (integrated into IIS allowing it to run .Net core apps)
                1. Click: All .NET 6.0 downloads
                2. Click: Hosting Bundle
                3. Accept the license agreement and install
    6. Close browser and server manager
13. Open Windows Explorer
    1. Go to C:
        1. Right click: New folder - catalog
14. Click Top right restore icon to have access to Desktop
15. Copying files:
    1. Open **publish**
        1. ctrl + c
        2. Go to VM and catalog folder
        3. ctrl + v
16. Tell to web-server that the folder is a web-app and must be accessed by web-server
    1. Search for IIS in wondows
        1. Open IIS
            1. Click the little arrow besides catalog vm
            2. Right click on sites
                1. Add website
                    1. Site name: catalog
                    2. Physical path: point to catalog files filder
                    3. Binding:
                        1. Port: 8080
                    4. OK
            3. Click Browse link on the right hand side:
                1. Go with Google Chrome (make it a default browser)
17. The catalog App is accessible from inside the VM
    1. Memorize the IP address of the machine

        1. From portal, copy the public IP address and open the following in browser:

`http://<ip>:8080`

1. Doesn't open
    1. Resolution: Inside the VM
        1. Search Firewall
        2. Open Firewall
            1. Turn off
            2. OK
    2. Try again
        1. It didn't help
            1. Azure Networking feature blocks (only RDP is allowed)

Setting Up the Weather API

1. If we click on Weather
    1. Weather Server IP: If we enter the IP of the weather server, we can get the weather around the shop
        1. Visitors can decide what to wear
2. Minimize the VM window.
3. New VM:
    1. Search Virtual Machine
    2. Click Create
        1. Select the same Resource Group
        2. Name: weather-vm
        3. Region: Same
        4. Image: Ubuntu (Linux)
        5. Size: Smaller than previous
            1. B1s - $8.76 / month
                1. Strong image is not required
        6. Authentication type: Password (easier to remember)
        7. Credentials
        8. Disks:
            1. Standard SSD
        9. Networking:
            1. readit-app-vnet (same as catalog VM)
            2. IP: Static IP
        10. Management:
            1. Auto-shutdown Enable
            2. Notification: uncheck
        11. Review + Create
        12. Create
4. Tool to connect to Linux Machine:
    1. Download putty
    2. Select release compatible with OS
    3. Install
5. Copy IP address from Portal
6. Open Putty tool
    1. Hostname:
    2. Click open
    3. Click Yes
    4. Enter username and password
7. Install a weather app in the VM to return random weather:
    1. `sudo apt install git`
    2. `sudo apt update`
    3. `sudo apt install nodejs -y`
    4. `sudo git clone https://github.com/memilavi/weatherAPI.git`
    5. `cd weatherAPI`
        1. `ls`
            1. `index.js`
    6. Install packages:
        1. `sudo apt install npm -y`
        2. `cd node_modules`
            1. `ls`
        3. `cd ..`
    7. Start the app:
        1. `npm start` - listens on port 8080
    8. Note the private IP of the weather VM
8. Open catalog virtual machine
    1. Our Weather: :8080

Virtual Machines Tips and Tricks

1. If more disk is required in addition to default one:
    1. Go to **Disks** page
    2. Add disk
        1. Account for disk costs by checking in calculator
2. To backup VM to restore in case of failure?
    1. Go to **Backup** page
    2. Define **frequency of backup** & **retention period**
3. To define DNS name for the VM:
    1. Click DNS name
    2. Configure link in the overview page

A Quick Reminder...

1. Note IP + Port of Catalog app & Weather API

Azure Architecture Diagrams

1. When designing architecture for Azure apps, it's a good idea to use Azure symbols in the diagram
    1. There are hundres of them...
        1. Download Azure Icons:
            1. https://docs.microsoft.com/en-us/azure/architecture/icons
                1. Download SVG Icons
                2. Extract
            2. Icons are divided by resource Types
                1. Compute
                    1. VM icon - open in a browser that supports SVG (Chrome say)
                2. Scaleset icon
                3. Subscription icon
                4. Resource group icon
    2. Architecture for ReadIt App:
        1. Two VMs
            1. VM: Weather API
            2. VM: Catalog App

A Word of Caution

1. VM: Catalog App:
    1. Directly accessible from the internet
    2. Can be RDPed from anywhere
        1. Anyone who knows the public IP can RDP to it
            1. Solution: **Never leave a VM open to the internet this way**
                1. **What to be done will be learnt later**

Shut Down the Machines

1. Shut down Catalog VM and Weather VM

App Services

1. What is it?
    1. A fully managed web hosting for websites
        1. Allows publishing code - and it just runs
            1. VM creation is not required
            2. Maintenance of the VM is not required
                1. upgrade
                2. patching
                3. hot-fixes
            3. Hence no access to the underlying servers
                1. Cannot install or maintain OS
            4. Secured and compliant
                1. Microsoft is responsible for security and compliance of the underlying infrastructure & ensures it is up-to date
            5. **Integrates with many source controls and DevOps engines:**
                1. GitHub
                    1. We can upload a new version of our app and App Service will automatically pull it and deploy it
                2. BitBucket
                3. Azure DevOps
                4. DockerHub
                5. ...
            6. Supports following platforms:
                1. .NET
                2. .NET Core
                3. Node.JS
                4. Java
                5. Python
                6. PHP
                7. Containers
                    1. If we package our service using Docker and upload it, it will just run (no complex setup or long config)
    2. App Types supported:
        1. Web Apps
        2. Web API
        3. Web Jobs (Batch processes)
            1. Long running processes (large amount of data say)
2. Extremely easy to deploy:
    1. Develop your app
    2. Construct Web App (can be done from the IDE)
    3. Publish your code (usually with one click)
    4. Viola!

App Services Tiers

1. Tiers:
    1. Standard - Usually
        1. **Right combination of performance and scale for most web apps**
    2. Free - to play Around
        1. **Extremely limited storage space**
        2. **Uses shared servers**
    3. Basic:
        1. **Quite limited storage space & compute power**
        2. **Dedicated servers**
    4. Isolated:

1. **Completely isolated environment for app Services**
    1. **How? Its own virtual network**
5. Price is affected by the OS we select
    1. Linux app services are much cheaper than Windows
6. Price displayed is lowest but goes up with CPU and RAM
7. Full details: https://azure.microsoft.com/en-us/pricing/details/app-service/linux/

Auto Scaling App Service

1. App Service can be autoscaled to support spikes in load
    1. We can start with single instance.
    2. When the load of website suddenly increases, the autoscaling feature can automatically add instances to app services
    3. When everything goes back to normal, instances will be removed
2. Auto scale is based on **verious metrics**
3. It is extremely flexible (?)
4. Configuration is similar to VM Scaleset
    1. Example: Scale out rules: CpuPercentage or HttpQueueLength (if there are many requests in the queue that cannot be served when the server is busy)
        1. Instances are added when the condition is met

Setting the Inventory App Service

1. Download the zip file and copy its contents from folder
    1. `readit/inventory/<folder>`
    2. `code .`
    3. F5
        1. Inventory is up an running
2. portal
    1. Search: `app services`
    2. **Create**
        1. Basics:
            1. Subscription: Prev subscription
            2. Resource group: readit-app-rg
            3. Name: readit-inventory-mutthoju (inventory.azurewebsites.net exists so pick a unique name across internet)
            4. Publish: Code (Not Docker image)
            5. Runtime stack: .NET 6 (LTS)
            6. Operating System: Windows
            7. Region: West Europe
            8. App Service Plan:
                1. Windows Plan (West Europe): Free F1 (only free tier with free account)
                    1. 1 GB of memory
        2. Review + create
        3. Create
        4. Go to the resource
    3. readit-inventory:
        1. Click URL
            1. Contains a placeholder
    4. Go to VS Code:
        1. Stop app execution
        2. Go to terminal
            1. `dotnet publish -o publish`
        3. Right click on **publish** folder
            1. **Deploy to Web App..** (because of extension in VS Code)
            2. Click **Sign in**
            3. Enter login details of subscription
            4. Close the page
            5. Select **readit-inventory-mutthoju**
            6. Click **Deploy**
        4. Open output window: Switch to **Azure App Service**
            1. Click **Browse Website**
            2. Click **Open** for external website
            3. DNS name **readit-inventory-mutthoju.azurewebsites.net** is used to open the app
    5. In Portal:
        1. Go to **Development Tools/App Service Editor (Preview)**
            1. **Go** (it is a tool to edit various files on app-service itself)
                1. Click on file and we can modify
        2. Go to **Console** (gives a commandline interface into the machine where the app-service is installed)
        1 `dir` - list of files installed in the app-service
            2. It is a sandbox
                1. We cannot install Software
                2. We cannot format the machine
                3. We cannot delete what we want
        3. App Service plan
            1. Tier we selected for app service
                1. Free tier: 60 minutes of use per day
        4. Scale up (App Service plan)
            1. Dev/Test:
                1. D1 - more minutes / day
                    1. has Cost (< $ 10 / month)
                2. B1 - more expensive
            2. Production
                1. More tiers to select from
                2. Additional pricing tiers
                    1. S3 - strongest standard tier
            3. Isolated
                1. Not generally available
                2. Depends on many factors

5. Scale out
    1. Resource pricing tier does not allow auto-scaling (F1)
    2. Switch a different plan and go back to scale-out (it is identical to the scaleset)

App Services Tips and Tricks

1. App services can be accessed using http and https
    1. We can make it https in the TLS/SSL settings in the App Service menu
2. App service can run batch processes, or continuous jobs (not only web-apps with request/response paradigm)
    1. Go to **WebJobs** menu item
    2. Upload exe file that will run always or on scheduled times
3. To know IP address of the app-service
    1. Go to properties Page
    2. Find Virtual IP address of the App Service
    3. Find Outbound IP addresses
        1. More than one
4. To know the amount of storage
    1. Go to Quotas for the data

Shutting Down App Service

1. We can stop the App Service (using the Stop button at the top of the Overview page)
    1. It will stop the functionality of the App Service, but we will still pay for it
        1. Difference between App Service and VM
            1. We pay only when the VM is on
            2. We need to completely delete it to stop paying

Current Architecture

1. Architecture:
    1. App Service: Inventory app
    2. VM: Weather API
    3. VM: Catalog app

AKS

1. Azure Kubernetes Services
    1. Managed Kubernetes on Azure
        1. Allows deploying containers and managing them using Kubernetes on Azure
    2. Paying only on the instances (=VMs) used (no payment for AKS)
        1. We only can select instance size and VMs are implicitly managed

Containers

1. Traditional Deployment:
    1. Code was copied and built on the production server
        1. Sometimes it was pre-built and copied to the production server
            1. The piece of software that was run on the production server was not the same as the one that was used to build the software.
                1. Result: Problems were found on the servers that weren't found in the dev machines.
                    1. Resulted in wasting a lot of time and money to figure out what went wrong and **what the differences were between the two machines**.
                2. Solution: Containers
2. Containers:
    1. A thin packaging model
        1. Packages software, its dependencies, and configuration files
        2. Can be copied between machines
    2. The packaging is an atomic unit that can be executed using whatever software or files contained within it.
        1. It is completely independent of the machine that it is hosted in.
            1. Exception: **Container uses the underlying operating system**
3. Container vs VM:
    1. Differences:

        1. VM:

        ```
        Virtual Machine
            App A
            Guest OS (can run Windows or Linux)
        Hypervisor (responsible for running VMs, responsible for giving access to host resources (network, disk, ...))
        Infrastructure
        ```

        2. containers

        ```
        App A  App B   App C   App D
                Container Runtime (Similar to hypervisor, it runs containers, it gives access to host resources)
                Host OS
                Infrastructure
        ```

        1. Extremely light weight as compared with VM
            1. Containers share OS of the host
                1. If host runs Windows 2016, it is the OS of the containers
                2. If host OS is upgraded, containers will run on the upgraded OS
                    1. **Can be advantage or serious drawback**
            2. VM has its own OS
4. Why Containers?
    1. Predictability (more important for developer and architect)
        1. The same package is deployed from the dev machine to the test to production

1. If it runs on the developer's machine, it runs on the production machine.
2. Performance
    1. Container goes up in seconds vs minutes taken by VM
        1. Because we don't have to boot guest OS
3. Density
    1. One server can run thousands of containers vs dozens of VMs
        1. Containers are much lighter than VMs
            1. They require much less memory

5. Why not containers?
    1. Isolation
        1. **Containers share the same OS, so isolation is lighter than VM** (?)
            1. **It is easier to cross boundaries between containers than between VMs** (?)
                1. **If application contains sensitive code or data, it should probably be deployed in an isolated VM**

Introduction to Docker

1. Docker:
    1. The most popular container environment
    2. De-facto standard for containers
    3. Released in 2013
2. Docker Architecture:
    1. Client
        1.
    2. Docker Server
        1. Docker daemon
            1. Runs on the server
            2. It is responsible for managing the docker containers
                1. Turns them on or off
                2. Keep track of activity
                3. Builds containers based on images
                4. Exposes API for management tools
            3. It is the heart of docker environment and without it nothing will work.
        2. Images
            1. It is a set of definitions and software for a container to run
            2. Building blocks of the containers
            3. They are static files and they don't run
                1. They just lye on the disk and wait to be called.
    3. Registry
        1. A collection of images from where we can pull the image we want and either use it as is, or customize it to your needs.
            1. We can pull an image for baseline .Net application and customize it by copying into it our specific code components.
                1. We will get a new custom image as a basis for the container.
        2. Can be public
            1. DockerHub
                1. Used for baseline images
        3. Can be Private
            1. Accessible only for me or the organization
    4. Containers
        1. Image built and run (instances of images)
        2. It runs in a sandbox created and managed by the docker daemon and is a living breathing instance of an image
        3. It is a full blown process and it can do whatever it wants assuming it has permissions.
    5. client
        1. CLI
        2. Used to send instructions to the daemon
        3. Gateway to functionality of docker
            1. Pull Images
            2. Cusomize the Images
            3. Run cotainers
            4. List containers
            5. Shut them off
            6. Check the status
        4. **DevOps/IT guys spend most of the time with this**
3. dockerfile

    1. Customization is done using this file

        1. Contains instructions for building custom images
        2. It can instruct the Docker daemon to copy files, run commands, change working directory, etc...
        3. Usually quite small
            1. Baseline images exist and customization is not much

    2. Example:

```
WORKDIR /opt/node_app
COPY package.json package-lock.json* ./
RUN npm install --no-optional && npm cache clean --force
ENV PATH /opt/node_app/node_modules/.bin:$PATH
WORKDIR /opt/node_app/app
COPY . .
```

        1. Most files look similar to this
4. Support for Docker
    1. It is the most popular in the world and hence support in various platforms
        1. Supported by all major OSs (Windows, Linux, OSx)
        2. Supported by major cloud providers
            1. Amazon ECR
                1. Elastic Container Registry
            2. Azure ACR

        1. Azure Container Registry
    3. **We can develop in our machine, build an image and push it to the cloud and it will run**

Containers Management

1. Containers are a great deployment mechanism
    1. They are widely supported and predictable
    2. Gained popularity
    3. What happens when there are too many of them?
        1. Frontend - 2
        2. Backend - 2
        3. Database - 1
        4. Batch processes - 1
    4. Problems with too many containers:
        1. Deployment
            1. It is tiresome to deploy manually
            2. It is error prone when we deploy manually
        2. Scalability
            1. If the containers are loaded, we need to be able to scale them (to add more containers to distribute load)
        3. Monitoring
            1. Doing it manually is in-feasible
        4. Routing
            1. If we have more than one instance of a container, we need a routing mechanism similar to the load balancer to route the requests to the container instances
        5. High-availability
            1. How to make sure our container-based system can deal with crashes and errors
    5. With VMs, the above problems are solved by host-managers and load balancers
        1. How to solve them for containers?
            1. Container management tools

Introduction to Kubernetes

1. Kubernetes
    1. The most popular container management platform
    2. De-facto standard for container management
    3. Provides all aspects of management
        1. routing
        2. Scaling
        3. High-availability
        4. Automated deployment
        5. Configuration management
        6. ...
2. Architecture:
    1. Pod: Atomic unit in k8s
        1. Container of containers
            1. Docker container lives inside it
            2. It is wrapper for containers
            3. **Pod provides connectivity and monitoring**
            4. There could be multiple docker containers in one pod
        2. Pod exposes IP Address
            1. k8s communicates with the container using the IP
        3. Containers are accessible to the public network using service
            1. service: mechanism used by k8s to expose functionality to the outside world
                1. It masks the inner Containers
                2. It also Provides
                    1. Load balancing
                    2. Monitoring
                    3. High availability
                    4. ...

A Note About Working with Azure Container Registry

1. We are going to construct and use Azure Container Registry
2. Changes made to ACR
    1. We need permissions
        1. Go to Access control (IAM)
        2. Click + Add -> Add role assignment
        3. Switch to the Privileged administrator roles tab
        4. Select the Contributor role and click Next
        5. Click + Select members and select your user from the list
        6. Click Select, Review + assign, Review + assign again
            1. After assignment is complete, go again to Repositories
                1. This time we won't see the error

Installing Docker on Windows 10/11 Home

1. Go to: https://docs.microsoft.com/en-us/cli/azure/install-azure-cli
    1. Pick OS (Windows)
    2. Latest release of the Azure CLI
    3. Default options
2. Open cmd
    1. `az`
3. Download shopping cart Code and unzip
    1. `cmd`
    2. `code .`
    3. Run

4. Docker:
    1. Go to VS code
    2. Extensions
    3. Search for "Docker"
        1. Select first (by Microsoft)
            1. Can be used manage containers, images, and registries
    4. Open Dockerfile
        1. .net 6.0 image
    5. We can build docker image in ACR
5. ACR:
    1. Go to portal
    2. Search for registries
    3. Select Container registries
    4. Hit "Create"
        1. Subscription
        2. Resource group: readit-app-rg
        3. Registry name: readitmutthoju (unique)
        4. Location: West Europe
        5. SKU: Basic
            1. Less storage & cheaper
            2. Click on icon
                1. Difference between basic, standard and premium
                    1. Basic: 10 GB
        6. Click **Review + create**
        7. Click **Create**
    5. Go to Resource
        1. Provisioning state: Succeeded
        2. Check location
    6. Go to **Access keys**
        1. Enable **Admin user**
            1. Enables VS Code to access ACR and push code into it

Working with Containers

1. VS Code:
    1. Go to Docker extension
        1. Under registries
            1. Select Subscription
                1. Select the container registry
    2. Go to explorer:
        1. Right click **Dockerfile**
            1. Select **Build Image in Azure...** (Build Image uses locally installed Docker)
            2. Name: cart:latest
            3. Subscription
            4. Registry: readitmutthoju
            5. OS: Linux
        2. If you see error: cb1 failed after 2s. Error: failed to download context. Please check if the uRL is incorrect. If it has credentials, please check if they are expired
            1. Go to terminal
            2. `az login`
            3. Repeat the build
        3. Portal:
            1. cart
                1. latest
                    1. manifest used by container registry to upload and store docker image

Working with AKS

1. Portal:
    1. Search for **AKS** > Select **Kubernetes services**
    2. Click **Create**
        1. Resource group: readit-app-rg
        2. Customer preset configuration: Dev/Test (minimal configuration and resources to save cost)
        3. Cluster name: cart-aks
        4. Region: West Europe
        5. No availability zone
        6. Kubernetes version: default
        7. API server availability: 99.5% (optimized for cost)
        8. Node size: Change size
            1. Insuffient quota
                1. The maximum amount of resources we can use under this subscription
                    1. VNet
                    2. vCPUs
                    3. ...
                2. Open Portal
                    1. Subscriptions
                        1. Select name:
                            1. Usage + quotas
                                1. Total Regional vCPUs
                                    1. Location: Select "East Europe" and uncheck others - vCPUs used is 0
        9. Change to "West US 2"
        10. Change size:
            1. D-Series v5 > D2ads_v5 (2 vCPUs, 8 GB Ram)
        11. Scale method: Manual
        12. Node count: 1
        13. Integrations

            1. Contianer registry: readitmutthoju
        14. **Review + Create**
        15. **Create**
            1. It might take 5 - 10 minutes
    3. Go to resource:
        1. Location: westus2
2. VS Code
    1. cli for aks
        1. `az aks install-cli` **(M)**
    2. Include cli in path variable
        1. `set PATH=%PATH%;"c:\users\azcourseadmin\.azure-kubectl"`
    3. Connect:
        1. `az login`
        2. `az aks get-credentials --resource-group readit-app-rg --name cart-aks`
            1. default options
            2. Merge with context
        3. `kubectl get nodes`
    4. `deployment.yaml` - tells how to deploy containers in k8s

```
containers:
    image: readitmutthoju.azureacr.io/cart:latest
```

        1. `kubectl apply -f deployment.yaml`
    5. Portal:
        1. Services and ingresses
            1. External IP
            2. Click the name:
                1. Click Pods
            3. Click External IP

Current Architecture

1. VMs
2. App Service
3. AKS (Cart App)
4. ACR (Cart Docker)

Azure Functions

1. What are those?
    1. Small, focused functions running as a result of an event
    2. Great for Event Driven systems
        1. If we want to do something as a result of something else that happened in the system
    3. Automatically managed by Azure
        1. Can start
        2. Can stop
        3. Can autoscale
            1. Frees developer from handing them
    4. Flexible pricing plans
    5. Serverless
2. Serverless:
    1. Cloud resource that is completely managed by the cloud
        1. Users do not need to think about:
            1. VMs
            2. CPUs
            3. Memory
            4. etc.
        2. Handling underlying infrastructure (VMs, AKS, ...) does not exist
        3. It just works
            1. However, it still works on servers
3. Example:

```
namespace AzureCourse.Function
{
    () references
    public static class EventGridFunctions {
        [FunctionName("EventGridFunction")]
        () references
        public static async Task Run(
            [HttpTrigger(AuthorizationLevel.Anonymous, "get", "post", Route = null)] HttpRequest req,
            [EventGrid(TopicEndpointUri = "MyEventGridTopicUriSetting", TopicKeySetting = "MyEventGridTopicKeySetting")] IAsyncCollector<EventG
            ILogger Log) {
            string name = req.Query["name"];

            var myEvent = new EventGridEvent(...);
            await outputEvents.AddAsync(myEvent);
        }
```

```
    }
}
```

1. It is a single function & it is short

4. Triggers and Bindings

```
Triggers
* The event that made the function run
* Quite a few [triggers]
* Deeply integrated into other Azure services
* Technically not mandatory, but...
    1. We need a good reason

Bindings
* Declarative connection to other resource(s)
    1. Using it we can setup a connection between azure function and another resource in the cloud
    2. Other resources can be:
        1. Input (read from)
        2. output (write to)
        3. both
    3. Provided as parameter to the function
    4. Makes connecting to other resources extremely easy
    5. Not mandatory
```

5. Example:

    1. Trigger: Http call

    ```
    [HttpTrigger(AuthorizationLevel.Anonymous, "get", "post", Route = null)] HttpRequest req
    ```

        1. Anonymous users (without authentication) can use the function
        2. Applicable for "get" and "post" verbs

    2. Binding: EventGrid

    ```
    [EventGrid(TopicEndpointUri = "MyEventGridTopicUriSetting", TopicKeySetting = "MyEventGridTopicKeySetting")]
    ```

        1. Result of the function will be written in EventGrid

6. Trigger Types:

    1. Blob Storage
    2. Cosmos DB
    3. Dapr
    4. Event Grid
    5. Event Hubs
    6. HTTP Requests
    7. IOT Hub
    8. Kafka
    9. Queue Storage
    10. RabbitMQ
    11. Service Bus
    12. Timer

7. Binding Types (Input or Output)

    1. Blob Storage
    2. Cosmos DB
    3. Dapr
    4. Event Grid
    5. Event Hubs
    6. HTTP Requests
    7. IOT Hub
    8. Kafka
    9. Mobile Apps
    10. Notification Hub
    11. Queue Storage
    12. RabbitMQ
    13. SendGrid
    14. Service Bus
    15. SignalR
    16. Table Storage

8. Triggers and Bindings Examples:

    1. Run every 5 minutes (Timer Trigger) and calculate the sum of a column in a DB. If it's above 115, send an event in EventGrid (Binding)
    2. When a message arrives in the Orders Queue (Queue Trigger) save it in Cosmos DB (Binding) for future handling
    3. Receive HTTP Request (HTTP Trigger) with 4 numbers, and return the smallest one of them (no binding)

9. Supported Languages

    1. c#
    2. JavaScript (NodeJS)
    3. Java
    4. Python
    5. PowerShell
    6. F#

10. Cold Start (issue)

1. Azure Functions are completely managed by Azure
    1. After some time of inactivity, Azure might take down the Function's host
        1. If compute power is not used, it is brought down
    2. The next activation of the function will take time (underlying host needs to be powered up)
        1. 2-3 seconds before the code runs
            1. Not a problem in most types of functions
                1. Especially Async timers and triggers
            2. A problem mainly for HTTP-Triggered functions (synchronous and calling party requires quick response)
    2. What actually happens:

```
When app is cold:

    Azure allocates unspecialized server
        |
        v
    Worker becomes specialized (prepared for function activation)
        Files mounted to worker
        App settings applied
        |
        v
    Functions runtime resets (reloads)
        Function.json files read
        Extensions loaded
        |
        v
    Functions loaded into memory
        |
        v
    Code runs

When app is warm:

    Code runs
```

11. Cold Start:
    1. How to avoid cold start?
        1. Select the right hosting plan

Azure Functions Hosting Plan

1. Plans:
    1. Consumption
    2. Premium
    3. Dedicated
2. Consumption plan:
    1. Pay only for what you actually use
        1. Function execution time
        2. GBs
            1. Limit of 1.5GB RAM
    2. Example:
        1. 0.20 per million executions
        2. 1 million executions
        3. Executions / month: 9m
        4. Avg. memory consumed / execution: 800 MB
        5. Avg. execution duration: 1.5s
        6. Total seconds: 9m * 1.5s = 13.5m secs
        7. Total GB / sec = 13.5m * 0.8 = 10.8m - 400k free grant = 10.4m GB-s
        8. Payment for execution time: 10.4m * 0.000016$ = 166.4$
        9. Payment for executions: 9m - 1m free grant = 8m * 0.2$ / m = 1.6$
        10. Total = $168 / month
    3. Downsides:
        1. 1.5 GB RAM limit
        2. Cold start
3. Premium Plan
    1. Pay for pre-warmed instances (hosts)
        1. vCPU duration - vCPU: ~$123.37 vCPU/month (fixed)
        2. Memory duration - Memory: ~$8.833 GB/month (fixed)
            1. We can select instance size and prices will change
                1. 1 vCPU, 3.5 GB RAM
                2. 2 vCPU, 7 GB RAM
                3. 4 vCPU, 14 GB RAM
    2. Pay for scale-out instances
        1. If we set up scale-out
    3. What you get:
        1. No cold starts
            1. Host instances are always warmed up
        2. No memory limit (up to host RAM)
            1. Limited to what is mentioned
        3. Better performance
        4. VNet integration
            1. Useful security feature
        5. Predictable price
            1. We know up-front
    4. Example:
        1. Prices:
            1. vCPU duration: vCPU: ~$123.37 vCPU/month
            2. Memory duration: Memory: ~$8.833 GB/month
        2. Usage:

1. 1 pre-warmed Instance
2. 2 vCPUs, 7GB RAM
3. No scale out
4. Cost:
    1. vCPU cost: 123.37 x 2 = 246.74$
    2. Memory cost: 8.833 x 7 = 61.83$
    3. Total: 308.57$
5. Downsides:
    1. More expensive

4. Dedicated Plan
    1. The functions run on an existing App Service
        1. Can only be used if we have an app service up and running somewhere else in the subscription
        2. Great if server is under-utilized
        3. No additional costs
    2. Make sure **Always On** setting is activated to avoid disabling functions
        1. Appears in the configuration page of the App Service
            1. Otherwise, the function will be disabled by the App Service when it will see that they are not used.
    3. Downsides:
        1. No Auto-Scale

Durable Functions

1. What are they?
    1. Stateful functions that interact with external resources and keep track of flow
    2. They offer very simple syntax, hide complexities of managing state, retries, etc.
    3. Example:
        1. Useful for Function Chaining - call various Functions sequentially, and apply the output of each function to the next one.

            ```
            F1 -> State -> F2 -> State -> F3 -> State -> F4
            ```

            1. Each function has its own state, its own function and runs in its own time.
    4. Managing function chains becomes simple with durable functions:

        ```
        [FunctionName("Chaining")]
        public static async Task<object> Run(
            [OrchestrationTrigger] IDurableOrchestrationContext context
        ) {
            try {
                var x = await context.CallActivityAsync<object>("F1", null);
                var y = await context.CallActivityAsync<object>("F2", x);
                var z = await context.CallActivityAsync<object>("F3", y);
                return await context.CallActivityAsync<object>("F4", z);
            } catch (Exception) {
                // Error handling or compensation goes here.
            }
        }
        ```

        1. Functions are called by name with parameters.
        2. Once a function is called, this function goes to sleep. Once the function returns, it activates itself and calls the next function.
        3. `IDurableOrchestrationContext` - tells that the function is a durable one.

Running Functions Locally

1. Browser:
    1. Setup & Installation
        1. https://docs.microsoft.com/en-us/azure/azure-functions/functions-run-local
            1. Click: **Install the Core Tools and dependencies**
                1. Click v3.x - Windows 64-bit
                    1. Runtime environment that simulates the execution of Azure functions in local machine
                        1. For testing and debugging locally
                2. Install
        2. Download zip file from Resources
            1. Extract it
                1. order folder:
                    1. Azure function that handles all the ordering aspects of the book-shop
            2. When user clicks the place-order button in the shopping cart, the function is activated and makes the order.
            3. cd into order folder
                1. cmd
                2. code .
                    1. Two functions (with Cosmos DB, Storage)
                        1. ProcessOrderStorage.cs
                            1. HTTPTrigger
                                1. Read request body
                            2. Blob storage binding
                                1. A new blob object is created and stored in a storage
                    2. Order.cs
                        1. Order class
                        2. OrderItem class
                    3. local.settings.json
                        1. Contains config data for function to work
                            1. Later a connection string to Cosmos DB or Storage
                    4. ordersample.json - sample order
                    5. Extension (helps working with functions, creation of functions, uploading functions, ...)

                1. azure functions
                    1. Select **Azure Functions** by Microsoft
            6. Click Azure icon
                1. Expland Azure subscription
                2. Workspace
                    1. Local Project
                        1. List of functions in the code
            7. Open terminal:
                1. `dotnet restore` - ensure we have all the packages
                2. F5
                    1. Two URLs
                        1. http://localhost:7071/api/ProcessOrderStorage
    4. Install PostMan
        1. Browser
        2. Download PostMan
        3. Install PostMan
            2. POST:
                1. Copy ordersample.json
                2. Body
                    1. Raw
                        1. Paste
                        2. JSON
                3. Click **Send**
        4. Terminal:
            1. Function is executed
                1. output is logged

Changing the Default Log Source in Azure Functions

1. Update:
    1. Default log will be streamed to App Insights (not displayed in log console)
        1. Open log console
        2. Click App Insights Logs
        3. Change it to Filesystem Logs
            1. Logs will be streamed to console

Running Functions on Azure

1. Portal
    1. Open cloud shell (for trial accounts, we cannot instantiate a consumption based function app using the portal)

        ```
        az storage account create --name readitfuncstoragemutthoju --location westeurope --resource-group readit-app-rg --sku Standard_LRS

        az functionapp create --name readitfunctionappmutthoju --storage readitfuncstoragemutthoju --consumption-plan-location westeurope --resour
        ```

        1. Search **function app**
            1. Click function app
                1. App Service Plan
                    1. Properties
                        1. Pricing tier: Consumption
                2. Functions
                    1. No functions yet
2. VS code
    1. Hit refresh under RESOURCES
        1. WORKSPACE
            1. Hit deploy icon
            2. Deploy to function App
            3. Select the function App
            4. Deploy
        2. Click **Upload Settings** in popup (useful for future)
3. Portal:
    1. Hit refresh
        1. 2 Functions
        2. ProcessOrderCosmosDB
            1. Code + Test
            2. Click **Logs** at the bottom
                1. We can see what function writes to log
        3. Click Get Function URL
        4. Paste in postman
            1. Send
                1. See that order json is printed in Logs in portal

Current Architecture

1. Function App - Order Processing

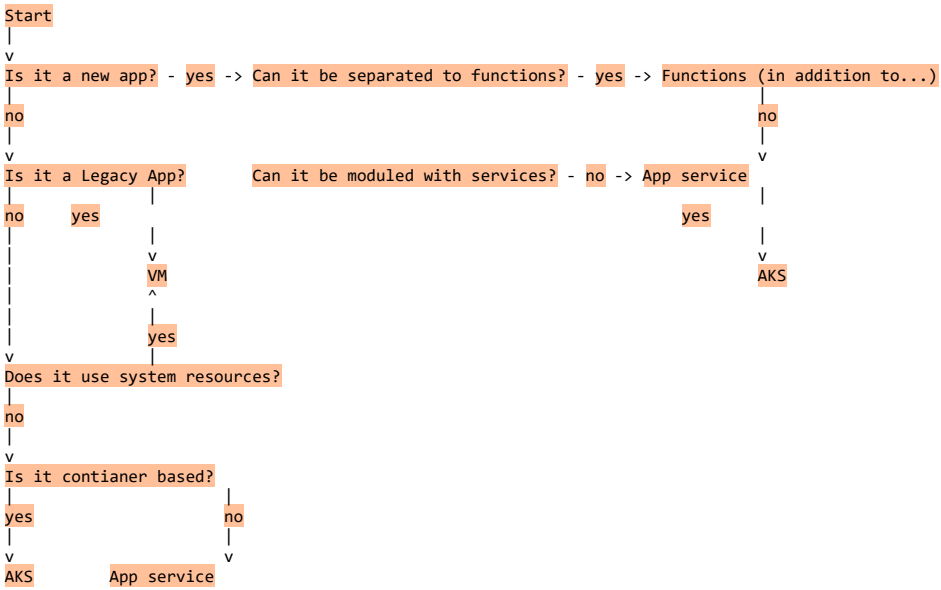
A Quick Reminder

1. Note down AKS Cart Service URL and Order Function URL in Resources Helper List

How to Choose Compute Type

1. There are 4
2. How to choose?

1. Use the flow chart



    1. Functions: small focused code segments, each performing specific task
        1. A whole system cannot be based on functions
    2. Moduled services: microservice based system
        1. No - monolith - App Service
        2. Yes - AKS
    3. Legacy app: outdated technologies
        1. VM - because managed platforms do not have support for legacy technologies
    4. System Resources (no managed solution allows access to system resources)
        1. Sockets
        2. Registry
        3. ...
    5. Container based: Does system already use containers?

More Compute Options

1. There are more:
    1. Logic Apps
        1. Workflow that integrates with other services
        2. Executes many logic steps
    2. ACI - Azure Container Instance
        1. Runs docker container without AKS
            1. For running single instance
    3. App Service Container - Deploy docker to App Service
        1. A container image can be deployed to App Service

Section 8: Azure Networking

Networking in Azure

1. Networking:
    1. All aspects of networking in Azure
    2. Deals with resources' network connections, firewalls, etc.
    3. Might sound boring and not so important
        1. However, networking is the foundation of cloud security
            1. It is the basis of security in every cloud
2. Cloud Architecture
    1. Never leave a VM open to the internet the following way:
        1. Directly accessible from the internet
        2. Can be RDPed from anywhere
    2. Two main threats:
        1. Brute force attacks on port 3389 (RDP)
            1. An attacker can bombard a port with a lot of requests (even millions) until they successfully guess the password for logging into the machine. They use the information to hack into the machine and break havoc.
        2. No line of defense in front of the VM web server
            1. When user is going to browse to this machine, there is no other appliance or protection in front of the VM
                1. We need a firewall or application gateway
3. Networking knowledge is what makes a good cloud architect - an amazing cloud architect.
    1. It is the important part of designing a secure system in the cloud.
4. Networking:
    1. 4 networking-related cloud services:
        1. VNets
        2. SubNets
        3. Load Balancer
        4. Application Gateway

Virtual Networks

1. VNets:

1. A network in which we can deploy cloud resources
    1. We can think of it as the organizational network where many organizational services are deployed.
2. Many cloud resources are deployed within VNets
    1. VMs
    2. App Services
    3. DBs
    4. etc.
3. There is a tab called **Networking**

2. Virtual
    1. It is based on physical network and is logically separated from other VNets
        1. Physical network: Network with cables, switches, and routers.
    2. We use existing networking resources as part of Azure and we define a logically separated network that we are going to use.
    3. **Resources in VNet can communicate with each other by default**
        1. Suppose we have 3 VMs in a VNet, they can communicate with each by default
        2. Suppose we have another VM in a different VNet, the first set of VMs cannot communicate with the VM in the second VNet and vica versa by default.
    4. It is a organization's private network
        1. In AWS, it's called VPC - Virtual Private Cloud
    5. VNet Pricing:
        1. VNets are free
        2. Limit of 50 VNets per subscription across all regions
3. Characteristics of VNet
    1. Scoped to a single Region
        1. Cannot span multiple regions
    2. Scoped to a single subscription
    3. Can be connected via Peering
        1. Allows resources in on VNet to communicate with resources in another VNet
    4. **Segmented using Subnets**
        1. A logical group of resources within a VNet
    5. Protected using NSG (defined on the Subnets)
        1. Network security group (not at VNet level)
4. Security and VNets
    1. The most important thing to think about when designing network:
        1. How to limit access to the resources in the VNet so that risk is minimized.
            1. While designing and architecting
                1. Security risk is as low as possible
5. Addresses of VNets
    1. Each VNet has its own address range
        1. Or IP Range
    2. By default - 65,656 addresses
        1. Can be customized
        2. All network devices must be in this address range
        3. Address range of VNet is expressed using CIDR Notation

CIDR Notation

1. What is it?
    1. Classless Inter-Domain Routing
    2. A method for representing an IP range
        1. Composed of an address in the range and a number between 0 and 32
            1. The number indicates the number of bits that are allocated to the address. The smaller the number - the larger the range
    3. Example:
        1. 109.186.149.240
            1. Each number is 8 bits - 8 binary digits of 0 or 1
                1. min - 0
                2. max - 255
        2. Range:
            1. 109.186.149.240/24
                1. 24 bits are allocated to the address
                    1. It leaves 8 bits allocated for the range
                        1. 109.186.149.0 - 109.186.149.255 - 256 addresses
    4. Example:
        1. 109.186.149.240/16
            1. 109.186.0.0 - 109.186.255.255 - 65,536 addresses (way to big probably)
    5. Example:
        1. 109.186.149.240/20
            1. 149 = 1001 0101 binary
                1. 1001 0000 to 1001 1111
                    1. 144 - 159
            2. Range: 109.186.144.0 - 109.186.159.255 - 4,096 addresses
    6. We don't have to remember this calculation:
        1. A lot of CIDR calculators exist
            1. https://www.ipaddressguide.com/cidr
    7. More good news:
        1. Azure usually shows the actual range (first address, last address, range)

CIDR Notation Tip

1. Another way to calculate:
    1. Size = 2^(32 - Range)
        1. 109.186.149.240/24 => 2^(32 - 24) = 2^8 = 256
        2. 109.186.149.240/20 => 2^(32 - 20) = 2^12 = 4096

Subnets

1. What is it?

1. A logical segment in the VNet
2. Shares a subset of the VNet's IP Range
3. Used as a logical group of resources in the VNet
4. It is a must.
    1. Resources must be placed in a subnet
        1. They cannot be placed directly in the VNet
5. When a VNet is created, a default subnet is created alongside
6. Resources in a subnet can talk to resources in other subnets in the same VNet
    1. By default
        1. Can be customized
            1. Example:
                1. Subnet - Frontend
                2. Subnet - Backend

2. Addresses of Subnets:
    1. Each subnet gets a share of the parent VNet's IP Range
        1. NEVER use the full range of the VNet in a Subnet
    2. Extremely hard to modify the range later
    3. Makes it hard to add future subnets
        1. If we take all the addresses for previous subnets
3. Subnet Pricing:
    1. Subnets are free
    2. Limit of 3,000 Subnets per VNet (thats a lot)

Looking at the ReadIt VNet

1. Portal
    1. Open VM in new tab
    2. Click on Catalog Machine
        1. Virtual Network/subnet - readit-app-vnet/default
    3. Click on Weather VM
        1. Virtual Network/subnet - readit-app-vnet/default
    4. Click on VNet/Subnet
        1. Address space: 10.0.0.0/24 (256)
        2. Virtual network cards (network interface)
        3. Private IP addresses in the range
    5. Click Subnets
        1. Single subnet: default
            1. Range: 10.0.0.0/24 - same as VNet
                1. Takes all the addresses
                    1. Another subnet cannot be created
    6. Click + Subnet
        1. Subnet address range:
            1. 10.0.0.0/28
                1. Error: Subnet address overlaps with another Subnet
            2. 10.1.0.0/20
                1. Address range is outside VNet address space
    7. Fix:
        1. Go to Address space
            1. Change to: 10.0.0.0/20
        2. Go to subnets
            1.
                - Subnet
            2. 10.0.1.0/26 - 64 addresses
                1. 59 + 5 reserved addresses
                    1. Azure needs the addresses

Creation of a Virtual Network

1. Search Virtual Network
    1. readit-app-vnet
    2. AKS VNet - required by AKS
    3.
        - Create
        1. Resource Group: vnet-test-rg
        2. Name: my-vnet
        3. Region: North Europe
        4. IP Addresses
            1. Default address space is offered
                1. Default: /24 - only 256
                    1. Previously, VNet was created as part of the VM creation and it was only for VM so all the addresses were part of the default Subnet
                    2. Here VNet is created explicitly
                        1. So we might need multiple subnets so subnet address space is small
                2. Click default
                    1. Name: front-subnet
                    2. Range: same
                    3. save
            2.
                - Add subnet
                1. Name: back-subnet
                2. Subnet address range:
                    1. 172.16.1.0/24
                3. Add
            3. Click **Review + create**
            4. Click **Create**
            5. Click **Subnets**

1. front-subnet
2. back-subnet

Working with VNets

1. Creation of two VMs and placement in two separate subnets in a single VNet
    1. Search for VM
        1. Add Virtual Machine
        2. Resource group: vnet-test-rg
        3. Virtual machine name: front-vm
        4. Size: B2S - 2 vcores and 4GB RAM.
            1. Click 'See all sizes' for various instance sizes.
        5. Username: mutthojuadmin
        6. Password
        7. Leave ports
        8. Next to Disks:
            1. Standard SSD
        9. Networking
            1. virtual network: selected my-vnet
            2. subnet: front-subnet
        10. Review + Create
        11. Create
    2. Second VM
        1. Add Virtual Machine
        2. Resource group: vnet-test-rg
        3. Virtual machine name: back-vm
        4. Size: B2S
        5. credentials
        6. Disks:
            1. Standard SSD
        7. Networking
            1. virtual network: selected my-vnet
            2. subnet: back-subnet
        8. Review + create
        9. Create
2. Go to Resource
    1. back-vm
        1. Virtual network/subnet: my-vnet/back-subnet
        2. Private IP: It is in the range of back-subnet
    2. front-vm
        1. Connect
        2. Enter credentials
    3. back-vm
        1. Copy the IP address
    4. front-vm
        1. Windows button
        2. Type: remote desktop connection
            1. Paste IP of back-vm
            2. Connect
        3. Enter credentials
        4. Connect
            1. If error: protocol error
                1. If the machines are quite small and if they don't have enough cpu and RAM
3. Another virtual Network
    1. Virtual Networks
        1.
            - Create
        2. Resource group: vnet-test-rg
        3. Name: my2-vnet
        4. Region: Region of first vnet
        5. IP Addresses
        6. **Review + create**
        7. **Create**
    2. VM:
        1. Delete back-vm
        2.
            - Add
            1. Resource group: vnet-test-rg
            2. Virtual manchine name: vnet-vm
            3. Region: Region of first vnet
            4. OS: Windows
            5. Size: Standard B2s
            6. Credentials
            7. Disks:
                1. Standard SSD
            8. Networking
                1. Virtual network: my2-vnet
            9. **Review + Create**
            10. **Create**
            11. Go to resource:
                1. VNet: my2-vnet
                2. Copy private IP
    3. Go to front-vm
        1. Connect
        2. Open RDP Inside
        3. Connect using the copied private IP

1. Doesn't connect
4. Delete resource group: **vnet-test-rg**

Network Security Groups

1. Usually called NSG
    1. A gatekeeper for Subnets
        1. Defines who can connect in and out of subnet
            1. Think of it as a mini-Firewall
                1. Stands at the entrance of the subnet and decides whether the connecting party can actually connect to the Subnet
        2. Should be a standard part of Subnet creation
            1. **Ensure that an NSG is attached to it**
        3. **It is free**
2. How does NSG work?
    1. Looks at 5 tuples:
        1. Source (= Where did the connection come from)
        2. Source Port (=The port the source is using)
        3. Destination (=Where does the connection request goes)
        4. Destination port (=To which port does it want to connect)
        5. Protocol (=TCP, UDP, Both)
    2. Based on the 5 tuples, the connection is either allowed or denied
    3. The is called **Security Rule**
        1. Multiple security rules combined is called **Security Policy** of the NSG
    4. Each rule is assigned a number
        1. **The lower the number - the higher the priority of the rule**
            1. **Ensure that this rules is applied to the connections and there are no other rules with higher priority that will override it.**
3. NSG and VMs
    1. An NSG is automatically created and attached to every newly-created VM's network interface
        1. Also applied to VM's network interface
    2. **By default - open RDP (on Windows) or SSH (on Linux) port to anyone
        1. **MUST be handled first thing after creation**

Setting Up the Catalog's NSG

1. Go to catalog-vm
    1. Click **Start**
    2. Click **Networking**
        1. Virtual network/subnet
        2. Network security group: catalog-vm-nsg
            1. Inbound port rules
                1. Priority:
                    1. 65500 - Deny/AllInbound (by default unless specified otherwise)
                    2. 65001 - AllowAzureLoadBalancerInBound
                    3. 65000 - AllowVnetInBound
                        1. Any traffic from peered Azure Network
                    4. 300 - RDP
                        1. Double Click
                            1. Source: Any
                            2. Source port ranges: *
                            3. Destination: Any
                            4. Destination Port: 3309
                            5. Protocol: TCP
                            6. Priority: 300
                        2. Open another page in browser:
                            1. What is my IP
                                1. Copy the IP
                            2. NSG:
                                1. Source: IP Addresses
                                    1. Source IP addresses/CIDR ranges:
                                        1. Paste the IP address
                                2. Save
                        3. Connect:
                            1. Download RDP file
                            2. Type credentials
                            3. Connected
                                1. Close the connection
                        4. Double click:
                            1. Change IP
                            2. save
                            3. Connect
                                1. RDP cannot initiate the remote connection
                    5. There is no port 8080 to browse to catalog app
                        1. Add inbound security rule
                            1. Source: Any
                            2. Source Port Range: *
                            3. Destination: Any
                            4. Destination port range: 8080
                            5. Protocol: TCP
                            6. Priority: 310
                            7. Name: Port_8080
                            8. Description: Allows browsing to the catalog app
                        2. Copy public IP
                            1. :8080
                                1. Opens the catalog service.
            2. Outbound port rules

Setting Up the Weather API's NSG

1. Go to weather-vm
    1. Start It
        1. Connect
            1. Open PuTTY
                1. Copy and paste public IP
                2. Open
                3. Enter credentials
        2. `cd weatherAPI`
        3. `npm start`
            1. Listens on port 8080
    2. Open Catalog app in Browser
        1. Go to Weather

            1. Copy paste private IP of weather-vm

            2. Enter the following url:

                `<private-ip>:8080`

                1. Hit **Get Weather**
                    1. Both catalog service and weather service are in the same VNet (from the inside)
    3. Go to Weather machine
        1. Networking:
            1. 300: SSH: 22: TCP
                1. Double click:
                    1. copy my public IP:
                    2. Paste IP address
                    3. Save
                2. Only local machine can SSH into the machine.

Moving the Weather API to a New Subnet

1. The machines are in the same subnet
    1. Each machine can be in their own subnets
        1. Helps separate resources
        2. Allows configuring separate NSG for each group of resources
2. Open readit-app-vnet
    1. Open Subnets
        1.
            - Subnet
            1. Name: weather-subnet
            2. Subnet address range: 10.0.1.0/24 (offered by Azure)
                1. 251 addresses by default (which is way above what we need)
            3. NSG: Leave it empty
                1. Can choose Network security group: weather-vm-nsg (automatically created but modified manually)
            4. Save
3. Move weather VM to newly created Subnet
    1. Go to weather-vm
        1. Click **Network Interface** name
            1. IP configurations
                1. Subnet: weather-subnet
                    1. **The VM will be restarted**
                        1. Save
        2. weather-vm
            1. Networking: virtual-network/subnet: readit-app-vnet/weather-subnet
            2. Private IP: 10.0.1.4
            3. Copy public IP
                1. Open PuTTY
                    1. Paste IP
                    2. Type credentials
                    3. `cd WeatherAPI`
                    4. `npm start`
        3. Copy private IP
            1. Go to Catalog in browser
                1. 10.0.1.4:8080
                2. Get Weather
                    1. Can connect because they are in the same VNet but different subnet
    2. Search Network Security Group
        1.
            - Add
            1. subscription
            2. Resource Group: readit-app-rg
            3. Name: weather-subnet-nsg
            4. Region: West Europe
            5. **Review + Create**
            6. **Create**
        2. Open weather-subet-nsg
            1. Inbound security rules
                1. Azure does not include SSH
            2. Attach NSG to Subnet
                1. Click **Subnets**
                    1.
                        - Associate
                        1. Virtual netowork: readit-app-vnet

2. Subnet: weather-subnet
3. OK

3. Catalog Page:
    1. Can still connect to weather API because we didn't explicitly block anything from the VNet
        1. Go to NSG:
            1. Inbound security rules:
                1.
                - Add
                1. Source: IP Address
                2. Source IP addresses/CIDR ranges:
                    1.
                3. Destination port ranges: 22
                4. Protocol: TCP
                5. Priority: 100
                6. Name: SSH
                7. Add
4. Weather VM page:
    1. Copy public IP
        1. Connect from PuTTY
            1. We can connect

Network Security Groups Tips and Tricks

1. We can also setup security rules using **Service Tags**
    1. They represent well known sources which can be used in the rule (we don't have to know their IP addresses)
        1. Source: Service Tag
        2. Source service tag:
            1. Internet
    2. Common examples of service tags:
        1. Internet: All the requests coming from the internet
        2. VirtualNetwork: Requests coming from other Azure's Virtual Networks (service tag used bydefault rule created for every NSG)
        3. LoadBalancer: Requests coming from Azure Load Balancer (used by default rule created for every NSG)
        4. AzureMonitor: Requests coming from Azure Monitor, and used for monitoring the app and more
    3. Note: Use Service Tags when rule applies to built-in resources & not resources created by us.

Network Peering

1. What is it?

    1. Sometimes, to increase security, we want to place some resources in a completely different VNet
        1. Not just Subnet!
    2. Examples:
        1. Separate systems (supposed to be in different VNet)
        2. System layers
            1. Front End
            2. Business Layer
            3. Data Layer
        3. Sensitive Databases

2. Why?

    1. Not to place non-public resources in a VNet that has public access

        1. Example:

```
VNet
        |
        v
    NSG: Open port 80
        |
        v
    VM: Front End
        |
        v
    VM: Database
```

            1. User might bypass and go to database

    2. Solution:

```
VNet 2

        VM: Database
```

    3. Problem:

        1. VNet 1 resources cannot communicate with resources in VNet 2
            1. Solution: Network Peering

3. What is Network Peering?

    1. Allows two VNets to connect to each other
    2. From the user's point of view it's a single VNet
    3. Make sure address spaces are not overlapped!
        1. If VNets share the same address space, they do not behave like a single VNet from user's point of view.
            1. Addresses collide
    4. Use NSG for protection (especially for peering)
    5. **Can work across regions**

6. **Not free** (pay for actual traffic)
    1. Outbound data transfer
        1. 100 GB x $0.0100 per GB
    2. Inbound data transfer
        1. 100 GB x $0.0100 per GB

4. Architecture:

```
VNet 2
    VM: Database

    NSG: Open Port 1433
        ^
        |
        peering
        |
        v
VNet 1
    NSG: Open Port 80 <- User
        |
        v
    VM: Front End
```

1. The resources can talk to each other

Moving the Weather API to a New VNet

1. Search Virtual networks
    1. readit-app-vnet
        1. Remember the address space
    2.
        - Create
        1. Resource group: readit-app-rg
        2. Name: weather-vnet
        3. IP Addresses
            1. Address space must be different from address space of original VNet
            2. Click **default**
                1. Subnet name: weather-subnet
                2. Subnet address range: /24
                    1. Enough address space for other subnets
                3. Save
        4. Review + Create
        5. Create
    3. Move weatherApp to new VNet
        1. There is no simple way
            1. weather VM
                1. Delete
                    1. Check Network Interfaces and Public IP address
                    2. Uncheck OS disk
                        1. Can be used for new VM
                2. Delete
            2. Search Disks
                1. weather-vm-xxxx
                    1. Click on the disk
                        1.
                            - Create VM
                        1. Virtual machine name: weather-vm
                        2. OS is pre-populated
                        3. Next Disks:
                            1. We cannot change the type (Standard SSD)
                        4. Next Networking:
                            1. Virtual network: weather-vnet
                            2. Subnet: weather-subnet
                            3. Public IP: Needed by PuTTY
                                1. Create new
                                    1. Assignment: Static
                                2. OK
                        5. Management
                            1. Enable Auto-shutdown
                            2. Uncheck Notification before shutdown
                        6. Review + Create
                        7. Create
                    2. Credentials are already stored in the disk
                2. Go to Resource
                3. The machine is in the VNet/Subnet
            2. Networking
                1. Double click: 300; SSH
                    1. Source: IP Addresses
                    2. Source IP addresses/CIDR ranges:
                    3. Save
                2. Copy public IP
                3. Open PuTTY
                    1. Paste public IP
                    2. Open
                    3. Enter credentials (same)
                    4. `cd weatherAPI`
                    5. `npm start`
        2. Go to catalog-vm

1. start
2. Copy public IP
3. Open: http://:8080
4. Go to Weather
    1. Copy private ip of weather VM
    2. :8080
        1. We have a problem
            1. The services are in different VNets

Using Network Peering

1. Go to weather-VM
    1. Click the Virtual network/Subnet name:
        1. Peerings
            1.
                - Add
            1. Peering link name: readit-peering
            2. Traffic: Allow
            3. Remote virtual network
                1. Peering link name: weather-peering
                2. Subscription: weather vnet subscription
                3. Virtual network: readit-app-vnet
            4. Add
        2. Two peerings are added
            1. From readit-app-vnet to weather-vnet and back
    2. Go back to catalog Service
        1. Weather
            1. Get Weather
                1. We can connect
2. Go to NSG:
    1. weather-vm
        1. Networking
            1. 65500 - AllowVnetInBound
                1. Source: VirtualNetwork (from any)
                2. Destination: VirtualNetwork (to)
                3. Action: Allow
            2. **Add inbound port rule**
                1. Action: Deny
                2. **Add**
                    1. Priority: 65500
        2. Go to catalog app and click **Get Weather**
            1. Doesn't work
        3. Networking:
            1. Double click Port_8080
            2. Source: IP Addresses
            3. Source IP: private ip of catalog service
            4. Protocol: TCP
            5. Action: Allow
            6. Description: Allow catalog VM to access the wather API
            7. Save
        4. Try to click **Get Weather** in catalog app
            1. Allows traffic from catalog vm only

Network Topology

1. Search: network watcher
    1. Network Watcher
        1. Monitoring:
            1. Topology
                1. Resource Group: readit-app-rg
                    1. Shows the current topology of the app
        2. Network diagnostic tools:
            1. Connection troubleshoot
                1. Allows us to check whether two resources can connect to each other.
                    1. To know why they cannot talk to each other
                2. Select Source Resource
                3. Select Destination Resource
                4. Click: **Check**

Current Architecture

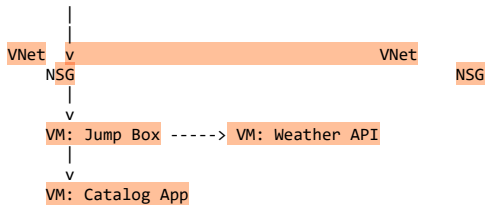
```
VNet                                                    VNet
    NSG <------- Peering -------    NSG

    VM: Weather API                             VM: Catalog App
```

Secure VM Access

1. Attack Surface
    1. VMs expose public API
        1. Hackers can hack into VMs and cause data breach and cause shut-down
            1. No alternatives so far
                1. Had to expose public IPs
                    1. For browsing
                    2. To connect using RDP or SSH
                        1. **Focus on this here**

2. Secure VM Access
    1. The larger the attack surface - the greater the risk
        1. Exposing multiple public IP addresses to the internet makes security risk much bigger.
            1. We want to minimize it as much as possible
                1. Leaving public IPs open is always a risk we want to avoid
                    1. We cannot always avoid it
    2. Secure VM: It is not directly related to the app design but important nonetheless
        1. Architect doesn't need to handle
    3. What can be done?
        1. How to protect the VM against malicious attacks? How to minimize risk caused by public IPs exposed to the internet
            1. 4 techniques:
                1. **JIT Access**
                2. **VPN**
                3. **Jump Box**
                4. **Bastion**
3. **JIT Access**
    1. Just in Time access
        1. Open port for access on demand, and automatically close it
            1. Not always open to the internet
                1. Only when RDP or SSH is required
                    1. Automatically closed
    2. When there is no access, hackers do not get access
    3. Can be configured from the VM's page in the portal
    4. **Requires Security Center License Upgrade**
        1. We currently don't have
            1. If we have the license
                1. Open catalog-vm
                    1. Click **Configuration**
                        1. Just-in-time VM access
4. VPN
    1. A secure tunnel to the VNet
        1. Communication between me and Azure is secure using secure tunnel and no one else can utilize the tunnel to connect to the machine
            1. **Challenges**
                1. Requires VPN software and license (not part of Azure)
    2. Can be configured so that no one else can connect to the VNet
        1. **Requires VPN software and license (not part of Azure)**
        2. **Configuration is complicated**
        3. **License is expensive**
5. Jump Box:
    1. Place another VM in the VNet
    2. Allow access ONLY to this VNet
        1. Not to any other VNet or other peered VNet
    3. When we need to access one of the other VMs - connect to this one and connect from it to the relevant VM
    4. **Only one port is open (still kind of a problem)**
        1. Single exposed port (still a problem but no port per VM)
            1. Only Jump Box port is open to the internet and every other VM is connected to the Jump Box and not exposed to the internet
    5. Cost: The additional VM (the Jump Box)
    6. Architecture:



6. Bastion:
    1. A web-based connection to the VM
    2. No open port is required
    3. Simple and Secure
    4. **Cost: ~140 $/ month**

Using Bastion

1. Open catalog-vm
    1. Start
    2. Search: Bastion
        1. Bastions
            1. Create Bastion
                1. Resource group: readit-app-rg
                2. Name: readit-bastion
                3. Region: West Europe
                4. Virtual network: readit-app-vnet
                    1. Error: subnet name must contain **AzureBastionSubnet** and prefix must be at least /27 (range)
                        1. Click **Manage subnet configuration**
                            1.
                                - Subnet

                1. Name: AzureBastionSubnet
                2. Address range: /24
                3. Save
            2. Subnet is automatically selected
        2. Public IP address: Create new
    5. Review + Create
    6. Create
        1. Can take time
3. Go to catalog-vm
    1. Networking
        1. Remove: 300| RDP
            1. Delete
    2. Settings > Connect
        1. Select **BASTION**
            1. **Use Bastion**
            2. credentials (of VM)
                1. If error: Always allow pop-ups ...
        2. A new window is opened
            1. VM screen is in the browser
                1. Open browser: localhost:8080
                    1. Catalog app
4. It is too expensive
    1. So Delete
        1. Go to Bastion
            1. Delete
5. Go to catalog VM
    1. Add inbound port rule
        1. Source: IP Addresses
        2. Source IP ...
            1.
        3. Destination port ranges:
            1. 3309 (RDP)
        4. Protocol: TCP
        5. Action: Allow
        6. Name: RDP
        7. **Add**
6. Remove public ip created for bastion:
    1. Go to resource group: readit-app-rg
        1. -ip
        2. Select
        3. Delete
        4. Yes
            1. We will end up paying nothing

2. Bastion Downsides:
    1. Cost
    2. Requires portal access
        1. We need to click **Connect via Bastion**
            1. RDP or SSH do not require portal access
        2. This is said to be handled by the Bastion team
            1. Working on a solution that does not require portal

Service Endpoint

1. A lot of managed services expose public IP
    1. i.e. Azure SQL Server, App Services, Storage, VM, ...
2. Sometimes the resources are accessed only from resources in the cloud
    1. i.e. Database in the Backend
        1. Shouldn't be accessed from the internet
            1. Security risk Otherwise
                1. Solution: Service Endpoint
3. What is it?
    1. It constructs a route from the VNet (where the resource connecting to the managed service is located) to the managed service
    2. The traffic never leaves Azure backbone
        1. Although the resource still has a public IP
            1. A direct connection from calling resource to called resource (doesn't go to the internet and return)
    3. Access from the internet can be blocked
    4. It is free!
4. How it's done?
    1. Enable Service Endpoint on the Subnet from which we want to access the resource
        1. Example: VM that wants to access Azure SQL (a managed service) is within a subnet. Enable service endpoint on the subnet
            1. On the resource (Azure SQL say), set the subnet as the source of the traffic
    2. Two configuration:
        1. One on connecting resource side
        2. One on connected resource side
    3. The services can then connect directly without going to the internet
5. Example:
    1. If there is a VM (Front End) that wants to talk to App Service (Backend App), it normally must go through the internet (to and back)
    2. With service-endpoint, VM will go to the public IP of the app service using Azure backbone (not the internet)
6. Notes:
    1. Traffic leaves the VNet
    2. There is a public IP on the PaaS service (App Service)
    3. Can't be used from on-prem network
        1. Almost
            1. Complicated
7. Resources supporting Service Endpoint:
    1. Storage

2. SQL Database
3. Synapse Analytics
4. PostgreSQL
5. MySQL
6. Cosmos DB
7. Service Bus
8. Event Hub
9. App Service
10. Cognitive Services

Private Link

1. A lot of managed services expose public IP
    1. Example: Azure SQL Server, App Services, Storage, ...
2. Sometimes the resources are accessed only from resources in the cloud
    1. Example: Database in the backend
    2. Or else it might pose a security risk
3. A newer solution to the problem (than service endpoint) and is Better
    1. Extends the managed service into the VNet
        1. The traffic never leaves the VNet
            1. Stays private all the time
    2. Access from the internet can be blocked
        1. It will not hurt the communication between the resources
    3. Can be used from on-prem networks
    4. **It isn't free**
4. How it's done:
    1. Configure the resource to connect to the VNet
        1. Example: Connecting Azure SQL to the VNet where the VM is located
            1. A private link is created between VM and SQL
    2. **Configure private DNS**
        1. Might cause a problem if you have your own DNS
            1. Done automatically by Azure
    3. Configuration:
        1. If no private link: If VM wants to connect to App Service, it has to go through the internet and public IP address
        2. If private link: App Service becomes part of the VNet and VM can directly talk to the App Service.
            1. No public traffic
    4. Note:
        1. Traffic never leaves the VNet
        2. The VM talks to the App Service via **private IP**
            1. VM is not aware of the public IP address of the app service
        3. Can be used from on-prem network
5. Resources supporting Private Link: (much longer and growing)
    1. Storage
    2. SQL Database
    3. Synapse Analytics
    4. PostgreSQL
    5. MySQL
    6. Cosmos DB
    7. KeyVault
    8. Redis
    9. AKS
    10. Search
    11. ACR
    12. App Configuration
    13. Backup
    14. Service Bus
    15. Event Hub
    16. Monitor
    17. Relay
    18. Event Grid
    19. App Service
    20. Machine Learning
    21. Automation
    22. IOT Hub
    23. SignalR
    24. Batch
6. Service Endpoint is legacy and Private Link is the way to go to connect privately

Service Endpoint vs Private Link

1. Comparison:
    1. Service Endpoint
        1. Security: Connects via Public IP
        2. Simplicity: Very simple
        3. Price: Free
        4. Supported services: Limited list
        5. On-Prem connectivity: Quite complex
    2. Private Link
        1. Security: Connects via Private IP
        2. Simplicity: More complex
        3. Price: Not free
        4. Supported services: Large list, probably will get larger
        5. On-Prem connectivity: Supported
2. Service Endpoint is legacy
    1. However:
        1. Setup simplicy

2. Free
3. On-prem is not required

3. Service Endpoint - on Application Gateway and Azure SQL Server
4. Private Link - on KeyVault

VNet Integration

1. It allows access from App Service to resources within VNet
    1. So that the resources should not be exposed on the internet
        1. Useful when App Service needs to access a VM with some internal resources
            1. VM is placed inside a VNet
    2. VNet integration allows App Service to connect into VNet and access resources inside it
    3. Supports same-region VNets
        1. For VNets in other regions - a gateway is required
2. What is the difference between SE/PL or VNet Integration?
    1. Service Endpoint/Private Link:
        1. We allow access from a resource inside a VNet to a managed Service
            1. Downstream connection (from VNet to Managed Service)
    2. VNet Integration:
        1. Allows connecting from managed service (App Service say) to a resource in a VNet
            1. Upstream connection (from Managed service to VNet)
3. Go to App Services
    1. readit-inventory-mutthoju
        1. Scale up (App Service plan)
            1. Production
                1. See additional options
                    1. S1 (Standard 1)
                2. Apply
        2. Networking:
            1. High level Overview
                1. VNet Integration:
                    1.
                        - Add VNet
                    1. Virtual Network: readit-app-vnet
                    2. Subnet: AzureBastionSubnet (only to an empty subnet)
                        1. Not with the one with other resources
                    3. OK
                        1. App service can connect to the resources in the VNet
        3. Scale up
            1. Dev/Test
                1. F1
                2. Apply

Access Restrictions

1. App Service Access Restrictions
    1. Similar to NSG - but for App Services
    2. Restricts traffic to the App Service
    3. By default - all inbound traffic is allowed (in relevant ports)
        1. 8080, 443
            1. App Service is a web-app and we want web-app to be open to the world
    4. Using access restrictions inbound traffic is restricted to the allowed IPs/VNets/Service Tag (general name for managed Azure service (Azure Load Balancer say))
2. Main use cases:
    1. Backend App Service that should be accessed from front end App Service/ VM only
        1. Backend app service should not be accessed from the internet
    2. App Service that sits behind Application Gateway / Load Balancer and shouldn't be accessible directly
        1. Because public access is through the gateway
    3. Open App Service to a specific customer only (with a specific ip range)
3. Open App Service Page
    1. readit-inventory-mutthoju
        1. Networing
            1. Inbound Traffic
                1. Access restrictions
                    1. Default rule: All traffic from any source
                    2.
                        - Add rule
                    1. Name: my-ip
                    2. Action: Allow
                    3. Priority: 100
                    4. Description: Allow access from my IP
                    5. Type: IPv4
                    6. IP Address Block:
                    7. Add rule
                    3. A new rule is added automatically
                        1. Priority: lowest
                        2. Denies all traffic
                2. All traffic is denied except from our System (public ip)
                    1. Go to overview:
                        1. Open URL in separate tab
                    2. Networking:
                        1. Click Access Restrictions
                            1. Change ip to something else
                            2. Update rule
                    3. Refresh the page - Error 403 - Forbidden
                3. Remove the rule

1. Allows from anywhere
    1. Access restriction - Off

ASE

1. App Service Environment
    1. Special type of app service deployed directly to a dedicated VNet
    2. VNet can be configured like any other VNet - Subnets, NSGs, etc...
    3. Created on dedicated hardware
        1. **Separate from other hardware which might host other app services**
            1. For regular app service, we don't have the option to define the VNet inside which we want the app service to be deployed. (not possible with standard tier)
    4. Quite expensive... ($ 800 / month to $ 1000 / month)
    5. Major use cases:
        1. Elevated security - complete isolation of app service from its environment in a different VNet with NSGs and subnet (all isolation that VNet can provide)
        2. Very high scale requirements
            1. Far exceeds what regular app-service provides.
                1. Consider ASE if we have very high scaling requirements.
2. ASE Architecture:

```
VNet 1
    Subnet Frontend
        ASE (Frontend App)
            |
            v
    Subnet Backend
        VM
```

    1. App service behaves like any resource that can be placed inside a VNet and it enjoys all the benefits of Virtual Network
3. Cannot configure with free account.
    1. Go to readit-inventory app service
        1. Scale Up (App Service plan)
            1. Select **Isolated (Advanced networking and scale)**
                1. Supported with regular subscription

Load Balancer

1. Azure service that distributes load and checks health of VMs
    1. When a VM is not healthy - no traffic is directed to it
2. Can work with VMs or Scale Set (collection of VMs that can be automatically scaled out or in as needed)
3. Can be public or private
    1. It can be accessed over the internet or used only within a closed VNet
4. Operates at layer 4 of the OSI model
    1. The OSI Reference Model

```
Application
Presentation
Session
Transport
Network
Data Link
Physical
    |
----+------ Physical Medium
```

        1. Layer 4: Knows about IP, Port, protocol, TLS and more. **Has no knowledge about the actual content**
            1. Only technical details
        2. Layer 7: Interacts with the application. Can see the content of the transmission. Example: **HTTP URL + path + params**
5. Load Balancer deployment:

```
VNet   |
       |
    TCP Port 80
       |
    Public Load Balancer
    |  |       |
    80 80      80
    |  |       |
    VM VM      VM (in Web Tier Subnet)
    |  |       |
    +---+---+
        |
    Internal Load Balancer
        |
    +---+---+
    |  |       |
    443 443 443
    |  |       |
    VM VM      VM (in Business Tier Subnet)
```

    1. Public Load Balancer: Exposed to the internet via port 80

        1. Distributes the traffic across VMs
    2. Internal Load Balancer: Received traffic from Web Tier
        1. Distributes the traffic across VMs

6. Load Balancer Distribution Algorithm:

    1. Load balancer uses the algorithm to distribute the traffic it gets & how to decide which VM it needs to route the request
    2. The algorithm is based on 5 tuple hash:
        1. Source IP
        2. Source Port
        3. Destination IP
        4. Destination Port
        5. Protocol Type
    3. Same tuples used by NSG
    4. Load Balancer uses the 5 tuple elements to decide how to route the incoming traffic

7. Load Balancer Types:

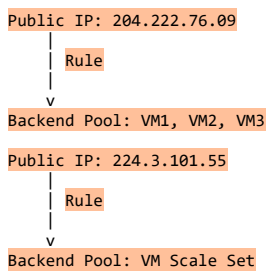
    1. Basic
        1. No redundancy
        2. Open by default
            1. It has no security protocols
        3. Up to 300 instances
        4. No SLA
        5. Free
    2. Standard
        1. Redundant
            1. If there is a problem in the underlying infrastructure supporting the load balancer, we have another instance of the load balancer which automatically starts working and we won't even know there is a problem.
        2. Secure by Default
            1. Applies some security measures that can prevent some kinds of attacks
        3. Up to 1000 instances
        4. 99.99% SLA (quite good)
        5. Not Free

8. Configuring Load Balancer

    1. 4 main configurations:

        1. Frontend IP configuration
            1. The public IP exposed by the Load Balancer
                1. Under the hood the load balancer distributes traffic to the underlying VMs
        2. Backend pools
            1. The VMs connected to the Load Balancer
            2. It contains a list of VMs that the Load Balancer will route the traffic to.
        3. Health probes
            1. Probes checking the health of the VMs.
            2. Non-healthy VM will not be routed to
        4. Load balancing rules
            1. A rule connecting Frontend IP with Backend pool.
                1. Each rule connects the front-end IP address to the backend pool
                    1. **Load balancer can expose multiple IP addresses with each leading to a different backend pool**

    2. Example:



9. Health Probes:

    1. They check the health of the VM
    2. A non-healthy VM will be marked as Down and will not be routed to
    3. Run in intervals (usually a few seconds)
        1. Every few seconds, the health probe checks the health of the VMs and if it finds any problem, it will mark the VM as down
        2. It can run on TCP, HTTP, HTTPS (Standard LB only)
    4. We can configure unhealthy threshold - how many times a check should fail for the VM to be marked as Down (default is 2)
        1. If the health probe detects problem 2 times in a row, it marks the VM as a problematic one (as down).
            1. No traffic will be routed to it
                1. Configuration: it can be 1
    5. Health Probe definition:
        1. Add health probe:
            1. Name:
            2. Protocol: TCP, ... (HTTP for web-apps)
            3. Port: 80
            4. Interval: 5 seconds (checking interval)
            5. Unhealthy threshold: 2
    6. Probe runs on VM's host
        1. No network traffic outside the host ever occurs using the probe
            1. Runs on host machine hosting the VMs

7. Always originates from the same IP: 168.63.129.16
8. Allowed by default in NSG
    1. 65001 - AllowAzureLoadBalancerInBound

10. When to Use Load Balancer:

    1. Great for internal resources
    2. Do not use for external resources
        1. Especially on Web Apps / Web API / etc.
        2. Can't handle HTTP
            1. It cannot see HTTP paths in the route
        3. Doesn't route based on path
        4. No protection
    3. For this we have the application gateway
        1. Which improved HTTP and web capabilities

Application Gateway

1. What is an application gateway?

    1. Web traffic load balancer
        1. Load balancer with improved web capabilities
    2. It can function as the external endpoint of the web apps
    3. Works with:
        1. VMs
        2. VM Scale Sets
        3. App Services
        4. Kubernetes (requires some hacking...)
            1. AKS

2. It is similar to load balancer:

    1. Comes with additional features:
        1. SSL Termination
        2. Autoscaling
        3. Zone redundancy
        4. Session affinity
        5. URL based routing
            1. Routing traffic based on URL path
        6. WebSocket and HTTP/2 support
        7. Custom error pages
        8. Header & URL rewrite
        9. WAF
        10. ...

3. Application Gateway operates at layer 7 of the OSI model:

    1. Layer 7: Interacts with the application
        1. Can see content of the transmission
        2. Example: HTTP URL + path + params

4. Operates at layer 7 of the OSI model

```
                        Application
                        Gateway          - /images/* -> ImageServerPool

                          WAF                    - /video/* -> VideoServerPool
User - contoso.com ->
                          L7 LB
```

5. WAF:

    1. Web Application Firewall
        1. Protects web apps against common attacks
            1. i.e. Cross-site scripting, SQL injection, etc.
        2. Protection rules based on OWASP Core Rule Set
            1. Take Software Architecture Security course
            2. It can work in detection or prevention mode
                1. Detection mode: If WAF detects a security related problem in the traffic, it simply notifies and warns that there is a problem but it will not deny the traffic.
                2. Prevention mode: If WAF detects a security related problem in the traffic, it blocks the traffic and notify
        3. Protection rules are based on OWASP Core Rule Set
        4. Updates continuosly
    2. It is usually based on 3rd party products (Palo Alto, Fortinet, Imperva etc.)
        1. In these cases, there's no need for the WAF in the Application Gateway
            1. Application gateway can be used as-is without WAF addition

6. Application Gateway SKUs

    1. It comes in 2 SKUs
        1. Standard_V2 - Includes all the features mentioned, excluding WAF
        2. WAF_V2 - Includes everything (almost double the price...)

7. Application Gateway Networking:

    1. Application Gateway is placed in its own Subnet
    2. Often in its own VNet
    3. Must make sure that backend resources are:

1. **Accessible from the AG Subnet**
2. **Not accessible from anywhere else...**
    1. If we can bypass Application Gateway and go straight to the VM or App Service, then AG is useless.

8. Application Gateway Deployment:

```
AG VNet
    AG Subnet
    |          |
    v          v
    VM         App Service
```

    1. We must ensure that the resources are not accessible from anywhere else, we must employ traffic restrictions.
        1. In case of VM, the restriction will come as an NSG
            1. We will use NSG to limit the traffic to the VM's VNet (traffic is allowed only from AG)
        2. In case of App Service: We will use Service Endpoint + Access Restrictions OR Private link
            1. Blocks all traffic that is not from AG subnet

9. Configuring Application Gateway

    1. 5 main configurations:
        1. Backend Pools - The VMs, Scale Sets, or App Services connected (not just VMs or Scale Sets) to the Application Gateway
        2. HTTP settings: (Didn't exist in load balancer) Settings for the incoming HTTP Requests (parameters)
        3. Frontend IP configuration: The public IP exposed by the Application Gateway
        4. Listeners: Receives requests on a specific port and protocol
        5. Rules: A rule connecting Listener with Backend Pool

Creation of Application Gateway

1. Go to pricing calculator of Azure:
    1. Search: Application Gateway
        1. Application Gateway
        2. View
            1. REgion: West Europe
            2. Tier: Standard_V2 (No WAF)
            3. Capacity Units: 1 (single)
            4. Price: $195 / month
2. Portal:
    1. Search Catalog
    2. Go to Catalog VM
    3. Start
    4. Search Application Gateway:
        1. Application Gateways
            1.
                - Create
            2. Resource group: readit-app-rg
            3. Name: readit-app-gw
            4. Region: East US (cheapest)
            5. Tier: Standard_V2 (no WAF)
            6. Enable Autoscaling: No
            7. Instance Count: 1
            8. Availability Zone: None
            9. HTTP2: Enabled/Disabled
            10. Virtual Network: Create new
                1. Name: readit-appgw-vnet
                2. Address range: must not overlap with the catalog vm's vnet
                    1. We need to peer the vnets later
                        1. Check catalog vm's VNet
                            1. Search Virtual Networks:
                                1. Click
                                2. Address space
                3. Subnet:
                    1. Subnet name: gw-subnet
                4. OK
            11. Next Frontends
                1. Frontend IP address type: Public
                2. Public IP Address: Add new
                    1. Name: readit-ip
                    2. SKU: Standard
                    3. Assignment: Static
                    4. OK
            12. Next Backends
                1. Add a Backend pool
                    1. Name: inventory-pool
                    2. Target Type: App Service
                        1. Other options:
                            1. VMSS
                            2. Virtual Machine
                            3. IP Address or FQDN (Fully Qualified Domain Name)
                        2. readit-app-inventory-mutthoju
                    3. Add
                2. Add a Backend pool
                    1. Name: catalog-pool
                    2. Target Type: Virtual Machine
                        1. We cannot see the VM
                    3. Add Backend pool without targets: Yes
                    4. Add

13. Next: Configuration
    1. Routing rules
        1. Add a routing rule
            1. Rule name: catalog-rule
            2. Priority: 100
            3. Listener
                1. Listener name: catalog-listener
                2. Frontend IP: Public
                3. Protocol: HTTP
                4. Port: 8080 (VM is listening on this port but we don't have to match it to this)
                    1. HTTPS needs certificate configuration
                5. Listener Type: Basic
                6. Error page url: No
            4. Backend targets
                1. Target type: Backend pool
                2. Backend target: catalog-pool
                3. Backend settings:
                    1. Add new
                        1. Backend setting name: catalog-settings
                        2. Backend protocol: HTTP
                        3. Backend port: 8080
                        4. Add
            5. Add
        2. Add a routing rule
            1. Rule name: inventory-rule
            2. Priority: 101
            3. Listener:
                1. Listener name: inventory-listener
                2. Frontend IP: Public
                3. Protocol: HTTP
                4. Port: 80
            4. Backend targets:
                1. Target type: Backend pool
                2. Backend target: inventory-pool
                3. Backend settings:
                    1. Add new
                        1. Backend setting name: inventory-settings
                        2. Backend protocol: HTTPS (app service is expecting HTTPS)
                        3. Backend Port: 443
                        4. Backend server's certificate is issued by a well-known CA: Yes (App Service certificate)
                        5. Add
            5. Add
14. Next: Tags
15. Review + Create
16. Create
    1. Might take 10 - 20 minutes

Troubleshooting the Application Gateway

1. We might encounter problems due to some changes implemented in the Application Gateway and App Service default settings
    1. Troubleshooting the errors:
        1. Make sure we're accessing the IP address of Application Gateway with HTTP protocol, and not HTTPS
            1. http://255.255.255.255
        2. If after connecting App Service to Application Gateway, we get 403 error, it might mean that App Service is not configured to allow HTTP access and only HTTPS access
            1. Go to App Service in portal:
                1. Configuration
                2. General Settings
                3. HTTPS Only to Off
                4. Save
        3. If we get an error that the connection of the site is not secure. .azurewebsites.net sent an invalid response.
            1. Browser has HSTS policy active
                1. We cannot browse to unsecure sites
            2. Chrome:
                1. Navigate to `chrome://net-internals/#hsts`
            3. Edge:
                1. Navigate to `Edge://net-internals/#hsts`
            4. In Delete domain security policy, enter full domain of App Service
                1. `appservice-name.azurewebsites.net`
            5. Click `Delete`

Connecting the Inventory App Service to the Application Gateway

1. Portal:
    1. Search Application Gateway
        1. Application gateways
            1. readit-app-gw
                1. Backend health
                    1. Application gateway will try to connect to backend pools and check the status
                        1. Unhealthy (Received invalid status code: 404 in the backend server's HTTP ...
                2. HTTP settings:
                    1. inventory-settings
                        1. Override with new host name: Yes (App Service has its own hostname and operates under DNS name provided by Azure)
                            1. AG should modify the host header to match the hostname of the App Service
                        2. Host name override:
                            1. Pick host name from backend target

3. Save
3. Backend health
    1. Healthy
4. Overview
    1. Copy public ip address
    2. Open in browser:
        1. Application gateway can access inventory app service
5. App service must not be accessible from outside
    1. Overview:
        1. Click Virtual network/subnet
        2. Click Service endpoints
        3.
            - Add
            1. Service: Microsoft.Web
            2. Subnets: gw-subnet
            3. Add
2. Search for App Service
    1. readit-inventory
        1. Networking
            1. Access Restrictions
                1.
                    - Add rule
                1. Name: gw-access
                2. Action: Allow
                3. Priority: 100
                4. Description: Allow access from the GW service endpoint
                5. Type: Virtual Network
                6. Virtual Network: readit-appgw-vnet
                7. Subnet: gw-subnet
                8. Add rule
                2. The new rule allows access from AG VNet and Subnet only
        2. Overview:
            1. Copy URL and open in new tab: 403 forbidden
    2. Access the application through AG
        1. It works

Connecting the Catalog VM to the Application Gateway

1. Search for VM
    1. Open catalog-vm
        1. Copy Public IP address
        2. Paste in browser: :8080
2. Connect catalog-vm to Application Gateway
    1. Peering between VNets
        1. Go to catalog-vm
        2. Peerings
            1.
                - Add
            1. Peering link name: appgw-peering
            2. Remote virtual network: Peering link name: catalog-peering
            3. Subscription
            4. Virtual network: readit-appgw-vnet
            5. Add
3. Go to overview of catalog-vm
    1. Copy private IP of virtual machine
        1. AG is going to route the traffic to the private ip address of catalog app
4. Go to Application Gateway
    1. Go to Backend Pools
        1. catalog-pool
            1. Target type:
                1. IP Address or FQDN
                2. Paste private ip of catalog app
        2. Save
    2. readit-app-gw
        1. Backend health
            1. Both backend pools are healthy
5. Go to catalog-vm
    1. Networking:
        1. Inbound port rules
            1. Delete Port_8080 rule
                1. The traffic is allowed using ALlowVnetInBound rule (Source: VirtualNetwork)
    2. Open :8080
        1. The traffic is acceptible only from application gateway
            1. Testing: Copy public ip of VM
                1. Open browser and paste: :8080
                    1. Browser cannot access
6. AG is the only gateway to catalog app service and inventory app service

Cost of Application Gateway in This Course

1. If cost of AG exceeds free budget of $200
    1. Invest the money: Upgrade Azure account so that I will pay for the resource (just a little something)
    2. Finish fast: Go through the course as fast as possible (we will not pay for the full month)
        1. Not recommended
    3. Remove the application gateway: Best option may be
        1. Remove it an use the knowledge to architect in customer's systems

1. Access the apps directly
    1. Delete AG and Public IP attached to it
    2. Delete AG's VNet
    3. Delete Service Endpoint of Inventory App Service (the one connecting the App Service to Application Gateway's VNet)
    4. Add security rule to the Catalog VM that allows inbound traffic on port 8080

Application Gateway and AKS

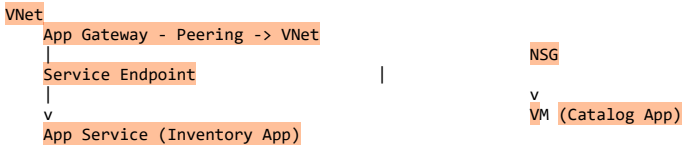
1. AKS:
    1. No built-in integration with AKS
        1. AKS has its own kind of gateway (=services)
        2. There's Application Gateway Ingress Controller (AGIC) that does this
            1. It connects AG to the AKS
                1. In preview mode, quite buddy, not recommended
    2. Better use 3rd party products
        1. For protection and routing

Application Gateway and Function Apps

1. Function Apps are basically App Services
    1. We had the option to connect to function app
2. They can be protected by Application Gateway the same way App Services are
    1. Configure in Backend pool
    2. Configure Access Restrictions (to block access to function apps that was not originated in the application gateway)
3. Our function won't be accessible from the web so no demo...
    1. **The procedure to setup AG for functions is the same as the one used to setup AG for App Services**

Current Architecture

1. App Gateway:

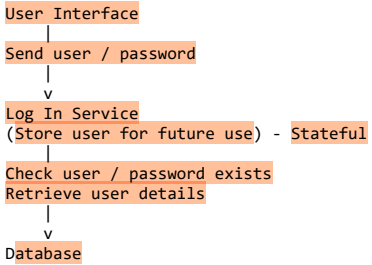


Affinity

1. Special setting in AG
    1. Add HTTP setting:
        1. Additional settings:
            1. Cookied-based affinity
            2. Connection draining
2. What is affinity?
    1. Makes sure user will always be directed to the same instance (VM / App Service) it began with
        1. If user 1 is directed to VM 1 for the first time, then subsequent requests will be directed to VM 1
            1. Problem from load balancer point of view (can overload the VM)
                1. Should be avoided when possible
3. Usually required in Stateful apps
4. Usually a sign of bad design
    1. Always try to design Stateless app (avoid affinity)

Stateless Architecture

1. Stateless:
    1. Most important pattern
    2. The application's state is stored in only two places - the data store and the user interface
    3. What is state?
        1. Application's data (temporary or persistent)
2. Architecture:



    1. Cornerstones of good architecture:
        1. Scalability
            1. Grow and shrink as needed
                1. When there is peek, a scalable system can add more resources but when there is no load, the system can remove resources
                2. Scale Up vs Scale Out
                    1. Scale Out is usually preferred
        2. Redundancy

1. Allows the system to function properly when resources is not working (using additional identical resource)
2. Also performs additional resource to replace malfunctioning resource
3. Example:
    1. A system with more than one server
    2. When a server goes down, the other continue working (with additional servers)

2. The architecture must be the following:

```
                User Interface
                     |
                     v
                Load Balancer
    |                |                          |
    v                v                          v
Log in Service  Log In Service  Log In Service
    |                |                          |
    v                v                          v
                Database
```

    1. Best practice is to use at-least 3 instances
    2. Load Balancer:
        1. Two roles:
            1. Distributes load
                1. If a server is very busy, the request is sent to another server
                    1. Ensuring that no server crashes under the load
                        1. This supports **scalability**
                            1. We can add or remove servers as we need and notify the load balancer
            2. Routes to live services only
                1. Checks status of servers and stops routing requests to malfunctioning ones
                    1. Load balancer periodically sends "Is Alive?" request
                    2. It expects a response with positive response
                        1. Any other response or no response indicates a problem with the server (or network connecting to the server), load balancer stops sends requests to the server
                2. This helps with achieving redundancy

3. A stateful application will have hard time with **scalability** and **redundancy**

    1. Why?
        1. If a user logs in in server 1
        2. Suppose user details are stored in Server 1
        3. If a user performs another request
        4. However, the load-balancer routes the request to server 2
        5. The server has no user details (which is in server 1)
        6. User will get response saying he is not logged in (but the user did that)
            1. This will confuse user
        7. Additional resources do not have any idea about the state and they assume it does not exist
    2. Stateless
        1. User state can be stored in data store (RDBMS, NoSQL, Distributed cache, ...)
        2. Once a user request arrives, the server queries the data store and comes to know who the user is (regardless of server used for log-in)
            1. Even if the server that was used for log in goes down.

4. Tips:

    1. Always use stateless architecture
    2. Supports Scalability and Redundancy

Application Gateway and Cookies

1. Super advanced topic
2. Traditional web-app
    1. App Gateway forwards request to App Service
    2. DNS:
        1. Used to resolve DNS name
    3. App Service has its own DNS name (myapp.azurewebsites.net)
    4. Suppose App Service returns a cookie (Domain Name is part of set-cookie response header)
        1. The cookie will contain: domain=myapp.azurewebsites.net
            1. There is a mismatch between the domain that user used and the domain returned in cookie response header
                1. Cookie will be dropped by browser (including login related one)
    5. Solution:
        1. Set custom domain for the App Service to be the same one as the Application Gateway
            1. Custom domains (page) of app service

Secure Network Design

1. Typical secure network design:

```
Frontend VNet <- peering - App Gateway VNet - peering -> Frontend VNet
                            |
                            peering (through NSG)
                            |
                            v
                           Frontend VNet
                            |
                            peering (through NSG)
                            |
                            v
                           Backend VNet
                            |
                            peering (through NSG)
                            |
                            v
```

```
Datastore VNet
```

1. App Gateway has its own VNet
2. Each layer has its own VNet
    1. Frontend VNet is peered to Backend VNet
    2. Backend VNet is peered to Datastore VNet

2. The access is extremely limited and hacker must work quite hard to gain access

3. The design pattern is called **Hub and Spoke**

    1. There is documentation

Section 9: Data in Azure

Introduction

1. Azure provides many data solutions as cloud services
    1. Relational databases
    2. NoSQL databases
    3. Object stores
    4. ...
2. They are fully managed services
    1. We don't have to deal with underlying infrastructure
3. Can be part of Azure app or completely independent (used by other applications that have nothing to do with Azure)
4. Various pricing models
5. Always better than unmanaged solutions
    1. Unmanaged: We control the VM that holds the database we are responsible for patching, hot-fixes, maintenance, ...
        1. We don't want to do this

Major Database Features

1. Features interested for us:
    1. What to look for when selecting a database?
        1. Security
            1. Network Isolation
                1. No one will be allowed to access the data if not allowed to
            2. Encryption
                1. Data stored in database must be fully encrypted
                    1. If someone steams the disk (or data) he wont be able to access the data
        2. Backup
            1. Backup Types
            2. Retention period
                1. What is the frequency of Backup
                2. How long is it stored before it is deleted
        3. availability
            1. SLA
            2. Replication
            3. DR

Database on VM

1. We have the option to install DB in a VM
    1. Azure VM can be setup with database software
    2. There are ready-made VMs in the marketplace containing pre-installed DBs
        1. Oracle Database 12.2.0.1 Enterprise Edition
            1. Click **Create**
2. Pros of Database on VM:
    1. Full Flexibility
        1. We can do whatever we want with the VM
            1. We can install any edition of the DB
            2. We can configure it anyway we want
    2. Full Control
        1. On VM and DB
3. Cons of Database on VM:
    1. You have to take care of Everything
        1. SLA
        2. Updates
            1. to VM
            2. to DB
        3. Availability
        4. Security
        5. Backups
        6. ...

Azure SQL

1. Managed SQL Server on Azure
    1. Works like any othe SQL Server using the same tools
    2. Great compatibility with on-prem SQL Server
        1. Depends on the exact Azure SQL Flavor (select the right flavor to get max compatibility with local server)
    3. Offers built-in security, backups, availability and more
    4. Flexible pricing models (can be confusing)
2. Azure SQL Flavors
    1. Azure SQL Database

    2. Elastic Pool
    3. Managed Instance
3. Azure SQL Database:
    1. It is a managed SQL Server on Azure
    2. It is a single database on a single server
    3. It has automatic backups, updates, and scaling
    4. It has good compatibility with on-prem SQL Server
        1. Not all features are supported
            1. We might see some features supported in local SQL server that may not be supported by Azure SQL database
    5. Security:
        1. IP firewall rules
            1. Only allowed ip addresses can communicate with DB
        2. Service Endpoints
            1. A close link between a managed resource and a vnet
        3. Offers SQL & Azure AD Authentication
            1. We can choose the authentication method to use.
        4. Secure communication (TLS)
            1. Traffic between app and db is always encrypted
        5. Data encrypted by default (TDE)
            1. Transparent Data Encryption
    6. Backup:
        1. Full Backup
            1. Every week
        2. Differential Backup
            1. Every 12 - 24 hours (depending on config)
        3. Transaction Log Backup
            1. Every 5 - 10 minutes
                1. Maximum data we are going to lose in case of failure is last 5 minutes
    7. Retention Period:
        1. Regular backup: 7 - 35 days (default is 7)
        2. Long term backup: up to 10 years
            1. More than we will ever need
    8. Availability:
        1. Backup is stored in a geo-redundant storage
        2. If we need, it offers **active geo-replication** which provides a fully active deployment of the database
            1. If a region is shutting down, the secondary region will have an up to date database which we can use immediately
        3. SLA: 99.9% - 99.995 % (depends on tier and redundancy option during creation)
            1. This is second highest in Azure (first is Cosmos DB)
4. Compute Tiers:
    1. Provisioned
        1. Pay for allocated resources regardless of actual use (ex: 4 vCores - we pay for them)
        2. Can be reserved
            1. To pay up-front fee and get hefty discount
                1. Might save a lot of money
    2. Serverless
        1. Pay for actual use - vCore + RAM / second
            1. Only consumed vCores and RAM
        2. It is automatically paused when inactive (when no one is using the database) (pay just for storage)
            1. Ex: If DB is going to be used only during normal working hours (9 - 5), rest of the day, we are not going to pay for it besides used storage
        3. Slight delay when warming up (similar to a function app)
            1. Not really a problem (it is not that long)
                1. Doesn't usually happen many times during a day (it might happen at the beginning of a workday)
        4. **Can't be reserved**
5. Elastic Pool:
    1. It is based on Azure SQL
    2. Allows storing multiple databases on single server
        1. Great for databases with low average utilization and infrequent spikes
            1. Ex: Spikes at 12:00 and 10:00 minutes. If we have multiple databases, each spike at different times
                1. At no point in time all databases are spiking. They are not in need of the full cpu individually (sharing of cpu)
    3. It is very cost effective
        1. Purchase the only compute resources you need, not the database
            1. We can put as many databases as we want as long as the resources of the server can support the number of databases
6. Managed Instance
    1. It is closer to the **on-prem SQL server** than any other flavor
    2. Near 100% compatibility with on-prem SQL
    3. We can also deploy it into VNet (cannot be done with other flavors)
    4. Business model is also close to on-prem on (as opposed to other flavors)
7. Main differences
    1. We don't have active geo-replication
        1. We cannot use this type to replicate database in real-time to other regions
    2. SLA: 99.99%
        1. Middle of other flovors
    3. Supports built-in functions (Azure SQL db does not support)
    4. Runs CLR code (?)
    5. No auto scaling & tuning
        1. We need to take control on scalability and fine tuning of the db
            1. It is done automatically with Azure SQL database
    6. No availability zone
        1. We have it with Azure SQL db
    7. No serverless tier
    8. No Hyperscale
        1. A specialized tier which offers superior performance
8. Use cases:
    1. If we want our Azure SQL to be as similar to the on-premises deployment as possible
        1. We need to compromise on cloud management capabilities
            1. Autoscaling
            2. Tuning

3. Availability

Azure SQL Pricing

1. It is quite complicated
2. Factors in pricing:
    1. Flavor
        1. Managed Instance
            1. Service Tier:
                1. General Purpose
                    1. Instance type:
                        1. Instance Pools
                            1. Instance (vCores)
                                1. 8..80 (80 core is extremely strong we usually don't need such a strong database)
                        2. Single Instance (a single hardware that is shared among multiple instances)
                            1. Instance (vCores)
                                1. 8..80
                2. Business Critical (offers improved performance and higher SLA)
                    1. Instance type:
                        1. Single Instance (no instance pool because we want hardware to be dedicated to our db for a business critical system)
                            1. Instance (vCores)
                                1. 8..80
            2. Example:
                1. Region: West India
                2. Instance Type: Single Instance
                3. Instance: 8 vCore
                4. Service Tier: General Purpose
                    1. ~ $ 1561.65 / month
            3. Example:
                1. Region: West India
                2. Instance Type: Single Instance
                3. Instance: 24 vCore
                4. Service Tier: Business Critical
                    1. ~ $ 12,437.08 / month
            4. Example:
                1. Region: West India
                2. Instance Type: Single Instance
                3. Instance: 24 vCore
                4. Service Tier: Business Critical
                5. Reservation: 3 year reserved
                    1. ~ $ 9,209.91 / month
        2. Azure SQL
            1. Flavor:
                1. Azure SQL
                    1. Type:
                        1. Single Database
                            1. Purchase Model:
                                1. DTU
                                    1. A unit of compute power by Microsoft to give a kind of indication about the underlying compute power of the database
                                        1. 100 DTUs ~ 1 vCore
                                    2. Service Tier: (Here they differ in power and SLA)
                                        1. Basic
                                            1. Instance
                                                1. vCores
                                                2. DTUs
                                        2. Standard
                                            1. Instance
                                                1. vCores
                                                2. DTUs
                                        3. Premium
                                            1. Instance
                                                1. vCores
                                                2. DTUs
                                2. vCores
                                    1. Added since most people would not understand the DTU model
                                        1. More clear and understandable
                                    2. Service Tier: (Here they differ in power and SLA)
                                        1. General Purpose
                                            1. Compute Tier:
                                                1. Serverless
                                                2. Provisioned
                                                    1. Instance
                                                        1. vCores
                                                        2. DTUs
                                        2. Business Critical
                                            1. Compute Tier:
                                                1. provisioned
                                                    1. Instance
                                                        1. vCores
                                                        2. DTUs
                                        3. Hyperscale
                                            1. Compute Tier:
                                                1. provisioned
                                                    1. Instance

1. vCores
2. DTUs

2. Elastic Pool
    1. Purchase Model:
        1. DTU
            1. Service Tier:
                1. Basic
                    1. Instance (vCores/DTUs)
                        1. vCores/DTUs
                2. Standard
                    1. Instance (vCores/DTUs)
                        1. vCores/DTUs
                3. Premium
                    1. Instance (vCores/DTUs)
                        1. vCores/DTUs
        2. vCores
            1. Service Tier:
                1. General Purpose
                    1. Instance (vCores/DTUs)
                        1. vCores/DTUs
                2. Business Critial
                    1. Instance (vCores/DTUs)
                        1. vCores/DTUs

3. Examples:

    1. Azure SQL Database
        1. Service Tier: Standard
        2. Performance Level: 20 DTUs, 250 GB included per DB, $0.0403 / hour
        3. Type: Single Database
        4. Purchase Model: DTU
        5. Backup Storage Tier: RA-GRS (?)
        6. Total: $29.43 / month
    2. Azure SQL Database
        1. Service Tier: Business Critical
        2. Type: Single Database
        3. Hardware Type: Gen 5
        4. Backup Storage Tier: RA-GRS
        5. Instance 12 vCore
        6. Purchase Model: vCore
        7. Total: $5,951.85 / month
    3. Azure SQL Database
        1. Service Tier: General Purpose
        2. Type: Single Database
        3. Compute Tier: Serverless
        4. Purchase Model: vCore
        5. Billed vCores:
            1. Maximum vCores: 8
            2. Minimum vCores: 2
        6. Total: $1,552.75 / month (highly inacurate - we cannot know upfront how much we will end up paying with serverless because it depends on the usage pattern of the database)

4. Use database that is the right fit to avoid paying too much money on a database

Which Azure SQL to Choose?

1. Questions to ask:
    1. Are you migrating an on-prem SQL?
        1. If yes: go for Managed instance (best compatibility with on-prem SQL server)
    2. Do you need multiple, mostly low-utilization DBs?
        1. If yes: go with Elastic Pool (which is heavily optimized for this scenario with multiple low-utilization databases on a single server)
    3. All other cases: Azure SQL

Creation and Connection to Azure SQL

1. Connecting from Azure SQL from VS Code:
    1. There is a bug in the latest version of MS SQL Extension for VS Code - it prevents connections defined using the connection string
        1. Solution: Copy the server name of Azure SQL (on Overview page)
            1. paste it into connection field of the extension
            2. Follow the instructions:
                1. Type database name
                2. Type username
                3. Type password
2. In portal, search for SQL
    1. SQL databases
        1.
            - Create
            1. Resource group: readit-app-rg
            2. Database name: readit-db
            3. Server: (underlying infrastructure)
                1. Create new
                    1. Server name: readit-db-server-mutthoju (unique across Azure)
                    2. Location: West Europe
                    3. Authentication:
                        1. Authentication method: use SQL authentication
                        2. Server admin login: mutthojuadmin
                        3. Password: Any

4. Confirm password

4. OK
4. Want to use SQL elastic pool? No
5. Workload environment
    1. Development (size)
6. Compute + storage:
    1. Click Configure database
        1. Service tier: Basic
            1. 5 DTUs
            2. 2 GB storage
            3. Price: $4.9 USD / month
        2. Apply
7. Backup storage redundancy:
    1. Geo-redundant backup storage
8. Networking
    1. Connectivity method: No access (which networks or endpoints can be used to connect)
    2. Minimum TLS version: TLS 1.2 (never select a lower version)
9. Review + create
10. Create

3. Click Go to Resource
    1. Status: Online
    2. Server name
    3. Look into database and query:
        1. VS Code
            1. Install SQL Server Extension
                1. Extensions
                2. Search sql server
                    1. SQL Server (mssql)
                    2. Install
            2. Select the Extension
                1. Connections: Shows various connections to the DB
                    1.
                        - Add Connection
                    1. Azure Portal
                        1. Connection strings
                            1. ADO.NET connection string
                                1. Copy
                    2. Paste the connection string
                        1. Parameters
                            1. Server Name
                            2. Port: 1433 (standard)
                            3. DB Name
                            4. Security parameters
                                1. user Id
                                2. type password
                    3. Hit Enter
                    4. Display Name: Readit DB
                    5. Error:
                        1. Client IP address does not have access to server
                            1. Nothing is open by default
                            2. Azure Portal:
                                1. Overview
                                    1. Set server firewall
                                        1. Public network access: Selected networks
                                            1. Save
                                        2. Click: Add your client (IPV4 address)
                                            1. Automatically adds
                                            2. Save
                    6. VS Code
                        1. Add connection
                        2. Paste connection string
                        3. Type Password
                        4. Readit DB
                        5. We see the directories
                2. Query History: Filled with quries done on the DB

Connecting the Catalog to the Database

1. Open Catalog App in VS Code

    1. appsettings.json
        1. "BooksDB": ""
    2. Retry as Admin
        1. Yes
    3. Close

2. Startup.cs

```
private bool useInMemory = false;
```

3. Terminal:

```
dotnet tool install --global dotnet-ef # installs options that we can use with entity framework library of .Net (library that interacts with db

dotnet ef migrations add InitialCreate # Statements are created to run against the DB for creation of required tables in the DB
# A migrations folder is Created
```

```
# Command for creation of database with books structure

dotnet ef database update # for creation of tables in Azure SQL DB
```

1. Open SQL Server extension
    1. Open Tables
        1. Books
            1. Right click and Select top 1000
                1. Runs a SQL query and selects top 1000 of the table
2. Run the app locally (F5)
    1. Click Load Books to DB
        1. The books will be in Azure SQL DB
3. Click execute query in VS Code
    1. Books are stored in DB
4. Terminal:

    ```
    dotnet publish -o publish
    ```

    1. Go to catalog-vm
        1. Start
    2. Connect
        1. RDP
            1. Download RDP File
            2. Connect
                1. Takes too much time and throws an error message
                    1. Networking
                        1. RDP:
    3. Connect
        1. Enter password
        2. Close server manager
        3. Search for IIS
            1. catalog-vm
                1. sites
                    1. catalog
                        1. Stop (to update)
        4. Open windows explorer
            1. C:/catalog
5. VS Code
    1. Right click on **publish** folder
        1. Reveal in file explorer
        2. Open publish
            1. Copy all the content
    2. Go to catalog Machine
        1. ctrl + v
        2. Override all the files
6. To to IIS
    1. Start

4. Portal:
    1. Go to application gateway
    2. Copy public IP
    3. :8080
        1. App cannot connect to the database
            1. Solution: Open firewall rule
                1. Copy the client IP
                2. SQL database
                    1. Set server firewall
                        1. Paste IP address
                        2. Rule name: catalog-vm
                        3. Save
    4. Open :8080 in browser
        1. It gets connected

Securing the Database Connection

1. We need to connect Azure SQL to catalog VM through service-endpoint
    1. Portal:
        1. Search for catalog-vm
            1. catalog-vm
                1. Networking
                    1. Click Name of network
                        1. Service endpoints
                            1. Add a service endpoint
                            2. Service: Microsoft Sql
                            3. Subnets: default (where catalog vm is located)
                            4. Add

2. Go to Database:
    1. Firewall:
        1. Click +Add existing virtual network
            1. Name: catalog-vnet
            2. Virtual Network: readit-app-vnet
            3. Subnet: default (where catalog-vm is located)
                1. Service endpoint status:
                    1. Enabled
            4. OK
        2. Remove Catalog-VM rule:
            1. Has IP address of catalog-vm
                1. Bad pattern to leave un-used open ports in the Firewall
        3. Save
3. Go to Catalog page:
    1. Refersh
        1. VM can still connect to the database
            1. We have connected using service-endpoint

Connecting the Inventory to the Database

1. Go to VS Code
    1. Close catalog-app
    2. Open with inventory project
        1. Pages > index.cshtml.cs

            1. Uncomment

```
try {
    books = _context.Books.ToList();
}
catch (Exception ex) {
    ViewData["Error"] = ex.Message;
}
```

                1. Go to terminal:

```
dotnet publish -o publish
```

                2. Right click on `publish`:

                    1. Ctrl + Shift + P - open cmmand pallette
                        1. **azure:SignIn**
                    2.

A Quick Reminder...

Cosmos DB

1. It is a fully managed NoSQL database

2. Pros:

    1. Amazing performance
        1. < 10ms for 99% of operations
    2. Globally distributed
        1. We can replicate data across multiple regions
    3. It has fully automatic management - updates, scaling, fixes, etc. (we don't have to do anything to make sure they happen)
    4. It exposes multiple APIs
        1. SQL
        2. Mongo
        3. Gremlin
        4. Azure Table
        5. Cassandra (per account)
            1. We can choose the API during creation, how we want the database to be

3. It is a hierarchical database:

```
Database Accounts (container for cosmos databases)
    Databases
        Containers (like tables)
            Items: JSON documents (all items don't need to have the same structure)
            Stored procedures
            user-defined functions
            triggers
            conflicts
            merge precedures
```

4. Cosmos DB Availability:

    1. Can be distributed across many regions (configurable)
    2. API automatically picks the closest one (we don't have to worry about which region)
    3. When using write replication SLA is 99.999% (write operation is replicated automatically across multiple regions)
        1. It has the highest SLA in Azure
    4. Availability managed automatically, no code changes are required if we decide to replicate the data to more regions
        1. It is a config only action (no impact on code)

5. Cosmos DB Backup:

1. Full backup every 1-24 hours (default is 4) (configurable)
2. Retention period 20 - 30 days (default is 30)

6. Cosmos DB Security:

1. IP Firewall rules
2. Service Endpoints
3. Private Endpoints
4. Azure AD Authentication
5. Secure Communication (TLS)
6. Data Encrypted by default

7. Cosmos DB Partitions:

1. Data items are divided into partitions
    1. Logical groups of items based on a specific property
        1. Example: In a cars database, the Model can be a partition property
            1. Partition #1: Model - Mercedes
                1. Contains all the Mercedes cars
            2. Partition #2: Model - Chevrolet
            3. Partition #3: Model - Alfa Romeo
            4. ...
            5. Partition #n: Model - Toyota
2. Partitions are the basic scale unit in Cosmos DB
    1. When we auto-scale (scale out or in), what are scaled are partitions (we add and remove partitions)
    2. Distribution and scale are per partitions
    3. Make sure items are divided as evenly as possible
        1. We want to make sure the number of items in each partition are as similar as possible
    4. It's extremely important to select the right partition property
        1. **This property cannot be modified**
            1. Put a lot of thinking in selecting the right partition property

SQL vs NoSQL Databases

1. Data Store:
    1. One of the more important decisions in project design (this is where the precious data is stored)
    2. How to do that?
        1. SQL vs NoSQL (concepts)

            1. SQL Database:

                1. Are relational databases (traditional)
                2. Popular products: Microsoft SQL Server, Oracle, MySQL
                3. Common characteristics
                    1. They store data in tables
                    2. Each table stores a specific type of entity
                    3. Each table has a concrete set of columns which represent the properties of the entities
                    4. Example:
                        1. Order entity:
                            1. Column Name
                                1. OrderId: Numeric, Non Nullable
                                2. OrderDate: DateTime, Non Nullable
                                3. CustomerId: Numeric, Non Nullable
                                4. DeliveryAddress: String, Non Nullable
                    5. Tables can have relationships with each other (hence the name relational)
                        1. Order has Order Items
                            1. Each order item relates to a specific order
                                1. Relationship is established using order id field in orders table
                    6. SQL Database - Transactions
                        1. Atomic set of actions
                            1. Either executes all the actions or executes none of them (only part of the transaction is not executed)
                                1. Example: If customer places an order, the items in stock must decrease by one (it should not happen that order was placed but the stock was not updated)
                        2. ACID: Transactions are defined with ACID acronym
                            1. Atomicity
                            2. Consistency
                            3. Isolation
                            4. Durability
                        3. A DB that support the above can claim to support transactions
                            1. Transactions are one of the most important capabilities of relational databases
                                1. Naturally they are widely used
                    7. SQL Database - Querying
                        1. Using SQL - Structured Query Language

                            1. Very mature

                            2. It allows querying and modifying data in an easy to understand language and is considered the defacto standard to access data in a relational database

                            3. Example:

                            ```
                            Select OrderID, OrderDate, CustoemrId, DeliveryAddress
                            From Orders
                            Where OrderDate > '01/01/2018'
                            ```

            2. NoSQL

                1. Opposite of SQL databases

1. Limitations of SQL databases
    1. Performance: A schema is maintained for each record
        1. Forced transactions
    2. Size
2. The problems that NoSQL movement is trying to solve (performance, size)
    1. Emphasis on scale and performance
        1. NoSQL DBs can become very huge and can get distributed to many servers
            1. Example: Bydu - 300 TB on MongoDB
            2. Example: Billions of entities in NoSQL DBs
3. Differences:
    1. NoSQL DBs are in general schema-less
        1. Do not force any Schema
        2. They can store completely different entities with completely different fields in the same table
    2. Data is usually stored in JSON format
        1. JSON is a fully flexible format
            1. We are not limited to a specific field or size
                1. **Good if we are storing semi-structured or un-structured data which do not have a concrete schema**
    3. Transactions:
        1. It is varied
            1. Most DBs support **eventual consistency**
                1. DB guarantees that the action will be performed but it will not guarantee when exactly it will be performed
                    1. It is usually seconds (not minutes)
                2. Data can be temporarily inconsistent
        2. NoSQL: Transactions:
            1. Why no ACID?
                1. Size & Performance - compromized by ACID (RDBMS are blocked from reaching large size and high performance due to transactions)
                2. NoSQL supports only a part of the ACID definition
                    1. Each DB selects its own version of transaction support
                        1. We need to look closely at the transactional support of the NoSQL DB we are going to work with
        3. NoSQL: Querying:
            1. There is no standard in accessing NoSQL DB
                1. Each DB has its own language and driver and requires its own learning curve
            2. It can be frustrating

2. Data Store - Summary:
    1. SQL Databases:
        1. Not Huge (10s of TB)
        2. Structured Data
        3. Transactions (ACID) are important
        4. Compatible with development platform
            1. Any SQL DB is fine
    2. NoSQL Databases:
        1. Huge [data]
        2. Un- or Semi-Structured
        3. Choices:
            1. MongoDB - most popular at this time
                1. It has support from various software vendors
    3. The table is handy while deciding on which DB to choose
    4. Lately, the line between SQL and NoSQL databases has started to blur
        1. We have capabilities leaked between the two types
            1. Traditionally NoSQL DBs were great at querying JSON documents but now SQL DBs like SQL Server and PostgreSQL have great JSON querying capabilities
            2. MongoDB has added full ACID support
                1. It wont impact performance

Cosmos DB Consistency Levels

1. Traditionally:
    1. With relational DB - We have strong consistency:
        1. Call returns only after successful commit in all replicas (High availability)
    2. With NoSQL DB - Eventual consistency: Call returns immediately, commit in replicas happens later (Provides with low latency but data is not consistent)
    3. High Availability != Low Latency
        1. We need to select between them
            1. If we prefer high availability, we need to go with RDBMS
            2. If we prefer low latency, we will go for NoSQL DBs
    4. Cosmos DB offer five consistency levels:
        1. Strong (<= As in regular relational DB, a transaction is committed only if all the data is replicated to all the replicas)
        2. Bounded Staleness
        3. Session
        4. Consistent Prefix
        5. Eventual (<= As in regular NoSQL DB)
2. The basic question with consistency is:
    1. If region X updates an item, and region Y reads this item, which version will it get?
        1. Strong:
            1. Region Y will get the last version of the item updated in region X
                1. Versions of items in both regions will always be identical
            2. Use-case: Used for mission critical data
                1. When we must make sure that data is replicated immediately across and no different versions of the same data exists in the system
            3. Timeline:
                1. Region X performs some updates of the same item
                2. Region Y will get the exact same versions in the same order
        2. Bounded Staleness:
            1. Region Y will lag behind region X by K versions or T time
                1. Region Y will not get the same version of the item but it might take some time

        2. The lag between the versions of data in Region X and Region Y is maximum k versions or T time
            1. Example: k = 2, If Region X has version i, then Region Y will have a version not lower than (i - 2)
        2. It keeps the order of the versions
            1. Order of versions that Region Y will see will be identical to the order of versions updated by Region X
        3. The consistency level is used for low write latency and when order is important (write performance is important but read performance is not that important)
        4. Timeline:
            1. Region X: u1 -> u2 -> u3 -> u4
            2. Region Y: u1 -> u2 -> u3 -> u4
                1. k = 2 in the above example
                2. When Region X performs u3, then Region Y will get u1 in the example etc.
    3. Session:
        1. In a client session - Strong consistency
            1. From the point of view of the client that performs the actual write, then we have strong consistency
                1. If the client writes to Region X and then reads from Region Y, then it will see the same version
                2. However, the other clients use - Consistent Prefix (sometimes Eventual) consistency level (next)
        2. It has two consistency levels
            1. One for the client in session has strong consistency level
            2. Other clients will have the Consistent Prefix consistency level
        3. Consistent Prefix:
            1. It keeps the order of the versions
                1. But no guarantee of the lag size (as opposed to Bounded Context)
                    1. k is not defined
            2. Should be used for low write latency and when reads are infrequent
                1. When write operations are top priority and there aren't many reads to the items
        4. Timeline:
            1. Region X: u1 -> u2 -> u3 -> u4
            2. Region Y: u1 ---> u2 -> u3 --> u4
    4. Eventual:
        1. There is no order guarantee
        2. There is no guarantee of the lag size (as opposed to Bounded Context)
        3. It can be used for count of Retweets, Likes, etc.
            1. No problem if numbers will fluctuate a little
        4. Timeline:
            1. Region X: u1 -> u2 -> u3 -> u4
            2. Region Y: u2 --------> u4 --> u3 -> u1
        5. Use-Case: Write performance is extremely important and we don't care about the order of the versions
3. Cosmos DB Consistency Levels:
    1. Configured at the account level
        1. But consistency can be relaxed on the request level
            1. We can choose a stronger consistency level at the account level but use a weaker consistency level at the account level
                1. Strong -> Bounded Staleness (say)
4. Cosmos DB offers a variety of consistency levels that are not offered by most other databases
    1. Ensure we are using the right level

Cosmos DB Pricing

1. Pricing:
    1. Based on RU/s
        1. Request Unit per Second:
            1. RU: Reading an item of size 1KB
                1. Getting an item by its ID, not by query
                    1. Example: item with ID = 123, size = 1KB, reading the item by its ID is considered as one read action
                2. 1RU/s = Read 1 item of 1KB size in 1 sec using its ID
                3. 400 RU/s = Read 400 items of 1KB in 1 sec
                4. Update, Delete, Insert, Query - cost more than 1 RU
                5. **We can see the actual RU consumed in the response heaer of the results**
                    1. We can do some estimation about how many RUs we are using
        2. Also based on
            1. Operation type:
                1. Provisioned
                2. Auto Scale
                3. Serverless
            2. Write Regions:
                1. How many regions we are going to write simultaneously
            3. No of provisioned RU/s
                1. If we provisioned 400 RU/s, then x is what are are going to pay
        3. Three types of database operations:
            1. Provisioned: Predefined number of RU/s (can be changed manually later)
                1. This offers reserved capacity up to 65% discount
                    1. This is similar to pre-defining the size of Azure DB
            2. Auto Scale: With this, we set the maximum RU/s. Cosmos scales up to this number (begins with a lower number)
                1. This is good for unpredictable loads
                    1. If we have no idea about what load the DB needs to handle, this is a good option
            3. Serverless: Pay for what you use
                1. In Preview
                2. No SLA
2. Example:
    1. DB Operations: Provisioned (We set DB size during creation)
    2. RU/s: 4 x 100
    3. Monthly price: $23.36 / month
3. Example:
    1. DB Operations: Standard Provisioned Throughput (Manual)
        1. Multiple Region Write (Multi-Master) (HA)
    2. RU/s: 50 x 100 (upto 5000 requests/second)
    3. Monthly price: $584 / month

4. Example:
    1. DB Operations: Autoscale
        1. Single Region Write (Single-Master)
    2. RU/s: 10 x 100
    3. Monthly price: $876 / month (max)
        1. It will probably be much lower

Creation and Usage of Cosmos DB

1. New Cosmos Account:
    1. Search: Cosmos
        1.
            - Create
        1. Which API best suits your workload?
            1. Core (SQL) - Recommended - Create
                1. Resource Group: readit-app-rg
                2. Account Name: readit-cosmos-db-mutthoju (unique)
                3. Location: West Europe
                4. Capacity mode: Provisioned
                5. Apply Free Tier Discount: Apply
                6. Limit total account throughput: Limit the total amount of throughput that can be provisioned on this account
                    1. We will not be able to provision DBs and containers which have more RUs than what is allowed under the free account
                    2. Uncheck it (because it can cause problems when we deploy database and container)
                7. Networking:
                    1. We can define connectivity method (who can connect to the database)
                        1. All networks [can connect] - select
                        2. Public endpoint [selected network]
                        3. Private endpoint
                8. Backup Policy
                    1. Every 240 minutes
                    2. Retention period: 8 hours
                    3. Backup storage redundancy: Geo-redundant backup storage (backup is not stored in the same region)
                9. Review + Create
                    1. Estimated Account Creation Time: 2 minutes
                10. Create
        2. Go to resource
            1. Two locations (multi-region deployment)
                1. Read Location (one location)
                2. Write Location (another location)
            2. URI of Cosmos DB
            3. Free Tier Discount is opted in
            4. Capacity mode: Provisioned throughput
            5. Firewall and Virtual Networks:
                1. Allow access from: All networks
                    1. Selected networks: Which IP addresses can connect to Cosmos (additional protection)
        3. Data Explorer
            1. New Container
                1. Create new: readit-orders
                2. Database throughput: Manual
                3. Estimate your required RU/s: 400 (lowest number possible)
                4. Container id: orders
                5. Indexing: Automatic
                6. Partition key: /priority
                    1. Priority will vary quite evenly across products
                7. OK
            2. Open DATA > readit-orders > orders (container)

                1. items

                    1. We can add items:
                        1. New Item: Paste contents of attached file
                        2. Save
                        3. Shows in the list of items
                        4. Cosmos added various fields in addition to fields defined by us
                            1. Helps Cosmos manage attachments and indexing
                        5. New Item: Paste the content of second file
                        6. Save
                        7. The items have different priority
                            1. 2 and 1

                2. New SQL Query button:

                    ```
                    Select * From orders o
                    Where o.total > 90
                    ```

                    1. Execute

                3. New SQL Query button:

                    ```
                    Select o.orderId From orders o
                    Where o.total > 90
                    ```

                    1. Similar to RDBMS

                4. New SQL Query button:

```
Select * From orders o
Where exists (
    Select value n
    From n in o.items
    Where n.name = 'Rama II'
)
```

1. It can query JSON like a table is quried
2. Hierarchical queries are supported

5. Update:
    1. Select first item
        1. modify orderDate: "01/08/..."
        2. Update
    2. Re-select the item to test it
    3. Field name can also be changed:
        1. Change "orderDate" to "date"
            2 Update
    4. The field names can be different in different items
        1. Not all elements have the same schema

6. priority:
    1. If we change the name to priority 1
        1. Error!
            1. We cannot modify the name since it is used for partitioning mechanism of Cosmos

4. Keys (similar to connection string)
    1. URI, Primary Key, Secondary Key, Primary Connection String, Secondary Connection String
        1. If we are going to replace one key, we can switch to the second key and then change the first key offline and then switch back to the first key
            1. Avoids downtime

Connecting the Orders Function to Cosmos DB

1. We want to connect the orders function to the orders database
2. Open VS Code:
    1. Orders directory
        1. ProcessOrderCosmos.cs
            1. Uncomment the two lines
                1. First line defines the `out` to be an Order object and it is going to be saved in `readit-orders` database in `orders` collection name (container)
                2. Connection string: `CosmosDBConnection`
                3. It goes to request body and deserializes JSON to `order` object
                    1. It is saved in the DB
        2. local_settings.json

```
"CosmosDBConnection": "<paste-cosmos-db-connection-sring-here>"
```

            1. Copy Primary connection string from Portal
        3. Test it locally:
            1. F5
            2. Open PostMan
                1. Copy the URL and paste here
                2. Verb: POST
                3. Body:
                    1. order_sample.json contents
            3. Send
            4. Check the log of the function
            5. Go to Portal > Data Explorer
                1. ...
                    1. Items: New order appears
            6. Change price from 55 to 99
            7. Send
            8. Refresh to see another item
        4. Deploy the updated function to Azure
            1. Go to Azure extension
            2. Workspace > Local Project > Functions
                1. Click upload
                    1. Deploy to Function App
                    2. Select function app
                    3. Deploy
            3. Upload settings (contains connection string)
            4. Yes to all (overwrite)
        5. Testing:
            1. Portal: Function App
                1. readitfunctionappmutthoju
                    1. Functions
                        1. ProcessOrderCosmos

                1. Get Function URL
                    1. Copy the URL
            2. PostMan:
                1. POST:
                2. Book name changed: "Qur'an in Amazon"
                3. Send
                    1. 204 status
            3. Open Cosmos DB
                1. We have one more item
                    1. Open to check the name of the book

        6. Open: ProcessOrderCosmos function

            1. Code + Test
                1. Logs console: Switch from App Insights logs to Filesystem logs
            2. Change the order in Postman: Add 2 at the end of the book name
                1. The log shows the name of the book
            3. Go back to Cosmos and refresh
                1. One more order was added
3. Flow:
    1. Postman sends request to function in the cloud
    2. Function stores order in the Cosmos DB

Cosmos DB Tips and Tricks

1. To modify the consistency level in Cosmos DB account:
    1. Click Default consistency item in the menu
2. To see how many RU/s were consumed by a query, we can find the number in Query Status tab in Data Explorer

Azure MySQL

1. A managed MySQL on Azure
    1. It works like any other MySQL database using the same tools
    2. It has great compatibility with on-prem MySQL databases
2. It offers built-in security, backups, availability, and more
    1. Expected from managed database
3. Security:
    1. IP Firewall Rules
    2. Service Endpoints
    3. Private Endpoints
    4. Regular & Azure AD Authentication
    5. Secure communication (TLS)
    6. Data encrypted by default
4. Backup:
    1. Depends on Service Tier:
        1. Basic - Full backup: daily
        2. General Purpose up to 4 GB
            1. Full backup: once a day
            2. Differential backup: twice a day
            3. Transaction log backup: every 5 minutes
        3. General Purpose up to 16 GB:
            1. Full backup: Once created
            2. Differential backup: once a day
            3. Transactional log backup: every 5 minutes
5. Retention Period:
    1. 7 - 35 days (default is 7)
    2. No native long term retention support (as opposed to what we have in Azure SQL)
6. Availability:
    1. Backup is stored in a geo-redundant storage
        1. In a General Purpose and Memory Optimized tiers
        2. In the Basic tiers, the backup is stored locally
    2. SLA: 99.99%

Azure MySQL Pricing

1. Pricing is based on:
    1. Tier:
        1. Basic - Offers light compute and I/O performance (for dev and test)
        2. General Purpose - Most business workloads
        3. Memory Optimized - When we require in-memory performance
    2. Compute (no. of vCores)
    3. Flexible Server deployment - currently in preview, not recommended
    4. Reservations exist
        1. We can select reservation for 1 or 3 years (hefty discount on the price)
2. Examples:
    1. Tier: Basic, Compute: Gen 5, 1 vCore (modest)
        1. Price: $29.05
    2. Tier: General Purpose, Compute: Gen 5, 4 vCore, Reservation: 3 years - 53% discount will be applied
        1. Price: $304.26

Constructing and Using Azure MySQL

1. Search: MySQL
    1. Azure Database for MySQL servers
        1.
            - Create

1. Type: Flexible server
    1. Create
        1. Resource group: Create new
            1. mysql-rg
        2. Server name: mutthoju-test-mysql (unique)
        3. Region: West Europe
        4. MySQL version: 5.7
        5. Workload type: type of usage we are going to use with the database
            1. For small or medium size databases
            2. Tier 1 Business Critical Workloads
                1. Compute + storage: 2 vCores, 16 GB RAM, 128 GB storage, 2000 IOPS
            3. For development or hobby projects (select this)
                1. Compute + storage: 1 vCores, 2 GB RAM, 20 GB storage, 360 IOPS (modest)
                2. Geo redundancy: Disabled
                3. Configure server
                    1. We can fine-tune the configuration of the server
                        1. Compute tier: Burstable (basic)
                        2. Compute size: Standard_B1ms (default)
                        3. Storage size (in GiB)
                            1. min: 20 GB
                        4. ...
                        5. Backups:
                            1. Backup retention period (in days): 7 days (default)
                            2. Backup Redundancy Options: Local Redundant
                                1. Geo-Redundant: Check "Recover from regional outage or disaster"
                                    1. Backup data is going to be stored in the paired region
                    2. Price:
                        1. $17.27 / month
                    3. Save
                4. Authentication:
                    1. Authentication method:
                        1. MySQL authentication-only (select)
                            1. Allows us to authenticate with user-name and password
                        2. Azure Active Directory authentication only
                            1. AD only
                        3. MySQL and Azure Active Directory authentication
                            1. For both
                    2. Admin username: mutthojuadmin
                    3. Password:
                    4. Confirm password:
                5. Networking:
                    1. Adding firewall rule
                    2. Click: + Add current client IP address
                        1. A new rule gets created
                6. Review + Create
                7. Create
    2. Go to resource
        1. Server name: .mysql.database.azure.com
        2. Server admin login name:
        3. Configuration: VM underlying the db
        4. Networking:
            1. Firewall rule is in place
        5. Compute + storage
            1. Configuration that we saw
        6. Connect
            1. Ways to connect to the DB
                1. Connect from your app
                2. MySQL Workbench
                    1. Install:
                        1. Download MySQL Workbench
                        2. Web installer (smaller)
                        3. Client only
                        4. Next
                        5. ...
                        6. Execute
                        7. Execute
                        8. Next
                        9. Cancel
                        10. Yes
                    2. Run:
                        1. Start
                        2. MySQL Connections +
                            1. Connection Name: MySQL Azure
                            2. Expand the steps in portal
                            3. Copy the server name and paste in the hostname field
                            4. Copy and paste username
                            5. Test Connection
                            6. Enter password
                            7. OK
                    3. Work with database:
                        1. New schema - a database

1. New schema button
2. Name: my-items
3. Apply
4. Apply

2. Schemas
    1. Right click on Tables
    2. Create Table
    3. Table Name: items
    4. Columns:
        1. item_id: INT: NN
        2. name: VARCHAR(45): NN
        3. price: DECIMAL(2)
    5. Apply
    6. Apply
3. Finish
4. Expand Tables
    1. Right click on Items table
        1. Seelct rows limit 1000 items
        2. Execute
5. Click: Create new SQL tab icon

```
INSERT INTO items VALUES (2, 'Book', 12.99);
```

    1. Double click on the schema to select it
    2. Execute the query
6. Run:

```
SELECT * FROM items;
```

3. Go to Portal:
    1. Databases
    1. Newly created database appears
    1. We cannot run SQL statements in portal
4. Delete resource group

Azure PostgreSQL

1. It is a managed PostgreSQL on Azure
2. It works like any other PostgreSQL database using the same tools
3. It has a great compatibility with on-prem PostgreSQL database
4. It includes Hyperscale deployment
    1. Provides improved performance (?)
5. Offers built-in security, backups, availability, ...
6. Security:
    1. IP firewall rules
    2. Service Endpoints
    3. Private Endpoints
    4. Regular & Azure AD Authentication
    5. Secure Communication (TLS)
    6. Data encrypted by default
7. Backup:
    1. Depends on storage size:
        1. up to 4 GB:
            1. Full backup: once a week
            2. Differential backup: twice a day
            3. Transaction log backup: every 5 minutes
        2. Up to 16 GB:
            1. Full backup: Once created
            2. Differential backup: thrice a day
            3. Transaction log backup: every 5 minutes
8. Retention Period:
    1. 7 - 35 days (default 7)
    2. No native long term retention support
9. Availability:
    1. Backup is stored in a geo-redundant storage
        1. In General Purpose and Memory Optimized tiers
            1. SLA: 99.99%

Azure Storage

1. It is an object store
    1. A specialized kind of db that is used to store objects (files, documents, videos, images, ...)
        1. It is not used to store relational data (tables, records, relationships) or json documents that can be queried
2. It is massively scalable
    1. Huge storage
3. It is accessible via HTTP or HTTPS
    1. Easy to work with
4. We have client libraries for almost every language to help us access Azure storage
5. Azure storage is durable and highly available

6. Five Types of Storage:
    1. Blobs
        1. Object store
            1. Most interesting part of Azure storage
            2. Blob - Binary Large Object
            3. It is great for files, vides, documents, large texts etc.
            4. Up to 4.77 TB per file
                1. 190TB per file in Preview
            5. Extremely cost effective
                1. Cheapest data store of all
            6. Massively scalable
            7. It has great availability options
            8. It is extremely easy to use
                1. A very simple API allows access to the Azure storage
            9. It is usually used in conjunction with SQL / NoSQL database
                1. So that DBs can store the actual data of the system
                2. Blob storage stores related files such as scanned documents, vidoes, images, ...
            10. Security:
                1. IP firewall rules
                2. Service Endpoints
                3. Private Endpoints
                4. Shared Access Signatures
                5. Access Keys & Azure AD Authentication
                6. Secure communication (TLS)
                7. Data encrypted by default
            11. Structure:
                1. Account (Azure storage account: Sally)
                    1. Container (logical group of blobs: pictures, movies)
                        1. Blob (img001.jpg, img002.jpg; mov1.avi)
                2. It is similar to Cosmos DB
            12. Azure Blob Storage Redundancy
                1. 6 options:
                    1. LRS
                        1. Locally Redundant Storage - Data stored in Azure storage is synchronously copied 3 times within the same zone
                            1. Zone - a data center
                            2. Doesn't leave data center boundaries
                    2. ZRS
                        1. Zone Redundant Storage - Data is synchronously copied to 3 zones in the Region
                            1. If a region has multiple availability Zones
                            2. If a region does not have multiple availability zones, this option is not available
                    3. GRS
                        1. Geo Redundant Storage - Data is synchronously copied 3 times within the same zone, and then copied asynchronously to paired Region. Data in the secondary Region is accessible only after a failover process.
                            1. A lot of regions have a region that is defined as their pair
                            2. It is not accessible immediately after a copy in the paired region
                                1. Only after a failover is initiated
                    4. GZRS
                        1. Geo-Zone Redundant Storage - Data is synchronously copied to 3 zones in the Region. and then copied asynchronously to paired region. Data in the secondary region is accessible only after failover process
                    5. RA-GRS
                        1. Read Access-Geo Redundant Storage - Data is synchronously copied 3 times within the same zone, and then copied asynchronously to paired Region. There's a read-access to the data in the secondary region.
                            1. Data is accessible for read in the paired region even if no failover process is initiated
                    6. RA-GZRS
                        1. Read-Access-Geo-Zone Redundant Storage - Data is synchronously copied to 3 zones within the same Region, and then copied asynchronously to paired region. There's a read-access to the data in the secondary region.
                2. Azure Blogs Storage Failover:
                    1. Normal circumstances:
                        1. User works against the primary region
                            1. Storage account is: GRS/RA-GRS
                        2. Asynchronous Geo replication is taking place to the secondary region
                        3. Primary region fails
                        4. Failover is initated to the secondary region
                            1. Subsequent requests to the storage account will be automatically redirected to the secondary region after the failover completes
                    2. Failover can be initiated via:
                        1. Portal
                        2. Azure CLI
                        3. PowerShell
                3. Azure Blobs Storage Tiers:
                    1. Blobs are uploaded to one of three tiers:
                        1. Hot (Blobs used frequently)
                            1. Data that's accessed frequently
                            2. Best SLA (99.9%)
                            3. Highest storage cost
                            4. Lowest access
                            5. Examples:
                                1. Photos to display (on website)
                                2. Documents to show
                        2. Cool (For things we want to keep for a relatively long time)
                            1. Data that's accessed infrequently
                            2. Slightly lower SLA (99%)
                            3. Lower storage costs
                            4. Higher access costs
                            5. Data in cool tier must be stored for at least 30 days (or early deletion fees is applied)
                            6. Examples:
                                1. Short term backup

2. Data for future processing
3. Archive
    1. Data for archival
    2. Data is stored offline
        1. No SLA
            1. It can take hours to retrieve the data
    3. It offers the lowest storage costs
    4. It offers the highest access costs
    5. The data must be stored for at least 180 days (or else early deletion fees is applied)
    6. This tier is used to store only things that should be stored for a very long time (long term backups)
4. Azure Blobs Storage Tiers:
    1. As we go through the tiers, the storage costs go down but the access cost goes up
    2. Retrieval time is the same in Hot and Cool tiers
        1. Retrieval costs are different
    3. Archive tier does not support the following redundancy
        1. ZRS
        2. GRS
        3. RA-GRS
    4. If we use RA-G(Z)RS, SLA improves to 99.99% (Hot) and 99.9% (Cool)
    5. Tier is set at account level but can be modified per blob
    6. Moving between tiers can be automated by lifecycle rules

2. Files
    1. File shares for cloud and on-prem deployments
        1. They are related more to compute and are quite low level
3. Queues
    1. Queues
        1. Used for messaging
4. Tables
    1. NoSQL data store
        1. Similar to Cosmos DB but:
            1. Less optimized for performance
            2. Less optimized for availability
            3. It's cheaper
5. Disks
    1. Storage volues for Azure VMs
        1. They are managed by VM, we have nothing to deal with here

Azure Blob Storage Pricing

1. Based on:
    1. Redundancy option
        1. One of the six options mentioned previously
    2. Access tier
        1. Hot
        2. Cool
        3. Archive
    3. Capacity
        1. Actual storage used
2. Example: Redundancy - LRS, Access Tier: Hot, Capacity: 1000 GB
    1. $19.60 (storage), Write operations: $0.54 (100000 operations)
3. Example: Redundancy - LRS, Access Tier: Cool, Capacity: 1000 GB
    1. $10.00 (storage), Write operations: $1.00 (100000 operations)
        1. Storage cost went down but access cost went up
4. Example: Redundancy - RA-GRS, Access Tier: Cool, Capacity: 1000 GB
    1. $25.00 (storage), Write operations: $2.00 (100000 operations)

Constructing and Using Storage Account

1. Portal:
    1. Search: Storage accounts
        1. Two created Automatically
            1. For cloud shell
            2. For function app
        2.
            - Create
            1. Resource group: new
                1. storage-rg
                2. OK
            2. Storage account name: storagecoursemutthoju (unique)
            3. Location: West Europe
            4. Performance: Standard
                1. Premium: For disk
            5. Account tier: Storagev2 (standard)
            6. Replication: RA-GRS
            7. Networking:
                1. Similar to databases
            8. Data protection
                1. Deals with restore and soft delete
                    1. Soft delete is used to restore blobs deleted by mistake
            9. Review + Create
            10. Create
    2. Deployment is quite quick
    3. Overview
        1. Location: Two regions (RA-GRS)
            1. Primary region & Secondary region
            2. Both are available (Status)

2. Replication: RA-GRS
3. Containers
    1.
        - Container
    1. Name: storeimages
    2. Public access level: Private (no anonymous access is allowed to the container)
        1. To store or retrieve anything, we need to be identified
        2. Other access levels:
            1. Blob: We can anonymously access specific blobs but we cannot see the list of blobs in container
            2. Container: We have full anonymous access to the container
        3. We need to think hard when deciding access level
            1. If we want the container to store public images that we want anyone to have access to:
                1. Blob or Container
            2. If the container has to store only blobs that should be accessed from my own machines or code, then go for private
        4. Select Container public access level
    3. Create
2. Go into container
    1. Click: Upload
        1. Select a file in machine
        2. Open
        3. Expand: Advanced
            1. Acces Tier: (every blob can be uploaded to one of three tiers)
                1. Hot
        4. Upload
    2. File is uploaded to the container
    3. How to access?

        1. Go to browser and open URL:

            ```
            https://storagecoursemutthoju.blob.core.windows.net/storeimages/view1.jpeg
            ```

            1. We have defined access level to the container

        2. Go to storeimages container:

            1. Change access level
                1. Private
                2. OK

        3. Refersh the page

            1. Error: Resource was not found

Accessing Private Blobs with Keys and SAS Token

1. When the access level is private, there are three ways to access the blobs inside the private container
    1. Azure AD
    2. Access Keys
    3. Shared Access Signature
2. Using Access Keys:
    1. Go to storagecoursemutthoju storage account
    2. Click: Access keys
    3. Access keys are used as strong identifier for a client who wants to access the storage account
        1. Click Show keys
            1. Two keys
                1. We can use connection string to connect to storage account and perform actions
                    1. Connection string give full permission to do whatever we want
                    2. Connection string can be used with client libraries
                2. Why two keys?
                    1. We can change keys without downtime
                        1. We can refresh the keys
                            1. Yes
                                1. Any client that was using the pervious key will immediately stop working
                                    1. Solution: The client library can be updated to use the secondary key, we can re-generate the first key, and then move back the client library to use the first key
                3. When do we want to regenerate key?
                    1. When the key is used by a malicious third party
    4. Libraries:
        1. https://learn.microsoft.com/en-us/azure/storage/blobs/storage-quickstart-blobs-portal
            1. Menu on the side:
                1. List of languages that have client libraries
3. SAS (Shared Access Signature) to give more limited access
    1. Using it we can define a unique connection string to the storage account with a very limited access, and which can expire in a specific time
        1. Go to Shared Access Signature
            1. We can define what SAS will allow:
                1. Allowed services: Blob (uncheck the others)
                2. Allowed resource type: Object (not container or service)
                3. Allowed permissions: Read (no other permission)
                4. Blob version permissions: Uncheck Enables deletion of versions
                5. Start and Expiry date/time: 8 hours (default)
                6. Allowed IP addresses: default is all
                7. Allowed protocols: HTTPS
                8. Signing key: key1 (based on key 1)
                9. Click: Generate SAS and connection string
                    1. Three rows:
                        1. Connection string (can be used by a client library)
                        2. SAS token: What comes after the URL?

1. Concatenate the token to the URL
    1. Copy the token
    2. Paste the SAS token to the URL
    3. Hit enter
3. Blob service SAS URL: Gives full URL when using the SAS token
10. If we corrupt SAS token and refresh, error is shown that the authentication failed

2. Share the SAS token to third parties but not the key

Networking and Fail Over of Storage Account

1. The ability to know how much actual capacity we are using in the storage account:
    1. Menu:
        1. Monitoring
            1. Insights
                1. Capacity
                    1. Account
                    2. File
                    3. Queue
                    4. Table
2. Networking
    1. We can select where to allow access from
        1. All networks by default
        2. Selected networks (similar to other databases)
            1. From existing VNet or specific IP addresses
    2. Private endpoint connections
        1. Allows us to define private links between the storage account and other resources
        2.
            - Private endpoint
            1. Name: storage-plink
            2. Next: Resource
                1. Resource Types: Microsoft.Web/sites
                2. Resource: readit-inventory (app service)
                    1. We are defining a private link between the inventory app-service to the storage account
                    2. Error: App service tier does not support private endpoint
3. Geo Replication
    1. RA-GRS
        1. Primary region
        2. Secondary region
    2. Click: Prepare for failover (for testing)
        1. Explanation
            1. Secondary region is going to become primary region and redundancy is going to change to LRS
        2. Cancel
    3. Go to URL:
        1. Account name: `storagecoursemutthoju-secondary`
            1. We read it from the secondary region

CDN and Automation

1. Configure container to be public:
    1. Change access level: Container
        1. OK
2. Azure CDN
    1. A CDN is a content delivery network
        1. A network of small data-centers spread across the world
        2. It's target is to bring the static content closer to the user
            1. Example: Suppose a user is in India
                1. Some user will go to some data center in India instead of going to West Europe and back
        3. CDN networks are automatically synchronized and can be configured to use Azure storage
            1. Blobs in Azure storage will be automatically distributed to CDN networks
    2. CDN Profile: It is a configuration of CDN that sets up CDN and synchronizes storage with the storage account
        1. New endpoint:
            1. Create new
                1. images-cdn
                2. Pricing tier: StandardMicrosoft
                3. CDN endpoint name: azcoursecdnmutthoju (unique)
                4. Origin hostname: storagecoursemutthoju.blob.core.windows.net
                    1. CDN will point to this URL
                5. Create
                    1. It can take a lot of time
    3. Lifecycle Management (automated way or rule based policy for moving blobs between tiers (Hot and Cool, Cool and Archive - backup data to be kept for longer term))
        1.
            - Add a rule
            1. Rule name: move-to-archive
            2. Rule scope:
                1. Apply rule to all blobs in your storage account
            3. Next
                1. Rules are allows only based on the last modified day
            4. Last modified (also covers day of creation)
            5. More than (days ago): 60
            6. Move to archive storage
            7. Add
                1. All blobs will be moved to archive tier after 60 days and we will save costs
            8. Code View
                1. Paste the JSON attached to the lecture (Alt+Shift+F)
                    1. After 30 days, move to cool

2. After 90 days, move to archive
3. After 2555 days, delete

4. Go back to Azure CDN
    1. Endpoint is up and running
    2. Copy the hostname
        1. Replace based URL with the Azure CDN endpoint (remove SAS token)
        2. Hit enter
    3. If an error occurs saying too many redirects or object not found, wait for few more minutes and retry

Constructing Storage Account for the ReadIt App

1. Remove the resource group the storage account is in `storage-rg`
2. New storage account:
    1.
        - Create
        1. Resource group: readit-app-rg
        2. Storage account name: ordersreadit
        3. Location: West Europe
        4. Review + Create
        5. Create
    2. Go to resource
    3.
        - Container
        1. Name: neworders
        2. Public access level: Private
    4. Create
3. The storage account will contain new orders that should be processed by the function app

Azure Storage Explorer Name Change

1. Storage Explorer: Allows browing storage account contents
2. Storage Explorer (Preview): Browser-based tool in portal
    1. Now Storage Explorer

Azure Storage Explorer

1. Storage explorer is a desktop tool that provides access to storage accounts and enables running a lot of actions with the storage account
    1. Go to: https://azure.microsoft.com/en-us/features/storage-explorer
    2. Download
    3. Install
    4. Install with Default options
2. Launch Microsoft Azure Storage Explorer
    1. Connect to Azure storage:
        1. Add an Azure Account
        2. Next
        3. Sign in to our account
        4. Apply
    2. Storage Accounts
        1. click: ordersreadit (shows properties)
        2. Open it
            1. Blob Containers
                1. neworders
                    1. Upload
                        1. Upload files
                            1. Select a file to upload
                        2. Target Access Tier: Hot (default)
                        3. Upload
                    2. Double click the File
                        1. It will download and open
                        3.Click: Delete
                        2. Yes
3. Portal:
    1. Storage Explorer (preview)
        1. UI is similar to the Storage Explorer Desktop app
            1. We can upload, delete etc.

Azure Redis

1. It is managed Redis on Azure
2. Redis:
    1. It provides lightning-fast in-memory, distributed cache
    2. It is the de-facto distributed cache currently in the industry
    3. It is great for short-lived, frequently accessed data
        1. i.e. shopping cart, stock quotes
    4. It is fully compatible with OSS Redis (community edition) and Enterprise Redis - it depends on service tiers we are going to use
3. Security:
    1. IP firewall rules
    2. Service endpoints
    3. Private endpoints
    4. Secure communication (TLS)
        1. No encrypted data because nothing is stored on the disk
4. Azure Redis Service Tiers:
    1. Basic
        1. It is based on a single VM
        2. It has no SLA
        3. It has no distribution (only one machine)

4. It is good for dev/test
2. Standard
    1. It is based on two VMs
    2. VMs are Replicated
    3. SLA - Up to 99.9%
3. Premium
    1. High-performance
    2. Better throughput
    3. Low latency
    4. SLA - 99.95%
4. Enterprise
    1. Based on Redis Enterprise
    2. It offers additional features:
        1. RediSearch
        2. RedisBloom
        3. ...
    3. SLA - Up to 99.99%
5. Enterprise Flash
    1. It uses non-volatile memory
        1. It reduces storage cost
    2. SLA - Up to 99.99%
5. Most clients will go for standard or premium (depending on the exact characteristics required)

Azure Redis Pricing

1. Pricing is based on:
    1. Tier
    2. Memory (consumed)
2. Examples:
    1. Tier: Basic, 250 MB Cache: $16.06 / month
    2. Tier: Premium, 26624 MB Cache: $1,619.14 / month
3. We have the option for reservation:
    1. 3 years - 55% discount

Constructing Redis and Connecting the Catalog

1. We will Redis for Shopping Cart data
2. VMs:
    1. catalog-vm
        1. Start
3. Search: Redis
    1. Azure Cache for Redis
        1.
            - Create
        1. Resource group: readit-app-rg
        2. DNS name: readitredismutthoju
        3. Location: West Europe
        4. Cache type: Basic C0 (smallest instance type)
        5. Networking:
            1. Public Endpoint
        6. Review + Create
        7. Create
            1. It can take a lot of time
    2. Go to resource
    2. Host name: .redis.cache.windows.net
    3. Status: Basic 250 MB
    4. To scale:
        1. Scale
            1. Select a different instance type
    5. Advanced settings:
        1. Minimum TLS version: 1.2 (1.0, 1.1 are deprecated and should not be used)
    6. Save
4. Connect Redis instance to catalog project: (Redis is used for shopping cart, when we add items to shpping cart, they will be stored in Redis)
    1. Open Catalog project in VS Code:

        1. appsettings.json

        ```
        "Redis": {
            "ConnectionString": "<primary-key>@<hostname>:[<port>]?ssl=true"
        }
        ```

            1. Portal:
                1. Access keys:
                    1. Copy Primary key
                    2. Paste in VS Code

        2. indexcshtml.cs

            1. Uncomment the lines of code that use Redis
                1. Shopping cart info from Redis
                2. Adding products to shopping cart

        3. Testing:

            1. F5
                1. Error: Cannot connect to SQL server
                    1. Fix: Copy IP address
                    2. Portal:

                                    1. SQL databases
                                        1. readit-db
                                            1. Set server firewall
                                                1. Add client IP
                                                2. Save
                                    3. Refresh the page
                            2. Click: Add to shopping cart
                                1. 1 Item in the shopping cart (connection to Redis is working)
                                    1. Page got the number of items when it queried Redis
                            3. Click: Add to shopping cart
    5. Publish catalog to catalog VM
        1. Go to terminal
            1. `dotnet publish -o publish`
        2. Portal:
            1. catalog-vm
                1. Connect:
                    1. Download the RDP file
                        1. Error: NSG rule needs to be updated
                            1. Networking
                                1. Update the IP
                            2. Save
                    2. Connect again with RDP
                        1. username, password
                        2. OK
                        3. Yes
                    3. Open IIS
                        1. Stop catalog website
                        2. Open folder containing catalog files
                            1. C:/catalog
                            2. VS code:
                                1. Right click on publish folder
                                2. Reveal in file explorer
                                3. Copy all the contents
                                4. Paste files in VM catalog folder
                                5. Replace all
                        3. Start catalog website
        3. Testing:
            1. Go to application gateway
                1. readit-app-gw
                    1. Copy public IP
                    2. Open the :8080 in browser

Connecting the Shopping Cart to Redis

Current Architecture

1. Major differences:
    1. Addition of databases
        1. Azure SQL connected to catalog app, inventory app, cart app
        2. Redis connected to the catalog app, cart app
            1. Contains data about shopping cart
        3. Cosmos DB connected to function app
            1. It stores orders to handle
2. Are we done?
    1. No
        1. The orders function is publicly available and synchronous (not optimal)
            1. We don't want anyone having the orders function URL will be able to add orders
        2. The inventory page is open for everyone
            1. Only the store employees should have access to the page to manage the inventory of the store
        3. We don't really know how the app is functioning
            1. If there will be a problem, we will have no idea until we get angry phone calls from customers
        4. The website is not redundant - what happens if the whole region goes down?
            1. Do we want website to go down?
            2. Do we want it to function even if a region is fully shutdown

How to Select Data Store Solution

1. Data Type:
    1. Relational
        1. Used for: Structured data (Reational)
        2. Examples:
            1. Items in store
            2. Demographic data
        3. Options in Azure:
            1. Azure SQL (best redundancy, best SLA)
            2. MySQL
            3. PostgreSQL
    2. NoSQL
        1. Used for: Semi-structured data
        2. Examples:
            1. Reviews
            2. Log records
            3. When flexibility is required
        3. Options in Azure:
            1. Cosmos DB
                1. With the following API:

            1. SQL
            2. Mongo
            3. Azure Table
        3. Graph
            1. Used for: Data representing relationships
            2. Examples:
                1. Family tree
            3. Options in Azure:
                1. Cosmos DB (with Gremlin API)
        4. Blob
            1. Used for:
                1. Files
                2. Videos
                3. Docs
            2. Examples: Items' photos
            3. Options in Azure: Azure blob storage
    2. Keep the table when making decisions

Messaging in Azure

Messaging in Azure

1. It is one of the most important aspects of every system design
    1. Azure has a **comprehensive offering of messaging Services**
2. Messaging is extremely important aspect of Software Architecture
    1. It is widely used (especially between services)
3. Must be able to handle load, throughout, and have great latency
4. It is a core part of every microservices architecture
5. Azure has **4 fully managed messaging services**
    1. Storage Queue
    2. Service Bus
    3. Events Grid
    4. Event Hubs

Storage Queue

1. It is part of Azure Storage Account
2. It is the simplest queue Implementation
    1. Components:
        1. Creation of queue
        2. Send message (into the queue)
        3. Receive Message
3. No special pricing for queue
    1. Included in storage account
4. Same availability as Storage Account (Blob storage)
5. Performance:
    1. For Requests of 1KB of size:
        1. 20K messages/second/account
        2. 2K messages/second/queue
    2. Max message size in a storage queue: 64 KB
6. Architecture:

```
Storage Account (myaccount)
    Queues     (incoming-orders, outgoing-orders)
        Messages
            ((IN) "01-2345-678", (IN) "90-1234-567", (OUT) "98-7654-321")
```

7. Development: Against storage Queues
    1. Client libraries for many development languages
        1. .Net, Java, Python, Node.JS, C++, PHP, Ruby
        2. They have an extremely simple model
            1. Easy to learn

Using Storage Queue

1. New storage account:
    1. Search: Storage accounts
    2. Create:
        1. Resource group: queue-rg
        2. Storage account name: storage-queue-mutthoju
        3. Location: West Europe
        4. Performance: Standard
        5. Account kind: StorageV2
        6. Replication: LRS
        7. Review + Create
        8. Create
    3. Go to the account

1. Click **Queues**
    1.
        - Queue
        1. Queue Name: store-messages
        2. OK
2. Click on the queue
    1.
        - Add message
        1. Message text:

            ```
            {
                "id": 4,
                "status": "success"
            }
            ```

        2. Uncheck: Message never expires
        3. Uncheck: Encode the message body in Base64
        4. OK
    2. We have a single message in the queue
2. Download the zip file in the resources and download the contents in the PC
    1. Extract.
    2. Open `code .` inside the folder
        1. `Program.cs`

            ```
            QueueClient queueClient = new QueueClient(storageConnectionString, queueName);

            if (queeuClient.Exists()) {
                while (true) {
                    // Get the text message
                    QueueMessage[] retreivedMessages = queueClient.ReceiveMessages(1);

                    if (retrievedMessages.length > 0) {
                        // Process (i.e. print) the message in less than 30 seconds
                        Console.WriteLine($"Dequeued message: '{retrievedMessages[0].MessageText)'");

                        // Delete the message
                        queueClient.DeleteMessage(retrievedMessage[0].MessageId, retrievedMessages[0].PopReceipt);
                    }
                }
            }
            ```

        1. Go back to portal:
            1. Search storage accounts
                1. storagequeuemutthoju
                    1. Access keys: Show keys
                        1. Copy connection string
        2. VS Code:
            1. Past the connection string in Program.cs

                ```
                private const string storageConnectionString = "<paste-here>";
                ```

            3. Hit F5
                1. Code displays the message pulled from the queue
            4. Refresh the page: store-messages
                1. The message is consumed
            5. Add another message:

                ```
                {
                    "id": 5,
                    "status": "failed"
                }
                ```

                1. Uncheck the check-boxes
            6. The code automatically pulled the message and displayed on the console
3. Cleanup: Delete the resource group

Event Grid

1. It allows building event-based architectures
    1. Publishes events to interested parties
2. There is no queue/ no order
    1. There is no guarantee that the order of events sent through event grid will reach its subscribers in the same order
3. Advantages of event grid:

    1. It has strong integration with many Azure services
    2. Cost effective
    3. It has one of the simplest pricing scheme in Azure
        1. No Tiers
        2. HA is built in
            1. No need for special customization and configuration to enable HA

4. Architecture:

```
Event Sources
    1. Azure Blob Storage
    2. Azure resource groups
    3. Azure subscriptions
    4. Azure Event Hubs
    5. Azure Media Services
    6. Azure IoT Hubs
    7. Azure Service Bus
    8. Azure Maps
    9. Azure Container Registry
    10. Azure SignalR Service
    11. Azure App Configuration
    12. Azure Machine Learning
    13. Azure Communication Services
    14. CloudEvents Sources
    15. Custom Events (anything)

    |
    v

Event Grid

    |
    v

Event-Handlers
    1. Serverless Code
        1. Functions
    2. Workflow and Integration
        1. Service Bus
        2. Logic Apps
    3. Buffering and Competing Consumers
        1. Event Hubs
        2. Storage Queues
    4. Other Services and Applications
        1. Hybrid Connections (WebSockets)
        2. WebHooks (Anything)
        3. Azure Authentication
```

5. Terminology:

    1. Event
        1. Defines what happened
            1. Examples: Storage blob added, IoT telemetry received
    2. Publisher
        1. Who instantiated the event
            1. Examples: Microsoft, my organization, other organization
    3. Event Source
        1. Where the vent happened
            1. Examples: Storage account (storage blog added event), IoT Hub (IoT Telemetry received event)
    4. Topic
        1. Where the event is sent
            1. Used to logically group related events
        2. Where the subscribers are listening to
    5. Subscription
        1. Where events interest me
            1. Examples: Storage blob added, IoT telemetry received (or both)
    6. Event Handler
        1. Where the event is sent
            1. Example: Azure function, Event hubs, etc...

6. Annotating the architecture:

```
Publisher: event sources are inside the publisher
    1. List of services that can publish events
Event: flows to Event Grid
Topics: In the Event Grid
Subscription: flows from event grid to Event Handlers
    1. Event handlers have subscriptions to the events
```

7. SLA:

    1. Built in SLA of: 99.99%
    2. Max event size: 1 MB (much larger than max msg size of storage queue (64 KB))

8. Performance: (very strong emphasis is put on this)

    1. 10,000,000 events / second
    2. 5,000 events / sec / topic

9. Latency: (very strong emphasis is put on this)

    1. Subsecond end-to-end latency in the 99th percentile

Event Grid Pricing

1. Based on:
    1. Number of operations
        1. First 100K operations are free per month (simple)
            1. No additional factors
                1. No Tiers
                2. No instance Size
                3. No Region
                4. ...
2. 1000000 operations per month: $0.54 per month
    1. Extremely cost effective
    2. It has integrations with Azure services
    3. It has amazing performance

A Note About using EventGrid in Code

1. Names of libraries used to access EventGrid have changed recently
    1. In the next video, after opening the code file, type:

        ```
        dotnet add package Microsoft.Azure.WebJobs.Extensions.EventGrid
        ```

        1. The step is required to remove an error in the code file ans add a package which is not included in the original project
    2. Right after that, type the following:

        ```
        dotnet add package Azure.Messaging.EventGrid
        ```

        1. It will remove other errors in the code file and enable it to run

Connecting Event Grid to the Orders Function

1. Search: catalog-vm
    1. Open the VM
        1. Start
2. Search: Event Grid System Topics
    1. Select Event Grid System Topics - Triggered on events that are built into the system (example: When a new blob is created in a storage account)
        1.
            - Create
            1. Topic Types: Storage Accounts (Blob & GPv2)
            2. Subscription
            3. Resource Group: readit-app-rg
            4. Resource: ordersreaditmutthoju (that we want to base the topic on)
                1. When a new blob or order is stored in the storage account, a new event will be triggered and it is handled by the event grid
                2. Event grid will trigger the orders function (instead of HTTP trigger)
                    1. Connecting the storage account to the function
            5. Topic: orders-topic
            6. Region: Automatically selected to be the same as that of the storage account
            7. Review + Create
            8. Create
3. Download zip file from resources:
    1. Copy the file and paste it in `readit/order` folder (replaces)
    2. Open VS Code and open `ProcessOrderCosmos.cs`

        ```
        [EventGridTrigger] EventGridEvent eventGridEvent // We run the function based on the event of the event-grid
        [Blob("neworders", FileAccess.Write, Connection = "StorageConnectionString")] CloudBlobContainer con
        [CosmosDB(databaseName: "readit-orders", collectionName: "orders", ConnectionStringSetting = "..."]
        ```

        1. Display details
        2. Serialize event-body
        3. Download the blob
        4. Deserialize and store the order object in CosmosDB
    3. Package:

        ```
        dotnet add package Microsoft.Azure.WebJobs.Extensions.EventGrid
        ```
    4. Open `local.settings.json`
        1. `StorageConnectionString: "<paste-the-connection-string-here>"`
        2. Go to portal:
            1. Search: Storage accounts
                1. ordersreaditmutthoju
                    1. Access Keys
                        1. Show
                        2. Copy
        3. Paste it in VS Code `local.settings.json` file
    5. Define the storage account that will be used by the function app:

        ```
        "AzureWebJobsStorage": "<connection-string-of-storage-account-that-was-created-automatically-to-store-data-and-metadata-of-the-function-ap
        ```

        1. Open Storage Accounts:
            1. `readitfuncstoragemutthoju`
                1. Show connection string
                2. Copy

2. VS Code:
    1. Paste it

6. Deploy the function app:

    1. Go to Azure Extension
        1. Workspace > Functions
            1. Click Deploy to Function App...
                1. Select the function App
                2. Upload Settings

4. Defining event-grid to trigger the function:
    1. orders-topic
        1. Go to resource
            1.
                - Event Subscription (The subscription is going to trigger the function as a result of the event in the storage account that we defined earlier)
                1. Name: order-subscription
                2. Event Schema: Event Grid Schema
                3. Topic Details:
                    1. Topic type: Storage account
                    2. Source Resource: ordersreaditmutthoju
                    3. Topic name: orders-topic
                4. Event types: (What are the events that are going to trigger an event into this topic)
                    1. Keep Blob Created checked
                    2. Uncheck Blob Deleted
                5. Endpoint type: (What is actually going to be triggered by the subscription)
                    1. Azure Function:
                        1. Click **Select an endpoint**
                            1. Azure automatically selects the function
                                1. The function has event-grid trigger which makes it eligible for to be triggered by this topic
                            2. Confirm Selection
                    2. Create
        2. Go to Event Subscription (on the left side panel)
5. Testing:
    1. Search: Storage Accounts
        1. Containers
            1. neworders (container) - New orders are uploaded here
    2. Search: Function App
        1. readitfunctionappmutthoju
            1. Functions
                1. ProcessOrderCosmos
                    1. Triger: EventGrid
                    2. Code + Test
                        1. Metadata
                        2. Switch to Filesystem Logs
    3. Search: Cosmos
        1. readit-cosmos-db-mutthoju
            1. Data Explorer
                1. readit-orders (database)
                    1. Orders (collection)
                        1. Items
                            1. We want to see a new item added
    4. Test:
        1. Upload sample order from order project
        2. Go to neworders container in storage account
            1. Click Upload
                1. Browse for files
                    1. Order project
                        1. ordersample.json
                        2. Upload
        3. Go to function:
            1. Output shows that the function was executed
            2. Event data: ordersample.json
            3. Order text
        4. Go to cosmos db
            1. Refresh
                1. A new item was added
    5. Flow:
        1. A new order was added to the storage account
        2. The event grid was triggered as a result of a new blob inserted into the container
        3. Event grid triggered the function sending it the details of the blog that was added
        4. The function reads the blob from the storage account and saves it into cosmos db

Connecting the Shopping Cart to the Storage Account

1. Copy `index.cshtml.cs` file attached to the lecture and replace it in `cart/Pages/index.cshtml.cs`
    1. Open the cart project in VS Code
        1. Open `index.cshtml.cs`

Protecting the Orders Function

Service Bus

1. It is a fully managed, full blown message queuing service built into Azure

2. It is extremely durable

    1. The chance of losing a message that we send in Service Bus is extremely slim

3. It supports point-to-point (Queue) and pub/sub (Topic) scenarios
    1. Combines storage queue and event-grid services
4. It is compatible with AMQP protocol
    1. Widely used with IOT devices
5. It is compatible with JMS 2.0 API (Premium only)
6. Service Bus Queues Architecture:

```
Message Sender:
    Web App
    Mobile App
    Service
    |
    v
Service Bus Namespace:
    Queue
    |
    v
Message Receiver:
    Service Or
    Application
```

    1. Messages are sent to a service bus namespace
        1. SB namespace has a queue which maintains the order of the messages
    2. Message receiver is a single receiver which listens to the queue
7. Service Bus Topics Architecture:

```
Message Sender
    Web App
    Mobile App
    Service
        |
        v
Service Bus Namespace
    Topic
        Subscription 1 --> Message Receiver
                               Business Logic
        Subscription 2 --> Message Receiver
                               Audit
```

    1. Message sender sends messages to a topic
    2. Topic has multiple subscriptions
    3. We can have multiple receivers and each receiver listens to a specific subscription
        1. This is similar to event-grid
8. Advanced features (not found in other services)
    1. Message sessions (guarantees FIFO)
    2. Dead-letter queue (holds messages that couldn't be sent)
    3. Scheduled delivery
    4. Transactions (for async service apps using service bus)
    5. Duplicate detection
    6. ...
9. Availability:
    1. SLA: 99.9%
    2. Can be configured for geo-disaster recovery
        1. If there is a problem with the region in which SB operates in, it can automatically switch to another region
10. Security:
    1. IP Firewall rules
    2. Service endpoints
    3. Private endpoints
11. Service Bus Tiers:
    1. Basic
        1. Queues & scheduled messages
            1. no topics
    2. Standard
        1. Fit almost any requirements
    3. Premium
        1. Needed for
            1. Larger message size
            2. Increased security (using resource isolation)

Service Bus Pricing

1. Based on two factors:
    1. Selected Tier
    2. Number of Operations
2. Example:
    1. Tier: Standard

2. Messaging operations: 10 million / month
3. $9.81 / month (quite low)

3. Example:
    1. Tier: Premium
    2. Daily message unit: 2
        1. Message unit: A dedicated resource for our service bus which can provide higher throughput and consistent performance
    3. Price: $1,354.15 / month (quite expensive)

Using Service Bus

1. Search: Service Bus
    1.
        - Create
        1. Resource group: New
            1. sb-rg
        2. Namespace name: sb-demo-mutthoju (unique)
        3. Location: West Europe
        4. Pricing Tier: Basic
        5. Review + Create
        6. Create
    2. Go to the resource
        1.
            - Queue (topic is grayed out because tier is basic)
            1. Name: my-items
            2. Create
        2. Entities:
            1. Queues
                1. my-items
    3. Shared access policies
        1. RootManagedSharedAccessKey
            1. Copy: Primary Connection String
    4. Download zip file and unzip
        1. Project will connect to service bus and send messages
        2. `code .`
            1. Program.cs
                1. paste the connection string against `connectionString`
                2. `queueName = "my-items"`
                3. Main method will display a messages and sends the message to service-bus
                4. We initiate client of service bus using the connection string
                5. We will construct a sender of service bus
                    1. An object that can send messages
                6. We will define a message and the content of the message will be "test message"
                7. We will send a message and print on console saying that a message was sent
            2. Hit F5
    5. Queues:
        1. my-items
            1. 1 message
                1. We cannot see the contents
            2. Service Bus Explorer
                1. Receive
                    1. Click Receive
                    2. Yes (recieve and delete the message)
                2. Doucble click on line under Sequence Number section
                    1. Test message
    6. Delete the resource group
        1. Overview
            1. Click the resource group
                1. Click Delete resource group
                    1. Enter the resource group name
                    2. Delete

Event Hubs

1. It is Big Data streaming platform and event ingestion service
    1. Note: No "messaging" in the description
    2. It is basically a managed Kafka implementation
        1. Kafka - big data streaming platform
            1. Many organization are using it as a messaging platform because it is extremely robust and flexible
    3. Pros:
        1. It can receive millions of events per second
            1. It is good for systems that can expect heavy load

2. Architecture:

```
Event Producers
    |
    HTTP
    AMQP
    |
    v
    Azure Event Hubs
        Partition 1 -+-> Consumer Group1 --> Event Receiver
                     |
                     +-> Consumer Group2 --> Event Receiver
        Partition 2 -+-> Consumer Group1 --> Event Receiver
                     |
                     +-> Consumer Group2 --> Event Receiver
        Partition 3 -+-> Consumer Group1 --> Event Receiver
```

```
                    |
                    +-> Consumer Group2 --> Event Receiver
Partition 4 -+-> Consumer Group1 --> Event Receiver
                    |
                    +-> Consumer Group2 --> Event Receiver
```

1. Event producers:
    1. Components generating the events
    2. Can be done by anyone with the client library / HTTP client
    3. Needs to use a simple connection and simple API
        1. It is extremely easy to instantiate one
2. Partition:
    1. It is a single event stream
        1. Similar to topic
    2. It can be thought of as a single queue
    3. It guarantees order
    4. Cons:
        1. It has a limited availability
            1. There is no way to make a single partition highly available
                1. Solution: It is better to spread messages across partitions to improve availability
                    1. Cons: Order is not guaranteed (not across partitions)
    5. Maximum of 32 partitions on a single Event Hubs
3. Event Receivers:
4. Consumer Group:
    1. A logical group of receivers that belong to the same application
        1. Example: Receivers for processing telemetry belong to one consumer group
        2. Example: Receivers for storing the telemetry in a database belong to another consumer group
5. Event receiving is done via AMQP protocol
    1. Extremely lightweight
    2. Adopted for messaging
6. SLA:
    1. Basic and standard tier: 99.95%
    2. Dedicated: 99.99%
7. Event Hubs Throughput Units
    1. Throughput is measured in Throughput Units (TU)
        1. 1 TU =
            1. Ingress (Input) - 1 MB / sec OR 1000 events / sec
            2. Egress (Output) - 2 MB / sec OR 4096 events / sec
    2. TUs are mentioned when pricing Event Hubs
    3. TUs are prepurchased, billed by the hour
        1. Taken into consideration for pricing of Event Hubs

Event Hubs Pricing

1. It is based on:
    1. Tier:
    2. Ingress: Incoming traffic
    3. TU: That we are going to use
2. Example:
    1. Tier: Basic, Ingress: 10 million events, TUs: 1
        1. $ 11.23 / month
    2. Tier: Dedicated (most expensive tier), Ingress & TUs are included, Capacity Units (servers to be dedicated): 4
        1. $ 19,999.00 / month
3. EventHubs can become very expensive if we are not giving a thought to it

Using Event Hubs

1. Search: EventHubs
    1.
        - Create
        1. Resource group: new
            1. hub-rg
    2. Namespace: eventhub-demo-mutthoju (unique)
    3. Location: West Europe
    4. Pricing tier: Basic
    5. TU: 1
    6. Review + Create
    7. Create
2. Go to the resource
3.
    - Event Hub
    1. Name: my-telemetry
    2. Partition count: 2
    3. Create
4. Shared access policy
    1. Click
        1. Copy the connection string
5. Download zip file and unzip
    1. Open the project in VS Code
        1. Install EventHub extension
            1. Search: EventHubs
                1. Azure Event Hub Explorer
                2. Install
        2. Open Explorer
            1. Open command-pallete
                1. View > Command Palette

        2. Ctrl + Shift + P
    2. Command paletter - a specialized type of CLI allowing us to run various commands provided by VS code or the extensions
        1. Extension added commands for EventHub
    3. Type: event
        1. Select event hub
            1. Select Subscription
            2. Select Resource Group
            3. Select EventHub
            4. Select EventHub entity: my-telemetry
    4. Open terminal
    5. Open Command Palette
        1. EventHub
            1. Send message to EventHub
            2. Test Message
        2. Terminal shows that message was sent successfully
    6. Program.cs
        1. Paste connection string against `connectionString`
    7. What is the code doing?
        1. Define consumer group Name (default name created)
        2. Consumer or receiver for the EventHub
        3. While true, we are going to read the events from the consumer
            1. For each event we receive, we are writing it to the console
    8. F5
        1. The code received the test message
    9. Send message again:
        1. Command palette
            1. ...
            2. Test Message 2
    10. The debug console shows the second message
        1. The code is extremely simple
            1. We don't have to deal with AMQP protocol
    11. Delete the resource group containing the EventHub

Selecting Messaging Solution

1. Selecting Messaging Solution:
    1. Service
        1. Storage Queue
            1. Used For...: Dead simple queueing (quick and dirty queuing solution)
            2. Guarantees Order: Yes
            3. Max Msg Size: 64 KB
            4. And also...:
                1. Extremely simple
                2. No additional cost
        2. Event Grid
            1. Used For...: Event driven architectures
            2. Guarantees Order: No
            3. Max Msg Size: 1 MB
            4. And also...: Great integration with other services
        3. Service Bus
            1. Used For...: Advanced queueing solutions (duplicate detection, dead-letter queue, ...)
            2. Guarantees Order: Yes
            3. Max Msg Size: 256KB
            4. And also...:
                1. Advanced messaging features
                2. Durable
        4. Event Hubs
            1. Used For...: Big data streaming
            2. Guarantees Order: Yes
            3. Max Msg Size: 1 MB
            4. And also...:
                1. Low latency
                2. Designed for heavy load
2. Use the table to decide which service to use

Current Architecture

1. Addition of event-grid topic
    1. Storage account & function app
        1. Makes the function more secure and asynchronous

Azure AD

Introduction

1. What is Azure AD:
    1. Short for Azure Active Directory
    2. It is a central identity and access management cloud service
    3. It is used to manage access to thousands of apps
        1. Among them - the Azure Portal
            1. To resources inside Azure (including apps)
    4. It is a secure, robust, and highly intelligent access management service
2. Advanced Features:
    1. MFA
    2. Conditional Access

3. Device management
4. Hybrid identity
5. Identity protection
6. Monitoring and reports
7. ...

3. We're interested mainly in:
    1. Control access to Azure resources
        1. By setting up
            1. users
            2. Groups
            3. Roles
        2. Not entirely architecture - and dev - related
            1. But still extremely important
                1. While designing for cloud, we need to take into account the authentication and authorization aspect
                    1. Azure AD is one of the best candidates for implementing that
    2. We can use Azure AD to add authentication to our apps (readit website)
        1. We can add it to our own app (code we write)
            1. Powerful feature of Azure AD
    3. Authentication to our own app can be done also via Azure AD B2C
4. Azure AD Features:
    1. Integrates with more than 2,800 apps
    2. It manages more than 1.2 billion identities (worldwide)
    3. It processes over 8 billion authentications every day
    4. It is secured using 3,500 security experts in Microsoft
        1. Which invests more than $1bn annually on cybersecurity
            1. Microsoft takes security quite seriously
    5. Azure AD is the largest identity and access management service in the world

Tenants

1. Tenant:

    1. It is **a specific instance of Azure AD containing accounts and groups**

        1. Also called as Directory (used interchangeably)
        2. It is not part of the subscription Hierarchy
            1. It exists beside the subscription
            2. For new subscriptions, a new tenant is created automatically

    2. A tenant can be assigned to multiple subscriptions

        1. If we want the same users to use multiple subscriptions

    3. Design:

```
Azure AD Tenant -----> Subscription
                       Resource Group 1
                       Resource Group 2
                       Resource Group 3
```

        1. It is an external resource that can connect to apps and one of then is Azure

A Look Around Azure AD

1. Portal: Search Azure Active Directory
    1. We don't have to instantiate Active Directory (in contrast to other resources)
        1. A tenant was automatically created and attached to the subscription when the subscription was created
    2. Default Directory:
        1. Default name of the directory or tenant that was created for us when subscription was created
        2. It appears under the account name
    3. Properties:
        1. Name: Azure Course Directory
        2. Save
    4. Refresh the page
    5. Users:
        1. Contains all users part of the directory
            1. It consists of me
                1. Name of user
                2. Principal Name or ID of user
                3. User type: Member
                4. Directory synced: No (such as on-prem directory)
                5. Identity issuer: Later
        2. Click the user:
            1. When the user was created
            2. Assigned roles:
                1. What I am allowed to do in this directory
                    1. Global Administrator - allows us to manage all aspects of Azure Directory and Microsoft services that use Azure Active Directory Identities (?)
                2. The role is related to the Azure AD and it is not related to Azure
                3. Click Sign-ins
                    1. List of recent sign-ins we have made
                    2. For each item, where we signed into
                    3. Filter:
                        1. Last 7 days
                            1. Authentication requirement - single factor authentication (weakest authentication possible)
                                1. We just provide username and password
        3. Password reset

1. Turn on the features that allow the user to reset their password without contacting the help centre
    1. Useful feature
        1. Basic Tier doesn't support
            1. Need to upgrade to a higher tier
4. Changing Tenants
    1. Single account can be attached to multiple tenants
        1. It is sometimes useful to switch between tenants if we want to work on a tenant that is different than the one we are working on right now
    2. Click on: Directories and Subscriptions Link
        1. Identical to settings button (both will bring to the same page)
            1. Portal settings:
                1. List of tenants our account is attached to
                    1. Switch button: switch to another tenant

Users and Groups

1. Two of the main three objects managed by Azure AD
    1. The 3rd one is roles (later...)
2. Using users and groups, Azure AD Manages and stores the users that are part of the tenant
3. Groups the users in Groups (users who belong to the logical groups)
    1. IT Admin
    2. Developers
4. Using groups, we can define roles to groups instead of each user
    1. Makes roles definitions easier

Working with Users and Groups

1. Azure Active Directory:
    1. Users:
        1.
            - New user
        1. Name: david
            1. Domain name is created automatically when the tenant was created
                1. Remember the domain name
                    1. account + .onmicrosoft.com
        2. Fullname: David Jones
        3. Password: Auto-generated password
        4. Show password
            1. Copy the password
        5. Groups: none
        6. Roles:
            1. Azure AD roles and not Azure roles
        7. Create
    2. Identity issuer: tenant
2. Open portal in-congnito
    1. Sign-in using the new user
        1. david@azcoursememilavi.onmicrosoft.com
        2. Enter password
        3. Update Password
        4. Sign in
        5. Skip for now...
        6. Yes
        7. May be later
    2. New storage account:
        1. Don't have a subscription (attached to our account)
            1. Start with an Azure free trial
            2. Manage Azure Active Directory (we are part of)
            3. Access student benefits
        2. Why?
            1. A user is created in an Azure AD but we didn't connect the user with Azure subscription
    3. Close
3. Click: Sign ins
    1. David Jones successfully signed in
4. Guest user: (a user that is not going to belong to the active directory becuase he does not belong to the same organization or does not want to be part of the same Active Directory)
    1.
        - New Guest User:
    1. Invite user:
        1. Name: memi-lavi-2
        2. Email address: memi@memilavi.com
        3. Personal message: You are invited to my Azure AD
        4. Invite
    2. An email is sent
    2. Copy the link
        1. Open in-cognito mode
        2. Paste in browser
        3. Enter credentials
        4. Review the permission
            1. Accept
        5. All Apps that can be accessed using the directory
        6. Go to portal
            1. There is no subscription
    3. Close
5. New group:
    1. Go to Groups
        1.

- New Group (Security group managed by Azure AD)
1. Group name: Azure Architects
2. Members: Click No members selected
    1. Search: David Jones
    2. Select David Jones
    3. Select
3. Create

2. Click the group
    1. Members

Azure AD Licenses

1. License model of Azure AD:
    1. Azure AD licenses have great effect on the functionality and price of Azure AD
        1. Important to understand the differences and right solution for the right system and the right customer
    2. Licenses:
        1. Free: (default license model)
            1. Max Objects: 500,000 (users & groups)
            2. Users & Groups: x
            3. MFA: x (All or nothing)
            4. Dynamic Groups: -
            5. Conditional Access: -
            6. Risk Detention: -
            7. Risk based Conditional Access: -
            8. Privileged Identity Management (PIM): -
            9. Price: Free
        2. Premium 1:
            1. Max Objects: Unlimited
            2. Users & Groups: x
            3. MFA: x (With Conditional Access)
            4. Dynamic Groups: x
            5. Conditional Access: x
            6. Risk Detention: -
            7. Risk based Conditional Access: -
            8. Privileged Identity Management (PIM): -
            9. Price: $ 6 / user / month
        3. Premium 2:
            1. Max Objects: Unlimited
            2. Users & Groups: x
            3. MFA: x (With Conditional Access)
            4. Dynamic Groups: x
            5. Conditional Access: x
            6. Risk Detention: x
            7. Risk based Conditional Access: x
            8. Privileged Identity Management (PIM): x
            9. Price: $ 9 / user / month
    3. When designing the authentication and authorization of the system, make sure to select the right license for our requirements

MFA

1. Authentication Type:
    1. Authentication is divided into three factors:
        1. Something you know (username / password, security question)
        2. Something you have (phone, smart card)
        3. Something you are (fingerprint, iris scan, other biometric data)
    2. Selecting Authentication type:
        1. Feasibility: Can we actually implement this factor?
            1. Not all computers / phones have fingerprint scanner
        2. Ease of use: username / password is much quicker than receiving a text and typing it down
            1. Besides fingerprints & facescan
            2. If the intended audience is general population including elderly people or others that have hard time figuring out complex authentication mechanism, we go for an easier method
        3. Sensitivity: Sensitivity of the data the system holds
            1. Biometric data is much more difficult to hack than username / password
            2. The more sensitive the data, the safer authantication mechanism we want to use
                1. Think about how sensitive is the data before choosing the authentication mechanism to protect
2. Two Factor Authentication:
    1. For extremely sensitive systems (Banks, medical, etc.)
        1. We pick two of the three factors (something you know, something you have, something you are) and require them for authentication
            1. Common factors used:
                1. Something you know (username / password)
                2. Something you have (code in Text)
                    1. If Something you know was correct
            2. Other examples:
                1. Combination 2:
                    1. Something you know (username / password)
                    2. Something you are (fingerprint)
                2. Combination 3: (rare)
                    1. Something you have (phone)
                    2. Something you are (fingerprint)
    2. **When selecting the authentication engine, make sure the authentication engine suports two factor authentication** (as well as MFA)
        1. Reason: If the authentication requirements change, we cannot follow them with the current authentication engine
    3. The capabilities should come out of the box
        1. We should not require development of those features (advanced authentication types or MFA)
    4. The decision about whether to use two-factor authentication or which authentication type to use:
        1. The decision will directly affect the end users

1. Solution: System analyst must be in the loop

Security Defaults

1. Nice feature: Security defaults
    1. Using security defaults we can:
        1. Increase protection of the organization in the Free tier
            1. Full MFA does not exist with free tier
                1. Solution: We can increase the protection of sign-ins even in the free tier
                    1. Adds preconfigured security settings:
                        1. Requires all users to use MFA (blocks 99.9% of account compromises)
                            1. No additional cost
                            2. Blanket MFA (all users will have MFA and we won't be able to define which user under which condition will have the MFA enabled)
                        2. Blocks legacy authentication
                        3. ...
                    2. For more fine-grained management - use conditional access (P1 and up)

Using Security Defaults

1. Portal: Azure AD:
    1. Properties
        1. Manage security defaults
            1. Enable Security defaults: Yes
            2. Save
2. Open in-cognito
    1. Log into
        1. Username
        2. password
        3. Verify your identity
            1. email with code (another factor)
            2. Enter code
        4. Verify
        5. No thanks
    2. MFA is addition using security details
        1. We can disable it if we don't want this feature

RBAC

1. Role Based Access Control (RBAC)
    1. In the past, authorization we defined per user or user group
        1. Example:
            1. David is allowed to add items to the inventory
            2. John is not allowed to read data of other doctors
    2. This was very granular, extremely hard to maintain
        1. Complications:
            1. If David leaves the company:
                1. SysAdmin has to go over all the authorization definitions of David (dozens may be) and cancel each one of them
            2. Suppose we have a new category in the store & we want to allow all emloyees to add items to its inventory
                1. With the legacy system of authorization, we need to go through all users in the system and grant each and everyone of them the right to work with the new category
        2. RBAC solves the above problems
2. With RBAC:
    1. Three elements
        1. Users
            1. Jane
            2. David
        2. Roles
            1. Hotel Owner
            2. Front Desk
        3. Authorizations
            1. Set Price
            2. Check-In
            3. Check-Out
    2. Associate roles with Authorizations
        1. Hotel Owner - Set Price, Check-In, Check-Out
        2. Front Desk - Check-In, Check-Out
            1. We didn't talk about the actual users
                1. We have only mapped roles to privileges
    3. Associate actual users to roles:
        1. Jane - Hotel Owner
        2. David - Front Desk
    4. Advantages:
        1. Privileges of the roles are more or less constant (they do not change frequently)
        2. Association of users to roles is often one to one (simple)
    5. New user:
        1. Rachel - Associate with a role
            1. Front Desk
                1. She will automatically be granted all privileges of the front-desk

Azure Roles

1. RBAC in Azure
    1. In order to perform any operation, or access any data in Azure you have to have the appropriate role
        1. If you want to:
            1. Define resource groups

2. Access data in SQL
3. See metrics of App Service

2. Then we have to have the right role
    1. Otherwise, we will get an empty portal

2. When a subscription is created, the user is granted the highest role available in Azure - Owner
    1. For other users, we have to define the roles
3. There are three types of roles:
    1. Owner
        1. Can perform any action on the resource, including assigning roles to it
            1. We can add users and groups to resources to do various operations
    2. Contributor
        1. Can perform any action on the resource, but cannot assign roles to it
    3. Reader
        1. Can only view data, but cannot change anything
4. Examples:
    1. Virtual Machine Contributor
        1. Can manage virtual machines
    2. Cosmos DB Account Reader
        1. Can read Azure Cosmos DB account data
            1. No creation or modification of existing Cosmos DB accounts
    3. Service Bus Data Owner
        1. Allows full access to service bus resources including assigning new roles, users, or groups to the resources
5. Implementation:
    1. Users: Are defined in the Azure AD Tenant
    2. Role & Authorizations: Defined in Azure
    3. To assign roles in Azure we assign users and groups from Azure AD tenant into Roles in Azure
        1. It's always better to assign roles to groups and not to individual users
            1. It is easier to maintain
                1. **The granular the assignment, the harder it is to maintain**

Using Azure Roles

1. Portal: New resource group
    1.
        - Create
        1. Name: test-rg
        2. Review + Create
        3. Create
    2. Go to readit-app-rg
        1. Access control (IAM) (every resource has this)
            1. Allows us to assign users and groups and roles to resources
            2. Role assignments
                1. Only Owner
                    1. Role: Owner (whatever we want)
                    2. Scope: Subscription (inherited)
                        1. Owner of subscription so he can do whatever he wants inside the resource group
                2.
                    - Add
                    1. Add role assignment
                        1. Contributor (does to allow us to assign roles in Azure RBAC)
                    2. Next
                    3. Which users or groups to assigned to the role:
                        1. User group or service principal
                        2. Select members
                        3. Azure architects (group)
                        4. Select
                    4. Review + assign
                    5. Review + assign
                3. Azure architect group is added in the contributor section
                    1. Contributor of only this resource (Resource group)
                        1. Contributor in resource group but not other resource groups
                4. User is under subscription (and inherits ownership)
                5. Testing:
                    1. Sign in using David in-cognito mode
                    2. Next
                    3. Password
                    4. Skip for now ...
                    5. Yes
                6. Search: Resource Group
                    1. David can see only a single resource group
                    2. Go to Access control (IAM)
                        1. Role assignments
                            1. Add role assignments (disabled) (as a contributor I cannot add or modify role assignments of resources in Azure)
                                1. We can do other things as a contributor
                                    1. Overview
                                        1.
                                            - Add
                                            1. storage account
                                                1. Create
                                                2. Name: davidaccounttest
                                                3. Review + Create
                                                    1. Valdation passes
                                        2. Remove the resource group
                                            1. We cannot do it from here, so we have to do it as our user
    3. Granting permissions to resource groups is extremely useful
        1. We may want external users to perform various operations on a specific resource group

1. We don't want them to do anything else to other resource groups (which might hold resources of other customers)

Managed Identities

1. It is the ability to assign Azure AD identity to Azure resource
    1. The resource can connect to other Azure resources using this identity
    2. We don't need to handle any credentials (usernames, passwords, etc.)
        1. The identity of the connecting resource is enough to identify the resource and allow it to access other resources
2. Two types of Managed Identities:
    1. System assigned: Managed by Azure, tied to the resource's lifecycle (when the resource is deleted - so is the identity)
        1. Most of the time we use this
            1. They are much easier to handle and manage
    2. User assigned: Managed by the user. Can be assigned to multiple resources, not tied to any lifecycle
3. Managed Identities:
    1. Resources that can be assigned Managed Identity: (that can have an identity when connecting to other resources)
        1. App Service
        2. Virtual Machine
        3. Event Grid
        4. Function
        5. ...
4. Resources that can be authorized using Managed Identity:
    1. SQL
    2. Event Hubs
    3. Service Bus
    4. Storage
    5. Key Vault
    6. ...
5. Example: Managed Identity assigned to inventory app service and use it to connect to the Azure SQL of readit website

Using Managed Identity with the Inventory App Service

1. Portal: Go to inventory app service
    1. Go to Identity
        1. System assigned:
            1. Status: on
                1. Going to have a managed identity that we can use
            2. Save
            3. Yes
        2. A managed identity gets created and it is registered with Azure AD
    2. Duplicate tab:
        1. SQL Server (machine that holds SQL database)
            1. readitdbserver
                1. Active Directory admin: Not configured (configuration allows resources with managed identities to connect with this database)
                    1. Click:
                        1. Set admin
                            1. David Jones (set the user as admin)
                            2. Select
                            3. Save
                        2. When we perform actions against the SQL server using active directory it will use the user as admin for the operation
                        3. Firewalls and virtual networks
                            1. Ensure that the client IP is allowed to access the server
                                1.
                                    - Add client IP
                                2. Save
    3. Go to the inventory app service page and copy the name of the app service
    4. Open cloud shell:

        1. Search: SQL Database

            1. readit-db (readitdbserver/readit-db)

        2. Command:

            ```
            sqlcmd -S <server-name-of-the-db-server> -d readit-db -U david@azcoursememilavi.onmicrosoft.com -P <password-of-the-user> -G -l 30
            ```

            1. `-d` - database name
            2. `-G` - Indicates we are going to use active directory authentication
            3. `-l` - length of the session

        3. Run SQL commands - Run commands that will allow managed identity of the app service to connect and perform various actions to the database

            ```
            CREATE USER [readit-inventory] FROM EXTERNAL PROVIDER;
            ALTER ROLE db_datareader ADD MEMBER [readit-inventory]; -- grant app service read permission
            ALTER ROLE db_datawriter ADD MEMBER [readit-inventory]; -- grant app service write permission
            ALTER ROLE db_ddladmin ADD MEMBER [readit-inventory]; -- change structure of db - modify table structure or creation of new tables;
            GO
            EXIT
            ```

            1. `GO` - Executes the commands specified previously

        4. Go to app service:

            1. configuration
                1. We need to modify the connection string because the current connection string holds the username and password for connecting to the database
                    1. We don't need it anymore - the db needs to be connected using managed identity of the app-service
                    2. Edit

1. Remove user id and password from the connection string
2. Due to some changes to Azure SQL behavior, add the following at the end of the conneciton string after removing the username and password

```
Authentication=Active Directory Managed Identity
```

3. OK
4. Save
5. Continue

2. Testing:
    1. If using application gateway, we cannot access the app service
        1. We need to go to the IP address of application gateway
    2. Duplicate the page:
        1. application gateway
            1. copy public IP
                1. Paste in browser
                    1. App service has retrieved data from the database even though we have not used username and password
                2. Change 0 to 3 and Save
                3. Refresh the page

Using Azure AD to Authenticate our App

1. Azure AD can be used as authentication engine on other apps
    1. Not just the Azure Portal
    2. It can be used on our own apps
2. Inventory app should be accessible to only a store employees
3. Process:
    1. Register the app in Azure AD
    2. Add code to use Azure AD as authentication engine
        1. For App Services - can be configured via the Portal (almost no code changes required)
4. The authentication is implemented using:
    1. OAuth and JWT

OAuth & JWT

1. What is OAuth2:
    1. Standard protocol for authentication & Authorization
    2. Widely used, mainly in web apps
    3. It has many moving parts and it is very complicated to implement
    4. We'll discuss only high-level details
    5. OAuth2 Components:
        1. User - The user who wants to access protected resources in the API
            1. Human being
            2. Software
        2. Client App - The client application accessing the API
            1. User uses client app
        3. Authorization Server - A specialized server that is used to authorize the user for the client application
            1. It identifies the user using whatever method of identification methods it supports and tells the client app, who the user is, and what is he allowed to do.
        4. Resource Server - The API being accessed
    6. Flow of OAuth2
        1. Client App accesses API anonymously (without passing any id data)
        2. Resource Server (API) identifies that the call is anonymous (no user data is passed) and tells the client app to redirect to authorization server
        3. Authorization server asks user to grant permission after identifying the user
        4. Authorization server returns access token to the client app
            1. Access token contains the data needed by the resource server (API) to identify the user and figure out what the user is allowed to do
        5. Client App sends access token to Resource Server (API)
        6. The Resource Server (API) now knows who the user is and whether he is allowed to access the API
    7. Go to www.feedly.com (one of the best RSS clients in the web - recommended)
        1. RSS clients are clients subscribed to blogs
            1. A great resource of learning in any field of knowledge
                1. APIs
                2. Software architecture
                3. Technological advancement
                4. Public speaking
                5. ...
        2. Login
            1. How do we want to login
                1. Which authorization server we want to Use
                    1. Facebook (functioning as authorization server)
                2. Enter credentials and login
            2. Logged-in to feedly
                1. Facebook told feedly I know who the user is
                2. Feedly didn't do anything related to identification
    8. App Registration
        1. Authorization Server should be familiar with the Resource Server (API) (Feedly)
            1. When feedly redirected the user to Facebook, then feedly told Facebook, I am feedly, and I want you to identify the user for me and return the info to feedly
        2. Resource Server must register itself with the Authorization Server
            1. It includes filling some info about the resource server (name, url address to which the authorization server needs to return the information to about the user)
        3. github:

1. App registration process:
    1. Application name: Order Management
    2. Homepage URL: http://www.ordermng.com
    3. Application-Description:
    4. Authorization callback URL: http://www.ordermng.com/callback (most important part)
        1. It is used by the authorization server to tell back the application who the user is and what he can do
    5. Register application
2. Important parts: (the keys that must be passed to the authorization server when asking for authorization so that the authorization server will know which application is asking the user authorization)
    1. client-id
    2. client-secret

9. Access token - returned by authorization server that the client app is sending to the resource server in order for user to be identified
    1. It is JWT
        1. JSON Web Token
            1. A small string containing the data the server needs in order to authenticate the user
            2. Returned by the OAuth2 authorization server
            3. It contains everything the API needs to exactly who the user is and what is he allowed to do
        2. JWT:
            1. Has three sections:
                1. Header - type of token (JWT) and signing algorithm (used to sign the JWT)

```
{
    "alg": "HS256",
    "typ": "JWT"
}
```

                2. Payload - actual data about the user (server needs to make a decision as to whether the user can access the resource or not)
                    1. There is no standard format

```
{
    "sub": "1234567890",
    "name": "John Doe",
    "admin": true
}
```

                        1. https://jwt.io/introduction/
                        2. It contains info about the user that the API needs
                3. Signature - JWT is signed electronically so that it can't be tampered with during the transfer from authorization server to the API
            2. The three parts are:
                1. Base64 encoded
                2. Concatenated with .
            3. JWT Debugger: jwt.io

```
Encoded <-> Decoded
```

                1. Modify name and token changes

Configuring Azure AD and the Inventory App Service for Authentication

1. Portal:
    1. Remove access restriction to app service so that traffic will be allowed (public app service)
        1. Reason: Application Gateway adds a layer of complexity when implementing authentication
    2. Search: App Services
        1. inventory-app-service
            1. Networking
                1. Access restriction
                    1. Rule allows access only from application gateway (all other traffic is denied)
                    2. Remove the rule (all traffic is allowed)
            2. Overview
                1. Click URL
    3. Duplicate the page
        1. Search: Azure AD
            1. App registrations (set up an application in Active Directory representing a real world application that will be authenticated using Azure AD)
                1.
                    - New registration
                1. Name: readit-inventory-web-app
                2. Supported account types: Accounts in the organizational directory only (Azure Course Directory only - Single tenant)
                    1. Only accounts in the organizational directory can access the app service
                3. Redirect URI (optional): URI the Azure AD will call after completing the authentication
                    1. Web platform
                    2. Copy the URL of the app service and paste in URL field
                    3. Update as follows:

`<url>/.auth/login/aad/callback`

4. Register

4. Housekeeping:
    1. Readit Inventory Web App
        1. Authentication
            1. Check:
                1. Access tokens
                2. ID tokens
            2. Required for web apps to complete the full flow of authentication
            3. Save
        2. Certificates & secrets (add new client secret which will be used to identify application registration in the code)
            1.
                - New client secret
                1. Description: readit inventory secret
                2. Expire: 12 months
                3. Add
            2. Copy and save the Value in a file
                1. inv.secret
5. Go to inventory app service (readit-inventory-memi1)
    1. Authentication
        1. Add identity provider (which identity provider we are going to use in order to authenticate users for this app service)
            1. Microsoft (other options are Facebook, Github, ...) - Azure AD to authenticate to this App Service
                1. App registration type: Pick an existing app registration in the directory
                2. Name of app ID: readit-inventory-web-app (existing)
                3. App service authentication settings:
                    1. Redirect access: Allow authentication access
                    2. Unauthenticated requests: HTTP 302 found redirect recommended for websites
                4. Add
                    1. Authentication is enabled on app service

Adapting the Inventory Code and Using Azure AD

1. Minor changes required for .Net core code (.Net framework or Node.js doesn't need these changes)
2. Download zip file attached
3. Unzip
4. Copy paste the files in inventory directory
5. Open the code in VS Code
6. EasyAuthExtention.cs
7. EasyAuthMiddleware.cs
    1. Executed before running the actual HTTP request to the inventory file
    2. Extracts the details of the user from the authentication token (JWT)
8. Startup.cs
    1. Modified is done to use the authentication
9. EasyAuth:
    1. Name of the authentication offered by Azure using the portal when we configured the authentication and authorization [for the app service]
    2. Use EasyAuth to authenticate the app service
10. Claimscshtml, Claimscshtml.cs
    1. To see the claims contained in the JWT token that we are going to receive from Azure AD
11. Run:
    1. `dotnet publish -o publish`
    2. Right click on `publish`
    3. Deploy to web-app
    4. readit-inventory-app-service
    5. Deploy
12. Browse Website
    1. Permission screen (AD asks us for permission to perform various actions on the inventory app)
        1. AD already knows who we are (we are logged in in the portal)
    2. Accept
13. Open in-cognito
    1. Copy the URL of inventory app service
    2. Enter username
    3. Enter password
    4. Verify your identity (optional)
    5. Email a code
    6. Enter the code
    7. Verify
    8. No thanks
    9. Logs in
14. Logging in using David Jones user
    1. Copy the URL of inventory app service
    2. Enter david@azcoursememilavi.onmicrosoft.com
    3. Enter password
    4. Skip
    5. Accept
        1. Asks to allow Readit inventory Web App to access some of our details (basic profile: name, picture, username, ...)
    6. Logs in
15. OAuth2 can be configured with minimal or no code changes and it is easy to configure
16. Another thing:
    1. Copy and paste URL of inventory app service
    2. `<url>/.auth/me`
        1. Displays JWT containing details of authenticated user passed to our app
        2. We can extract details
        3. Copy the JSON
        4. Open `jwt.io`
        5. Paste

3. Parameters:
    1. Unique
    2. Signature was verified
4. Claims received in JWT from Azure AD:
    1. `<url>/claims`
        1. Details returned about the user used by the app
            1. `name` claim is used for display
            2. `preferred_username`

Azure AD B2C

1. It is an Identity-as-a-service for your application
2. It is a Business-to-Customer (B2C) service
3. It enables integrating identity services in your app
    1. Services: Makes it different from Azure AD
        1. Allows working with various identity providers and Azure AD is one of them
4. Provides various user flows
5. Enables customization
6. What are the identity services provided by Azrue AD B2C:
    1. Sign Up
    2. Sign In
    3. Log Out
    4. Reset Password
    5. ...
7. We can use only configuration and no code
8. What is the difference between Azure AD B2C and Azure AD
    1. Azure AD:
        1. Identity Provider (Holds user's details)
        2. Single tenant can be used for authentication for many apps (Azure portal, Office 365, App #3 (readit website), ...)
    2. Azure AD B2C:
        1. Identity Service (It is used to perform identity-related actions)
            1. It doesn't hold user's details
        2. It works with various identity providers
        3. It can be used by business apps as the identity component
        4. Identity providers used by Azure AD B2C
            1. Microsoft
            2. Facebook
            3. LinkedIn
            4. Salesforce
            5. Google
            6. Twitter
            7. Github
            8. Slack
            9. WeChat
            10. Amazon
            11. Azure AD (used for the inventory app service)
            12. AD-FS
            13. Apple ID
            14. Fido
            15. Instagram
            16. ...
9. Authentication Features of Azure AD B2C:
    1. MFA
    2. Conditional Access
    3. Audit log
    4. Custom Policies
    5. Custom Pages
    6. ...
10. It is quite complex to set up
    1. There are a lot of moving parts
11. Good for configuring other identity providers

Current Architecture

1. Difference:
    1. Addition of Azure AD Authentication connected to inventory app service
        1. Protects inventory app from un-authorized access from users who are not store employees

Synchornizing Azure AD with On-Prem AD

1. Many organizations want to sync their on-prem Active Directory with Azure AD
    1. Useful when the organization has apps on prem and in cloud and wants to have a single user base
        1. User doesn't have to login twice (once in the cloud and once on prem)
            1. Solution: Azure AD Connect
                1. It is an agent connecting on-prem AD with Azure AD
                    1. The user's data is synced between those two engines
2. Two modes of authentication with AD connect:
    1. Password Hash Sync
        1. The passwords are copied to Azure AD and authentication happens in the cloud
        2. Pros:
            1. Good if on-prem AD is not available
    2. Pass-through
        1. Passwords stay on-prem and Azure AD passes data to on-prem AD for validation
            1. Azure AD is just a pipe that transfers data to the on-prem engine
        2. Pros:
            1. Data is more protected because passwords are not stored on the cloud

Monitoring in Azure

Introduction

1. What is monitoring and why do we need it?
    1. A working app is not enough
        1. We must know its status, how it performs, and hwne it has problems
            1. Done via monitoring
    2. Example: Current architecture
        1. Some of the moving parts can go wrong
            1. We need to figure out:
                1. What went wrong
                2. When
                3. Why
2. Azure offers a lot of built in monitoring mechanisms
    1. It's a good idea to be familiar with as many as possible
    2. A centralized monitoring hub where all monitoring data is streamed, and can be queried, or used for triggers
    3. Monitoring is extremely cost effective in Azure
3. Monitoring is based on two types of data:
    1. Metrics - Numeric values describing resource's various aspects at a specific point in time
        1. Example: CPU utilization, Disk performance, response time, ...
    2. Logs - Events that occurred in the system
        1. Textual & numeric
        2. Examples: System started, Error occurred

Resource Monitoring

1. Almost every resource in Azure has a "Monitoring" section
    1. Items under Monitoring section:
        1. Insights
            1. Offers various insights about the resource
            2. Differ between resources and each resource displays other insights
        2. Alerts
            1. Triggered when something happens
                1. Example: CPU utilization goes above a specific value
        3. Metrics
            1. Metrics about the resource (CPU, disk, RAM, etc.)
        4. Diagnostic settings
        5. Logs
            1. Log records about various events
        6. Connection monitor (classic)
        7. Workbooks

Using Metrics

1. Open catalog-vm
2. Start the VM
3. Overview
    1. Monitoring tab
        1. It has various metrics about the VM
            1. CPU utilization
            2. Network traffic
            3. Disk bytes
            4. Disk operations
        2. Data is displayed only for 1 hour by default
            1. We can change time filter
                1. 30 days say
        3. Disk bytes:
            1. Blue - disk read bytes (how many bytes)
            2. Orange - disk write bytes (how many bytes)
        4. Click on the graph
            1. Disk read bytes
            2. Disk write bytes
                1. Click:
                    1. Filtering:
                        1. Which metric we want to see
                        2. Aggregation type
                            1. Sum
                            2. Avg
                            3. Min
                            4. Max
                            5. ...
                    2. Click on Share
                        1. Download to excel
    2. Metrics
        1. We can build our own metrics and define the filter for that
            1. Metric: Percentage CPU
                1. Data will start coming in few minutes
                2. Time range: Last 30 days
            2. Add Metric
                1. Disk Write Operations
                    1. Second graph
        2.
            - New chart
            1. Metric: Network in Total

1. Second chart gets added
2. Click on three dots
    1. Chart settings
        1. Bar chart
        2. Tables
        3. ...
        4. Custom:
            1. Chart Title
        5. See legend or not
        6. Position of the legend (bottom or the right)
        7. Full legend or compact legend
    3. Metric: Memory (no such metrics)
4. Other resource metrics:
    1. Application Gateway
        1. readit-app-gw
            1. Monitoring
                1. Metrics
                    1. Metric: Backend First byte response time (how fast was the response from the backend)
                    2. Time range: Last 30 days
                    3. Click filter:
                        1. Metric: Blue dots scattered (this kind of metric supports filtering and grouping)
                    4. Add Filter (used for filtering further for specific backend pool say)
                        1. Select backend pool
                        2. Value: Inventory pool or Catalog pool
                    5. Click on Apply splitting
                        1. Data will not be filtered but split
                            1. Two separate graphs on the same chart
                    6. Legends:
                        1. Click one
                            1. We can select color or drag to change the color

Azure Dashboard

1. There is no save button in Metrics
    1. If we refresh, we will lose the configuration
        1. Metrics are not saved per resource but saved in dashboards
    2. Click: Pin to dashboard
        1. Create new
            1. Private (only we are going to see)
            2. Dashboard name: readit-metrics
            3. Create and pin
    3. Click on button at the top left corner
        1. Dashboard
            1. Click on My Dashboard
                1. Readit Metrics
                    1. Shows the metrics we defined previously
                    2. Click Edit
                        1. Drag and move it to place it wherever we want
                        2. We can resize
                    3. Clock - Add
                        1. We can change Parameters
                            1. 24 hours
                        2. Can drag and place
                    4. Resource groups - Add
                        1. List of resource groups in the subscription (we can click to open them)
                    5. (Any other widgets)
                2. We can remove widget by clicking on trashcan icon
                3. We can change the name of the dashboard: Course Dashboard
                4. Save
            2. Click on Download
                1. JSON file is downloaded
                    1. It describes the dashboard
                        1. Position of widgets
                        2. Which widgets are included in it
                        3. ...
                    2. We can use the JSON to construct new dashboards in other subscriptions
            3. We can also have multiple dashboards
                1.
                    - New dashboard
                1. Blank dashboard
                    1. Name: Shortcuts
                    2. Clock: 24 hours
                        1. Done
                    3. Resource groups
                    4. All resources
                    5. Users and groups (a clear view of users and groups in the dashboard)
                    6. Save
                2. The elements are interactive
                    1. We can click on them and go to the element
                        1. Click on DJ user
                            1. We can navigate straight to the elements we are interested in

Alerts

1. With alerts, we can get notifications about events in your resources
    1. Examples:

1. We can get alert when VM's CPU goes above 90%
2. More than 20% of requests fail in app service or other app
3. Server error occurs in the last hour (or any other time-frame)
2. Useful if we want to get real-time notifications of problems in the system
2. Three components to define for alerts:
1. Condition: Defines when to trigger the alert (i.e. CPU goes above 90% - condition is met and the alert is Triggered)
2. Actions: What to do? Usually - send notifications. (When an alert is triggered, what Azure should do?)
1. Notifications are sent to **Action Groups**
3. Details: Contents of notifications (what information is included in the alerts)
3. Alerts Pricing:
1. Per metrics measured (0.10$ / metric)
1. Metric: CPU goes above 90% say (This is one alert definition and we pay 0.10$ for this per month)
2. Per notification type
1. After a specific number of emails, sms, or push notifications, we are going to start to pay
1. 1K email, 1K push notifications, 100K web hooks, 100 SMS in the US are included for free each month
1. These numbers are quite enough
3. All in all - quite cost effective

Using Alerts

1. Go to catalog-vm
1. Metrics
1. Metric: Percentage CPU
1. Time-frame: 24 hours
2. Click: New alert rule (define an alert to be triggered when there is a problem with the CPU utilization)
1. Parts:
1. Condition: What should happen in order for the alert to be triggered (automatically created)
1. Click the condition
1. Only the last week
2. Operator: Greater than
3. Aggregation type: Average
4. Threshold value: 90
2. Evaluation based on:
1. Aggregation granularity (Period)
1. 5 minutes
2. Frequency of evaluation
1. Every 5 minutes
1. If CPU is > 90% for 5 minutes, the alert will be triggered, if not, the evaluation will happen after 5 minutes
3. Done
2. Actions
1. Action group: A group of recipients that will get notifications when the alert is triggered
2.
- Create action group
1. Resource group: readit-app-rg
2. Name: course-admin-ag
3. Display name: course-admin-ag
3. Notifications
1. Notification type:
1. Email Azure Resource Manager Role (Azure will email the key roles in the subscription)
2. Email/SMS message/Push/Voice (select this)
1. Check Email:
1. Email:
2. Enable the common alert schema: Yes
3. OK
2. Name: Email to admin
4. Actions: To trigger some APIs and software as a response for the action group
1. Action type: (Azure function, Event hub, ITSM, Logic App, Secure Webhook, Webhook)
5. Review + Create
6. Test action group (preview)
1. Select sample type: Metric alert - Static threshold
2. Test
7. Create
1. We get an email which indicates that alert definition succeeded
3. Details
1. Properties:
1. Severity: Warning (Info is default)
1. Nothing to immediately worry about but needs to be definitely looked at
2. Alert rule name: High CPU Utilization
3. Description: CPU of the VM is high, please take a look
4. Ensure enable upon creation is checked
4. Review + Create
5. Create
2. Alerts
1. Alert rules
1. Our alert rule appears
1. Target resource: catalog-vm
3. Remove the alert because alert costs 10$ / month
1. Check the alert
2. Click Delete
3. Delete

Logs & Analytics Workspace

1. Almost every Azure resource generates logs
1. Logs might contain data about performance, events, errors etc.

2. Logs records need central repository to be stored and viewed (we as a developer or maintainer can see the logs and query them)
    1. This is the Log Analytics Workspace
2. What is Log Analytics Workspace
    1. A central location for storing, organizing, and analyzing logs
    2. It aggregates logs from all connected, and monitored resources
    3. It uses specialized query language to query logs
    4. It is located in a designated region
    5. It better be the same region of the reosurces to save costs
        1. The traffic will not be cross region
3. Log Analytics Workspace
    1. Types of logs in the workspace:
        1. Event (VM)
        2. Syslog (VM)
        3. Heartbeat (VM)
        4. Performance (VM)
        5. Custom Logs
        6. AzureActivity (Almost every Azure Resource)
        7. AzureDiagnostics (Almost every Azure Resource)
        8. Usage
        9. other tables (VM, Almost every Azure Resource)
    2. All the logs are streamed into log-analytics workspace from which we can view them and query them

Constructing and Using Log Analytics

1. Search: Log Analytics
    1. Log Analytics Workspace
        1. One was created automatically when the subscription was created
        2.
            - Create
            1. Resource group: readit-app-rg
            2. Name: logs-ws-mutthoju (unique)
            3. Region: West Europe (close to me)
            4. Review + Create
            5. Create
        3. Go to resource
            1. Logs (we can view and query the various logs that were generated in our system and streamed to our log analytics)

                1. Select scope (which logs we want to analyze)

                    1. logs-ws-mutthoju (default)
                        1. Click on trashcan
                    2. Select: Azure subscription (all logs by the subscription)
                    3. Apply

                2. Scope has got changed to subscription

                3. Type the following:

                ```
                AzureDiagnostics
                ```

                1. Click: Run
                    1. No results
                        1. Why? The subscription is not configured to stream logs to the current log analytics workspace
                            1. Duplicate the page:
                                1. Search: Subscriptions
                                    1. Azure subscription
                                        1. We want to stream activity logs from the subscription (one of the most important logs - who did anything with Azure)
                                            1. Creation, deletion, modifications, ...
                                                1. Helps in auditing
                                    2. Click: Activity log
                                        1. Export Activity Logs (to stream activity logs)
                                            1.
                                                - Add diagnostic setting (What do we want to happen with logs of subscription)
                                            1. What kind of logs do we want to stream:
                                                1. Name: activity-logs-ws
                                                2. Categories: administrative, secuirty, recommendation
                                            2. Where do we want to export
                                                1. Destination details:
                                                    1. Send to Log Analytics workspace
                                                        1. logs-ws-mutthoju
                                                        2. Save
                                                    2. Archive to a storage account
                                                    3. Stream to an event hub
                                                    4. Send to partner solution (close list of partners that can interact with the logs)
                                                2. Close the page
                                            3. New entry appears
                                                1. It takes a few minutes for the logs to get streamed
                    2. Go back to log analytics workspace:

                        1. Query: AzureActivity

                        2. Run

                            1. Results
                                1. Quite a few columns

1. Click open arrow
    1. SourceSystem: Azure
    2. IP address from which the request was received
    3. Authorization: Of user performing the action
    4. Properties:
        1. Activity succeeded
        2. Entity: Shows something happened
        3. Message: ...
        4. Resource Group
        5. StatusCode: Created (the resource was created)

3. Query:

```
AzureActivity
| project Properties
```

    1. Display only properties field of the records (only one column)
        1. Expand:
            1. entity: VM
            2. Message: ...

4. Ordering results by whatever column we want: (usually by timestamp)

```
AzureActivity
| order by TimeGenerated
```

    1. `TimeGenerated` - always the timestamp column in log analytics

5. **Learn querying**

Log Analytics with Dashboard and Alerts

1. We can pin log query to dashboard
    1. The query and its results will always be available
2. Click: Pin to dashboard
    1. Dashboard: Course Dashboarn
    2. Pin
3. Duplicate the page
    1. Dashboards
        1. Course Dashboard
            1. Customize: Click on the icon
                1. Title: CPU Usage
                2. Update
            2. Save
            3. Edit:
                1. Move to near the clock
                2. Save
4. Go to analytics workspace:
    1.
        - New alert rule (alert based on a query)
    1. Threshold value: 90
    2. Actions:
        1. course-admins-actiongroup
    3. Review + create
    4. ...

App Service Logs

1. Go to catalog-vm
2. Go to inventory app service
    1. App Service Logs
        1. Logs generated by the app service
        2. We need to install ASP.NET Core site extension (enables application logging)
            1. Click to install
        3. Application Logging (Filesystem): On
        4. Level: Info
        5. Save
3. Go to Overview:
    1. Right click on URL and open it in a new tab
    2. Click: Log stream
        1. Shows realtime logs of inventory app service
            1. Go to inventory app and do some activity
    3. Click: Diagnosticsettings (preview)
        1. We can send logs from app service to log analytics workspace
        2.
            - Add diagnostic settings
            1. Category details:
                1. AppServiceHTTPLogs
                2. AppServiceAppLogs
            2. Destination details:
                1. Send to Log Analytics Workspace
                    1. Log Analytics Workspace: logs-course-ws-mutthoju
            3. Diagnostic setting name: inventory-diag
            4. Save
                1. Close the window: The new setting was created
        3. Go back to workspace

1. New Query
    1. Select a scope:
        1. uncheck
        2. readit-inventory (app service)
            1. No data - wait for few minutes and refresh

                1. Query:

                    ```
                    AppServiceHTTPLogs
                    ```

                    1. Logs document HTTP requests received by app service
                    2. Click: Chart
                        1. How many requests were received every minute
    4. There is no support for .Net Core for streaming app logs to analytics workspace (the app service is written using .Net Core)
4. Log Analytics Workspace is a recommended tool

Insights

1. It is a collection of metrics, statistics, and insights about the resource
    1. It is specific to resource type
    2. It is generated automatically
2. Code-based services (App Services, apps on VMs) can integrate Application Insights into the code and thus gain a lot of data about the app usage, performance, etc.

Using Insights

1. Portal:
    1. Search: Storage Accounts
        1. ordersreadit
            1. Monitoring
                1. Insights
                    1. It offers a lot of information about
                        1. Availability of the SA
                        2. Transactions
                        3. Latency
                        4. Performance
                        5. Capacity
                        6. ...
                    2. Transactions by storage type:
                        1. Blob transactions
                        2. File transactions
                        3. Queue transactions
                        4. Table transactions
                    3. Success E2E Latency: 54.83 ms
                    4. Availability: 100%
                    5. Failures: No Failures
                    6. Capacity (most interesting insight)
                        1. It shows the used capacity in the storage account
                            1. Table storage: < 7 MB
                            2. Blob storage: > 1 KB
                            3. File storage: x
                            4. Queue storage: x
                        2. Time range: 14 days
                            1. Overview
        2. catalog-vm
            1. Monitoring
                1. Insights
                    1. Performance
                        1. CPU Utilization
                            1. Change to last 7 days
                        2. Available Memory
                        3. ...
                            1. Helps figure out the sate of VM
                    2. Map
                        1. Displays connections between VMs and other resources
                            1. VM needs to run for few hours for data to show
                                1. Info to figure out what kind of communication exists between this VM and other resources
    2. Another kind of insights: App insights (specifically for app services)
        1. Very useful
        2. Gives a lot of info about performance, availability, reach of the app service

Azure Monitor

1. It is a resource
    1. A central location for all the monitoring aspects of Azure's resources
    2. It provides access to metrics, logs, insights, etc.
    3. It has additional capabilities not found in the individual resources

Using Azure Monitor and Application Insights

1. Turn on catalog-vm
2. Search: Monitor
    1. Monitor
        1. We don't need creation of Monitor
            1. It is pre-created for us and pre-configured

2. On the left: (They provide access to all the metrics of all the resources in the subscription)
    1. Alerts
    2. Metrics
        1. Select scope (the resource for which we want to view metrics)
            1. Expand readit-app-rg
                1. catalog-vm
                    1. Apply
        2. Metric:
            1. Percentage CPU
    3. Logs
        1. Select scope
            1. Expand readit-app-rg
                1. logs-ws-mutthoju
3. Insights:
    1. Virtual Machines
        1. Configure Insights
            1. Not monitored: catalog-vm
                1. Enable
                    1. Configure (takes a few seconds usually)
                        1. Catalog VM is configured to send data to the monitor
        2. Refersh the page
            1. Monitored (enabled)
        3. Performance
            1. Data is sampled every 1 minute by default
                1. CPU Utilization
                2. Available Memory
                3. Bytes Received
                4. Bytes Sent
    2. We can also see insights about other types of resources
        1. Storage Accounts (storage accounts in our subscription)
            1. Transactions in last for hours (time range can be changed)
            2. Timeline of Transactions
            3. Latency of querying the storage account
            4. Capacity:
                1. Size of each storage account
                    1. Can be used to estimate costs that will be associated with the storage account
                        1. How much storage account we are using

Application Insights Tips and Tricks

1. We can add Application Insights SDK to our code (supported for .NET, Java, Node JS & Python)
    1. Gives a full, comprehensive monitoring solution for the app which helps detect:
        1. Request rates
        2. Response times
        3. Failure rates
        4. Dependency rates
        5. Dependency failures
        6. Exceptions
        7. Page views
        8. Load performance
        9. ...
    2. https://docs.microsoft.com/en-us/azure/azure-monitor/app/app-insights-overview

Tags

1. Resource Tags:
    1. Helps in monitoring and cost analysis
    2. Resources are organized into Resource Groups
        1. Sometimes we want more information about resources. But we want to know more info:
            1. Which environment the resources belong? (Test? Production?)
            2. To which app it belongs?
            3. What is the role of the VM?
            4. Which system does it serve?
            5. Which group is responsible for it?
        2. Solution: Resource Tags
            1. Tags help organize resources using name-value pairs
                1. Examples:
                    1. Environment: Test
                    2. App: readit
                    3. Group: the-A-team
        3. Tags can be set during creation of resource or after that
            1. Useful for many scenarios
                1. Resource querying
                    1. Example: Show all resources of the A team
                        1. We can use simple queries
                2. Cost analysis
                    1. Example: How uch did the test environment cost last month?
                3. ... (including monitoring)

Using Tags and the Resource Graph Explorer

1. Search: catalog-vm
    1. We can leave it turned off
    2. Tags
        1. Name: env, Value: test
        2. Name: group, Value: A-team

3. Apply
3. Overview:
    1. Tags added (hit refresh if not showing)
4. Go to resource group:
    1. List of resources
        1. Add Filter:
            1. Filter: env
            2. Value: test
            3. Apply

2. Another useful tool:
    1. Search: resource graph explorer

        1. We can query resources using the tool which is quite similar to what we did with logs

            1. It displays a hierarchical list of resources and services on the left
            2. Open resources
                1. List of resources

        2. Query textbox: (we can query for resources)

```
resources
| where tags.env=='test'
| project name, kind, location, resourcesGroup
```

            1. Shows catalog VM

        3. Play with it

            1. Very useful when we have a lot of resources

Current Architecture

1. We have added App Insights (Monitoring) connected to the inventory app service
    1. We can monitor the availability of the app service and get notified when there are any availability problems with the app service

Security in Azure

Introduction

1. Summary of all security aspects important to know when working with Azure
    1. Security is one of the most important aspects when working in the cloud (this section covers all important security aspects we need to know)
2. Security:
    1. Security is one of the most important parts of every system
        1. Azure offers a lot of security measures for its resources
            1. It's extremely important to use those measures and follow security patterns to avoid security incidents
    2. Most of them are covered
        1. We need a summary and some new techniques

VM Security Best Practices

1. Restrict access to the VM as much as possible
    1. Make sure only the required ports are open to the internet (22/1389/443/80) (not anything else)
2. Limit access to specific IP addresses when possible
    1. Mainly for SSH and RDP
        1. Must be accessible only to specific users authorized to log into the machine
3. Prefer using Bastion for accessing the VM
    1. No need to open ports (especially for SSH and RDP)
4. If the VM is not public facing - place it in a VNet that's not connected directly to the internet
    1. We will make it a lot harder to hack into the machine

Networking Security Best Practices

1. VNet that contains private resources only - should not be exposed to the internet
2. ALWAYS use NSG to restrict access to subnets
3. Use Service Endpoints / Private Endpoints to restrict access to resources [that should be connected to the VNet]
4. Use the Hub-and-Spoke security model
    1. Single VNet
    2. App Gateway is located in it
        1. It has an external connection
    3. Other VNets containing private resources are connected to the VNet via peering and NSGs
        1. Access to the VNets is protected

Database Security Best Practices

1. Use encryption at rest and encryption at trasit (usually on by default)
2. Connect DB to relevant VNet using Services Endpoint / Private Endpoint
3. Access DB from app using Managed Identities (whenever possible)
    1. Do not store connection strings as plain text
4. Use DB's Firewall rule to restrict external access (only specific IPs can access the DBs externally)

App Services Security Best Practices

1. Don't expose it directly to the internet
    1. Always use Application Gateway in front of the app service
    2. Connect App service to App Gateway's VNet using Service Endpoint / Private Endpoint

1. Direct public access to the App Service is blocked
3. Use Azure AD for Authentication when relevant, enforce it with MFA
4. Use Managed Identity to access other resources when possible
    1. Especially when connecting from the app service to the database
    2. Prefer not to store the connection string to the database in clear text in some obscure config file

KeyVault

1. Many apps have secrets that need to be kept safely
    1. Connection Strings (to DB)
    2. Keys (for encryption and decryption)
    3. Certificates
    4. API Keys
    5. ...
2. The secrets are usually kep in configuration files, configuration DB etc.
    1. They are not really secure...
        1. The configuration are usually kept in clear text
        2. With DB also, the data is usually stored in an easy to read format
            1. Anyone who has the permission to read from the DB can access the secrets
    2. Solution: KeyVault
        1. What does the KeyVault do?
            1. It safely stores secrets of various types
            2. It has very restricted access
                1. It needs Azure AD authentication to access secrets and other config info that is stored inside it
            3. It supports Hardware Security Modules (for enhanced security)
            4. Easily manageable
            5. Accessed via REST API
                1. Extremely easy to work with KeyVault once we get access to it
            6. Very cost effective
                1. Million operations against keyvault - $3 / month
                    1. Operation: Read a secret from KeyVault

Troubleshooting KeyVault

1. We can configure a Private Endpoint for the KeyVault
    1. Only connections from catalog VM's VNet will be allowed
    2. And our own IP address which will be configured in Network section of the KeyVault
    3. Bug in Azure:
        1. Azure ignores the IP address allowed in the Firewall if a Private Endpoint was configured during the creation of the KeyVault
            1. The following error message is shown:
                1. Public network access is disabled and request is not from a trusted service nor via an approved private link
            2. Solution:
                1. Instantiate a new KeyVault
                2. Do NOT configure Private Endpoint for the KeyVault
                3. Add the secret
                4. Configure the Private Endpoint from the Networking section
                5. Continue working with the new KeyVault instead of the one originally created

Using KeyVault in the Catalog App

1. Integrating KeyVault into Catalog code:
    1. Start catalog-vm VM
2. Search: KeyVault
    1.
        - Create
        1. Resource Group: readit-app-rg
        2. Key vault name: readit-kv-mutthoju (globally unique)
        3. Pricing Tier: Standard
            1. Premium: Has Hardware Security model feature
        4. Access policy
            1. We can define who can access data in the KeyVault
            2. Check: Azure Virtual Machines for deployment
                1. VMs that are deployed inside Azure within our subscription will be able to access the KeyVault
                    1. Makes it easy for code to access data within the KeyVault
        5. Networking
            1. Private endpoint (a method of connecting privately two resources)
                1. Newer method of protecting traffic (used over service endpoint)
                2. We need to select the VNet we want to connect privately
                    1.
                        - Add
                        1. Resource group: readit-app-rg
                        2. Location: West Europe
                        3. Name: vault-pe
                        4. Target sub-resource: Vault
                        5. Networking:
                            1. VNet network: readit-app-vnet
                            2. Subnet: default (subnet where the catalog VM is located)
                        6. OK
        6. Review + Create
        7. Create
3. Open VS Code in Catalog project:

    1. Adding support for working with KeyVault (we want code to read connection string from KeyVault instead from config file)

    2. Terminal:

```
dotnet add package Microsoft.Extensions.Configuration.AzureKeyVault
```

3. Copy the contents of the attached Program.cs file and paste it in Program.cs file

```
...
.ConfigureAppConfiguration(ctx, builder) -> {
    var config = builder.Build();
    builder.AddAzureKeyVault(config["keyVault:BaseUrl"]);
});
```

4. Portal: readit-kv-mutthoju
    1. Vault URI: https://readit-kv-mutthoju.vault.azure.net
    2. Three types of things:
        1. Keys
        2. Secrets
            1. Connection string to a DB is considered to be a secret
        3. Certificates

Publishing the KeyVault Code and Using Insights

KeyVault Tips and Tricks

1. KeyVault can be used from on-prem apps too
    1. Register the app in Azure AD (making it identifiable)
    2. Add an access policy to the KeyVault allowing access to the identity
    3. There are client libraries for Python, Java, Node.JS: https://docs.microsoft.com/en-us/azure/key-vault/general/client-libraries
2. KeyVault has an automatic fail-over mechanism
    1. In case of a complete region shutdown, requests to KeyVault are automatically routed to paired region where its data is replicated
        1. We won't even know about it

A Quick Note About Security Center

1. Security Center: Main tool to get an overall look on cloud security in single pane of glass
    1. Renamed to **Microsoft Defender for the Cloud**

Security Center

1. Security Center: It is a central location for monitoring and alerting security-related issues in Azure
    1. It displays a summarized list of problems found in the subscription's resources
        1. In some cases, allows a single-click fix
    2. It is a good practice to take a look at security center once in a while
        1. To see if there are any security related issues that need our attention

Using Security Center

1. Portal
    1. Search: Security Center
        1. Getting Started page
            1. Skip
                1. Overview: A summary of security related info generated by security center
                    1. Number of subscriptions monitored
                    2. Number of active recommendations generated security center
                        1. 0 - check again
                2. Most important pages:
                    1. Recommendations
                        1. Same recommendations that are in inventory page
                        2. They are grouped by topic (not by resource)
                        3. Each recommendation gets a score (score by subscription)
                            1. Manage access and permissions
                                1. Managed identity should be used in your web app (completed)
                                2. Log Analytics agent should be installed on your VM
                                    1. Click:
                                        1. Affected resources
                                            1. Virtual Machines - complies with the recommendation (hence marked healthy)
                                3. Apply adaptive application Control (no resources that are unhealthy)
                                    1. It shows status of security for past hours
                                4. Implement security best practices
                                    1. Key vaults should have purge protection enabled
                                        1. Purge protection: If we delete a value from the KeyVault, it is not deleted (it is only soft deleted)
                                            1. Purge protection means that we cannot hard delete an item after soft delete until a retention period expires
                                                1. Prevents attackers from deleting secrets and certificates from KeyVault without our ability to restore it
                                        2. Solution: Go to KeyVault
                                            1. Properties
                                                1. Purge protection: Enable purge protection
                                                2. Save
                                            3. Go back to Security Center
                                                1. Freshness interval: How often the recommendation refreshes itself
                                                    1. 30 minutes
                                5. Public network access on Azure SQL database should be disabled
                                    1. ...accessed only from private endpoints...
                                        1. From VM say
                                        2. Specific IP address is okay

2. Security alerts
3. Inventory
    1. What are the resources security center is looking at
        1. Total Resources (monitored): 7
        2. Unhealthy Resources: 6
        3. Status of each resource:
            1. Recommendation column
                1. Click: readit-app-vnet
                    1. Recommendation: VNet should be protected by Azure Firewall
                        1. Status: Low
                    2. Click: Some more info
                        1. Remediation steps
                2. Click: catalog-vm
                    1. Two High
                        1. Install endpoint protection solution on VMs
                        2. Windows Web server should be configured to use secure communication protocols
                            1. Click:
                                1. Remidiation
    4. Security alerts: Alerts generated by security center to alert us when something really risky might have happened in our resources
3. Take a look once a week, twice a month or once a month

Current Architecture

1. Added KayVault
    1. Used by catalog app (on VM)
        1. VM qeries KeyVault for connection string using managed identity
            1. Much more secure

DR in Azure

Introduction to DR

1. What is DR in Azure?
    1. DR - Disaster Recovery
        1. **A plan to recover from a complete shutdown of a Region**
            1. Usually as a resut of a disaster (earthquake, flood, etc.)
        2. Some apps require it, some don't
            1. **Important step is to figure out whether we actually need DR**
        3. Might have substantial cost aspects
        4. Remember: A complete shutdown of a Region is extremely rare
    2. How DR Works?
        1. In order to set up DR, we need to do the following:
            1. Select a DR site
                1. A secondary Region that will function as our primary in case of a disaster
                2. Configure it to be ready for activation when necessary
        2. Example:
            1. Readit app is running in West Europe
                1. DR plan: North Europe
                    1. It has the same architecture
                2. West Europe is a primary region
                    1. If it is gone (could be any reason)
                        1. North Europe region becomes primary region
                3. North Europe is a secondary region
2. How to design and implement it?
3. What resources support it?

DR Concepts

1. Two concepts:
    1. Hot
        1. Failover to secondary site happens automatically with no downtime
        2. No data loss
            1. Nothing is lost
                1. We have all the data ready for us that we had in the primary region
        3. Requires duplicate infrastructure
        4. The most expensive method
            1. We pay twice for everything
    2. Cold
        1. Failover to secondary site takes some time
        2. Might be manual
            1. We might need to setup resources
            2. We might need to setup routing
            3. We might need to restore data
        3. Some data might be lost
        4. Less expensive
            1. No duplicate infrastructure
2. Hot or Cold - how to decide?
    1. Depends on the system's requirements
        1. A global e-commerce website, serving millions of customers - probably hot
            1. The website cannot allow itself any downtime
        2. An HR app for the organization - definitely Code (if at all...)
            1. Usually no DR
3. RPO / RTO
    1. RPO

1. Recovery Point Objective
    1. How much data we allow ourselves to lose in case of a disaster
    2. Usually measured in minutes
        1. In other words - what's the frequency of data sync to the secondary region
            1. Example: We have an RPO of 5 minutes (in case of a disaster, we lose last 5 minutes of data)
    3. If the data is extremely critical, we might need a better RPO than 5 minutes
2. RTO
    1. Recovery Time Objective
        1. How much downtime we can tolerate in case of a disaster (of the app)
    2. Usually measured in minutes
        1. In other words - How long it should take before the system is up again
    3. When system is up it is not necessarily with the most up to date data, depends on the RPO
4. RPO and RTO - How to decide?
    1. Depends on the system's requirements
    2. Example: A massive reporting system will probably go for low RPO (needs upto date data), but can compromise on the RTO (not realtime and doesn't control any critical system)
    3. Example: A global chat will focus on RTO
        1. We don't focus on data so RPO can be High because we only focus on current data and history is not very important

Basics of DR Implementation

1. Regions:
    1. Primary
        1. Data (how to integrate it)
            1. We need to define some kind of sync / backup
                1. Type of sync / backup will depend on RPO
                    1. The lower the RPO - The higher the chance we use sync
                    2. The higher the RPO - The higher the chance we use backup
        2. Compute / Networking
    2. Secondary
        1. Data
        2. Compute / Networking
            1. They can be: Active / Passive (Depending on the RTO)
                1. If Low RTO - Active (keep them running all the time)
                2. If High RTO - Passive (not running and will be turned on when needed)
    3. Routing (DNS)
        1. DNS that routes requests from the end-user to the app
            1. User request is directed to the primary region by default
            2. If the primary region is no longer active?
                1. Steps:
                    1. Restore the data (if not synced in real time)
                    2. Activate the compute resources in secondary region (if not active)
                    3. Modify routing (auto or manual)
                2. DNS will then route the requests to the secondary region insted of the primary region

DR of Data in Azure

1. Main question when designing the DR of data is:
    1. What is the RPO?
        1. Or how much data loss do we tolerate?
        2. If RPO = 0 (no data loss in case of disaster)
            1. We need database that always syncs with the secondary region
                1. Currently - three such databases in Azure
                    1. Azure SQL (with Geo-Replication and Failover Group config)
                    2. Cosmos DB (with multi-region account)
                    3. Azure Storage (with GRS redundancy)
            2. Example:
                1. Primary and Secondary has Cosmos DB R/W Region
                    1. Cosmos account is multi-region
                        1. Real-time replication is enabled
        3. If RPO > 0 (some data can be lost)
            1. Ensure DB's backup frequency is compliant with the RPO
                1. The backup storage should be GRS
                    1. It is replicated to the secondary region
                        1. Types of storage backup behavior
                            1. LRS
                            2. GRS
            2. Example:
                1. Azure MySQL
                    1. Geo-Redundant option:
                        1. Transaction log backups occur every five minutes
                2. Architecture:
                    1. Primary: MySQL, Storage Account
                        1. Normally, MySQL backs up the data to the Primary Storage Account
                        2. Storage Account replicates the data (GRS) to Storage Account in the Secondary Region
                    2. Secondary: Storage Account
                        1. If Primary goes down:
                            1. A new Azure MySQL is created
                            2. Data is restored from the Storage Account in the Secondary Region to the MySQL just created
                3. Implementation:
                    1. Create MySQL server page:
                        1. Data source (of new database): Backup (of another database)
                            1. This option allows you to restore from the most recent geo-redundant backup of any server in this subscription. The storage capacity of the server will be determined by the backup. Select a backup to continue
                        2. Backup: ...
                4. DR of Data in Azure:

1. Note:
    1. The ROP in the previous example is minimum 5 minutes (the backup frequency)
        1. Azure MySQL may not be a good choice
            1. There is no other database other than the real-time replication that has a frequency lower than 5 minutes
    2. The second example is much cheaper
        1. No secondary active database is needed when primary is active
2. Figure the RPO first and then design the strategy accordingly

DR of Compute in Azure

1. Main question when designing the DR of compute is:
    1. What is the RTO?
        1. Or How much downtime can we tolerate?
            1. How long can we tolerate not having the compute
    2. If RTO = 0 (no downtime in case of disaster)
        1. Compute in the secondary region should always be up and running
        2. Example:
            1. Primary:
                1. Compute - active
            2. Secondary:
                1. Compute - active (no downtime because is already up and running)
        3. For RPO = 0, Real-time Replication
    3. If RTO > 0 (some downtime is tolerated)
        1. Either:
            1. Have non-active (passive) compute on standby in secondary region
            2. Instantiate the compute when disaster occurs in the secondary region
        2. Example:
            1. Primary: Active Compute
            2. Secondary: Passive - offline Compute
                1. Compute in secondary region becomes active when Primary goes down
        3. Pros:
            1. The second example is much cheaper, no secondary active compute is needed when primary is active
                1. We don't pay for compute in the secondary region while the primary region is active

Routing in DR

1. During DR users should be routed to the secondary region
    1. Consider: Routing (DNS)
        1. When primary region goes down, the secondary region becomes active
        2. The routing element will need to route to the secondary region
    2. Three major methods:
        1. Inform the users about the new address of the app (in the secondary region)
            1. The users will simply direct their browsers to the new app
                1. No central routing mechanism
        2. Manually change DNS record to point to the secondary region
        3. **Use automatic routing**
            1. **Azure has two automatic routing services**
                1. **Traffic Manager**
                2. **Front Door**

Azure Traffic Manager

1. It is a DNS-based traffic load balancer
    1. It enables traffic distribution across global Azure regions
    2. It provides high availability and responsiveness
    3. Working principle:
        1. User makes a DNS query (www.abc.com) to Traffic Manager Profile with DSN name
        2. The Traffic Manager decides which region to go to...
            1. The decision is based on various algorithms
        3. The Traffic Manager returns the IP address of the appropriate region to the user
        4. User in turn makes the call to the region by making use of the IP address received from the Azure Traffic Manager
2. Routing Algorithms
    1. Priority (gives the DR capability)
        1. Always use primary service, when it's down - using backup endpoints (That's the DR...)
    2. Weighted
        1. Distribute traffic across endpoints according to weights defined by us
            1. West Europe - 60% of the traffic
            2. North Europe - 40% of the traffic
    3. Performance
        1. Uses the closest region to improve latency
            1. Traffic Manager looks at where the user request came from and will route the request to the closest region
    4. Geographic
        1. Direct traffic to specific geography based on the location of the DNS query
    5. Multivalue
        1. Returns a list of healthy endpoints so the client can choose what to do with them
    6. Subnet
        1. Maps source IP to endpoint
3. Pricing:
    1. Traffic Manager is extremely cheap
        1. Priced per DNS queries in the millions
            1. 10 million queries -> $5 per month

Using Azure Traffic Manager

1. Portal: Search Traffic Manager
    1. Traffic Manager Profiles

1.
    - Create
    1. Name: readit-mutthoju [.trafficmanager.net] (globally unique name)
    2. Routing method: Priority (DR capability)
    3. Subscription
    4. Resource group: readit-app
    5. Create
2. Go to resource
    1. Configuration
        1. Parameters:
            1. Probing interval
                1. The interval between health checks that the traffic manager does to the regions
                    1. Every 30 seconds, the Traffic Manager will check the health of the primary region, and if it fails, then it will redirect the traffic to the secondary region
                        1. Change it to 10 seconds
            2. Tolerated number of failures
                1. How many failures should the traffic manager recieve before announding that the primary region has failed / shut down
                    1. Change it to 1
                        1. After the first time the Traffic Manager detects failure from the primary region, it will redirect the traffic to the secondary region
            3. Probe Timeout:
                1. Change it to 9 seconds
        2. Save
            1. Standard 80 port is supported by the traffic manager
    2. Endpoints: (Traffic manager will send requests to these endpoints)
        1.
            - Add
            1. Type: Azure endpoint (routing the request to an Azure service)
            2. Name: readit-primary
            3. Target resoruce type: App service
            4. Target resource: readit-inventory (West Europe)
            5. Priority: 1 (highest priority)
                1. The traffic manager whenever possible will route the traffic to this endpoint
            6. Add
    3. Go to App Service:
        1. readit-inventory
            1. Disable Authentication of the App Service - to make Traffic Manager work with the authentication, we have to define custom domain for the App Service
            2. Go to Authentication/ Authorization
                1. App Service Authentication: Off
                2. Save
            3. Go to overview
                1. Right click on the URL and open it in a new tab
            4. Scale up (App Service plan)
                1. Upgrade to Standard (Traffic Manager will not work with Free App Service)
                    1. Production
                        1. S1
                        2. Apply
            5. Go back to Overview
                1. App Service plan: S1
    4. Go to Traffic Manager Overview (readit-mutthoju)
        1. Copy the DNS name
            1. Open a new page in the browser, paste and hit enter
                1. It opens Inventory Service (through traffic manager)
                    1. The request was routed to the App Service

2. Secondary App Service:
    1. Portal
        1. Search: App Services
            1.
                - Create
                1. Resource Group: readit-app-rg
                2. Name; readit-secondar-mutthoju
                3. Runtime stack: .NET Core
                4. Region: North Europe (different from primary)
                5. Size: S1 or higher
            2. Review + Create
            3. Create
        2. Right click on URL and open in a new tab
            1. Displays a generic page
        3. Traffic manager:
            1. Endpoints:
                1.
                    - Add
                    1. Type: Azure endpoint
                    2. Name: readit-secondary
                    3. Target resource type: App Service
                    4. Target resource: readit-secondary (North Europe)
                    5. Priority: 2
                    6. Add
    2. Refresh the traffic manager URL
        1. Routes request to primary app service
    3. Go to readit-inventory (app service)
        1. Click: Stop
            1. Yes
    4. Wait for a few seconds (takes effect after probing interval)
    5. Refersh traffic manager URL
        1. Re-directs the request to secondary app service

6. Start primary app service
7. Wait for a few seconds
8. Refersh traffic manager URL
    1. Re-directs the request to primary app service

Azure Front Door

1. Azure Front Door: It is a global entry point for web apps
    1. It works on Layer 7 (HTTP/HTTPS)
    2. It has multiple routing methods
    3. It is **similar to application gateway but in global scale**
2. Features: (similar to what we have in application gateway)
    1. URL-path based routing
    2. Session affinity
    3. SSL Offloading
    4. Web Application Firewall (WAF) integration
    5. URL Rewrites (?)
    6. HTTP/2 support
3. Pricing:
    1. Based on outbound and inbound traffic
        1. Traffic manager doesn't have any inbound traffic
            1. It just returns an IP address but traffic doesn't go through traffic manager
        2. Traffic goes through Azure Front Door
    2. There is special pricing for Web-Application Firewall (WAF)
    3. Overall, the price is not too high (~ $41.30)

Using Azure Front Door

1. Portal: Search: Front
    1. Froont Door and CDN Profiles
        1. It is a group of products
            1. Choose: Explore other offerings (Azure Front Door is a CDN and it not used here)
            2. Choose: Azure Front Door (classic)
            3. Continue
            4. Subscription:
            5. Resource Group: readit-app-rg
            6. Next: Configuration
                1. Frontend (address where the end-user will go to)
                2. Backend pools (compute pools the frontdoor will route to)
                3. Routing rules (connect frontend to the backend pool)
            7. Add a Frontend host:
                1. Host name: readit-mutthoju (globally unique)
                2. Session affinity: Disabled
                3. Web Application Firewall: Disabled
                4. Add
            8. Add a Backend Pool:
                1. Name: readit-pool
                2. Backends:
                    1.
                        - Add a backend
                        1. Backend host type: App service
                        2. Backend host name: readit-inventory.azurewebsites.net
                        3. HTTP Port: 80
                        4. Priority: 1
                            1. Front-door will always route to this app service (this is the only one with this priority)
                        5. Weight: 60
                            1. Allows us to distribute traffic according to predefined proportions between various backends
                            2. Suppose we have two app services at priority 1, then we can assign that one of them will get 60% of the traffic and the other will get 40%
                        6. Status: Enabled
                        7. Add
                    2.
                        - Add a backend
                        1. Backend host type: App service
                        2. Backend host name: readit-secondary.azurewebsites.net
                        3. HTTP Port: 80
                        4. Priority: 2
                        5. Add
                    3. Interval (seconds): 10
                    4. Sample size: 1
                    5. Successful samples required: 1
                        1. Load balancer in the front-door needs only a single failure to declare the backend as failed
                            1. It needs a single successful request to declare the backend healthy
            9. Add Routing Rules:
                1. Name: readit-rule
                2. Accepted protocol: HTTP and HTTPS
                3. Frontends/domains
                    1. the created frontend is selected
                4. Pattern matching: None
                5. No URL rewrite or caching
                6. Add
            10. Review + Create
            11. Create
            12. Go to resource
                1. Click the URL (open in a new tab)
                    1. If error, the front-door might not be ready so wait for some more time

2. Go to the app-service
    1. readit-inventory
        1. Stop
    2. Refresh the front door page
        1. We will see the other app service
    3. Start
    4. We want traffic to only go through the front-door
        1. Go to Networking in the app service
        2. Configure Access Restrictions
            1.
                - add rule
                1. Name: fd-traffic-only
                2. Priority: 100
                3. Source settings:
                    1. Type: Service tag (prefix)
                    2. Service Tag (prefix)
                        1. AzureFrontDoor.Backend
                            1. It means that the only traffic that was originated from the Azure front-door and is directed to the front-door backend is allowed to the app service
                            2. Add rule
        3. Referesh front-door url
            1. Takes some time for the changes to get reflected
    5. Go to app service
        1. Overview
            1. Click the URL
                1. Error 403 - Forbidden
3. It is one of the best routing services that Azure can offer

Traffic Manager vs Front Door

1. Which one to choose?
    1. Generally - if you need HTTP-related capabilities, go with front-door
        1. Examples:
            1. URL-path based routing
            2. SSL Offloading
            3. Web Application Firewall
        2. Front-door might respond much faster to backend pools so that if one goes down, front-door will switch quickly to the secondary one as compared to traffic manager
    2. Otherwise - go with Traffic Manager
        1. It is usually much cheaper

Current Architecture

1. Has a secondary region
2. Traffic manager
3. Front Door
    1. We have added DR capabilities

Managing Costs in Azure

Introduction

1. Cost is one of the most important aspects of every cloud system
2. Many decisions are made based on their cost impact
3. Cost should be: (to avoid surprises when we get monthly bill)
    1. Transparent
    2. Predictable
    3. Monitored
4. Azure Cost Management is the central hub for all cost aspects
    1. It displays reports with sophisticated filters
    2. It makes predictions
    3. It enables alerts
    4. It can manage costs of resources outside Azure
        1. AWS
        2. Google Cloud
    5. This is the place to handle costs in Azure

Looking at Cost Management

1. Search: Cost Management + Billing
    1. Billing scopes
        1. What is the scope of cost management we are going to see.
            1. Account
    2. Cost Management
        1. Allows us to see various information about the cost structure of the cloud resources and various other information
        2. Billing > Invoices
            1. List of previous invoices prepared for us
        3. Cost Analysis
            1. Most interesting info
            2. Actual cost
            3. Forecast for the month
            4. Graph
            5. We can select date
                1. Last month
                2. 7 days

6. Add filters
    1. location
        1. eu west
7. Cost by service name
8. Cost per location
    1. Cancel location filter
    2. This month
9. Cost per subscription

4. View field:
    1. Cost by resource
        1. List of resources in the subscription
            1. Take a look at the list once in a while
            2. Sorted by cost in reverse order
        2. If we select a resource it shows the breakdown
            1. Say:
                1. Fixed cost
                2. Capacity Units
        3. We can download and open it in excel
            1. We can sort and filter in excel
5. Cost alerts (based on budget)
    1.
        - Add
    1. Name: ...
        1. As before
6. Advisor recommendations:
    1. **Will recommend us with actions that we can do in order to save costs on Azure**
        1. Buy VM reserved instances to save money over pay-as-you-go costs
            1. But we are using auto-shutdown policy

Azure Policy

Introduction

1. What is it?
    1. Enforcing organizational standards and compliance at scale is not simple
        1. Quite important in the Cloud
            1. There are many standards and restrictions an organization would want to employ on its resources
                1. Examples:
                    1. Not anyone is allowed to instantiate any kind of resource
    2. Role-based access helps a little, but not enough
        1. Quite limited in its capabilities
        2. It focuses on permission and authorization aspects
            1. Not on the resources itself
2. Examples:
    1. All VMs should be built on a specific region
        1. If the system is targeted at Australian audience
            1. We want to ensure that no VMs are created in the other regions by mistake
    2. Only a specific types of VMs are allowed to be built
        1. Example, GPU enabled VMs are extremely expensive
            1. We don't want those VMs to be created in the subscription
    3. Tags must be specified on all resources in the resource group
        1. For efficient Monitoring
    4. App Services should only be accessible over HTTPS
        1. Not expose HTTP protocol
3. Azure Policy:
    1. Azure Policy is the mechanism for implementing the above:
        1. It allows:
            1. Defining Policies
            2. Assigning the policies to scopes (Subscription, Resource Group, Specific Resource)
                1. Scope: What are the resources the policy is applied to
        2. It is free (aside from auditing guest machines - rarely happens)
4. Azure Policy Evaluation:
    1. Policies are evaluated:
        1. When a resource is created/updated/deleted in a scope
            1. Azure will make sure the resource complies with the policy
        2. When a new policy is assigned to a scope
            1. Azure will make sure that all resources in the scope will comply with the new policy
        3. When a policy is updated
            1. Azure makes sure that the update is applied to all the resources
        4. Once every 24 hours (compliance evaluation cycle)
5. Azure Policy Outcome
    1. As a result of a non-compliant resource:
        1. Resource change is denied
        2. Resource change is logged (but not denied)
        3. Resource is altered before the change
        4. Resource is alterned after the change
        5. Related resources are deployed
6. Azure Policy Concepts:
    1. Definition
        1. The policy definition:
            1. What is allowed
            2. To what resources (eg. only specific VM sizes can be built)
                1. Resource: VM
                2. Allowed: Specific size
    2. Initiative
        1. Logical grouping of definitions

1. eg. VM definitions
    1. If we have multiple definitions all related to VMs, we might want to construct a new initiative called VM definitions and put all definitions related to VMs in it
3. Assignment
    1. Assigning the definition or initiative to a scope (Subscription, Resource Group, Individual Resource)
4. Effect
    1. What is the outcome regarding non-compliant resource

Defining Initiative and Policies

1. portal
    1. Search: Policy
        1. Select Policy
            1. Dashboard
                1. There are some default policies that Azure assigns for every newly created subscription
            2. Click on ASC Default initiative
                1. List of groups inside the initiative
                2. Select Policies: Separate policies
                    1. Storage account should use a private link connection
                        1. Every storage account should be connected to a resource using private link connection
                    2. ...
    2. Select: Definitions
        1.
            - Initiative definition
            1. Initiative location: subscription
            2. Name: VM rules
            3. Description: Policies for new VMs
            4. Policies: (what is allowed and not allowed and for which resource)
                1. Add policy definition
                    1. Search: location
                        1. Allowed locations (what are the allowed locations we can instantiate resources in)
                            1. Click the definition:
                                1. Read the description
                            2. Add
                2. Add policy definition
                    1. Search: sku
                        1. Allowed virtual machine size SKUs (limit the size and types of VMs that we can instantiate in our organization)
                        2. Add
            5. Policy parameters
                1. Allowed location: UK West, West Europe
                2. Allowed virtual machines SKUs: Standard_D2
            6. Review + Create
            7. Create
    3. Scope the initiative will apply to:
        1. Assignments
            1. Assign Initiative
                1. Scope: Resource Group
                    1. Subscription
                    2. Resource Group: readit-app-rg
            2. Initiative Definition:
                1. VM rules
                2. Select
            3. Assignment name: VM rules
            4. Review + create
            5. Create
    4. It might take upto 30 minutes to take effect

Custom Policies

1. So far we've used built-in policies
    1. We can construct our own if there isn't one that satisfies our needs
        1. Look very closely at existing definitions and sample, we might find what we're looking for
            1. We will be rarely defining a custom policy (there is usually a built-in policy or an example in github that satisfying our needs)
2. Custom Policies Authoring:
    1. Policy definitions are JSON-based documents
    2. They describe the various properties of the policy, and the rule
        1. Example: JSON document
            1. "displayName": "Deny storage accounts not using only HTTPS"
                1. If we instantiate a storage account and allow regular HTTP traffic into it, the policy will deny the storage account
            2. "effectType": "Deny" (do not allow creation)
            3. "if": "allOf": "equals": "type": "Microsoft.Storage/storageAccounts" & "field": "Microsoft.Storage/storageAccounts/supportHttpsTrafficOnly": "notEquals": "true"
                1. "then": "[parameters('effectType')]"

Defining Custom Policy and Testing the Policies

1. Go to Definitions:
    1.
        - Policy definition
        1. Definition location: our Subscription
        2. Name: Storage Account Required to ALlow only HTTPS Traffic
        3. POLICY RULE: Copy the contents of the attached file and paste
        4. Save
    2. Assignments:
        1. Scope: readit-app-rg
        2. Policy definition: Select firts one

3. Select
4. Review + Create
5. Create

3. Compliance:
    1. VM rules:
        1. 5 / 60 are not compliant
        2. click
            1. Details
                1. Click the non-compliant resources
                    1. catalog-vm
                        1. Allowed virtual machine size SKUs (size is not Standard_D2)
                    2. readit-secondary
                        1. Allowed locations
4. We can try to instantiate a non-compliant resource
    1. Virtual machines
        1.
            - Add
            1. Virtual machine name: test-policy-vm
            2. Size: See all sizes
                1. N Series
                1.Select
            3. username and password
            4. Review + Create
                1. Validation has failed
                    1. Click to view the details
                        1. Resource was disallowed by policy
                        2. Shows the policy definition violated
    2. Storage account
        1. ordersreadit
            1. Configuration:
                1. Secure transfer required: Disabled
                2. Save
                    1. Resource was disallowed by policy
                        1. HTTP is not allowed
                        2. Shows the policy

Completing the Demo and Saving Costs

1. Option 1: Delete the resource groups containing readit app
2. Option 2: To leave some of the resources
    1. Remove application gateway: most expensive reosurce used
        1. Point Traffic Manager & Front Door to point to inventory service directly
    2. Remove the AKS: Second most expensive resource used
        1. Move the code to new app service in free tier
    3. Ensure that the app services are in free tier (Traffic manager will not work with free app services)
    4. Ensure that VMs are turned off
        1. They need to have auto-shutdown policy
    5. Ensure that CosmosDB utilizes free tier
    6. Ensure that Azure SQL uses the cheapest configuration possible
    7. The above ensure that the monthly payment is just a few dollars (mainly on AzureSQL)

Architecting Apps for Azure

Introduction

1. Architecting for the cloud is different than classic software architecture
2. Two main differences
    1. Use existing services (in the cloud)
        1. Azure contains hundres of services
            1. Whenever possible - use them
        2. The services are usually: (we usually don't find with un-managed services)
            1. Managed
            2. Reliable and Scalable
            3. Cost Effective
    2. Consider cost
        1. Cloud architecture is cost oriented
            1. Always factor in the cost of the cloud service
        2. You'll sometimes go for limited service due to cost reasons
            1. Example: Storage Queue vs Service Bus (more sophisticated but much more expensive)
3. Topics:
    1. Review of various services we used and explain how to select the right one
    2. To discuss:
        1. Compute
        2. Data
        3. Messaging
        4. ...

Choosing Compute Platform

1. Strategy:
    1. Start:
        1. Is it a new app?
            1. No
                1. Is it a legacy app?
                    1. No

1. Does it use system resources? (registry, special networking, ...)
    1. No
        1. Is it container based?
            1. No
                1. App service
            2. Yes
                1. AKS
    2. Yes
        1. VM (allows us full control of the platform and its capabilities)
2. Yes
    1. VM (legacy apps will not work with app-services, functions or AKS, ...)
2. Yes
    1. Can it be separted to functions?
        1. No (for the parts that cannot be separted into functions)
            1. Can it be moduled with services?
                1. No
                    1. App Service
                2. Yes
                    1. AKS (using Docker images)
        2. Yes
            1. Functions (whatever is possible to be put in functions because functions are extremely easy to develop and are extremely cost effective)

Choosing Data Platform

1. Data Types
    1. Relational
        1. Used For: Structured data
        2. Examples: Items in store, demographic data
        3. Options in Azure:
            1. Azure SQL
            2. MySQL
            3. PostgreSQL
    2. NoSQL
        1. Used for: Semi-structured data
        2. Examples:
            1. Reviews
            2. Log records
            3. when flexibility is required
        3. Options in Azure:
            1. Cosmos DB (with SQL, Mongo, Azure Table API)
    3. Graph
        1. Used for: Data representing relationships
        2. Examples: Family tree
        3. Options in Azure: Cosmos DB (with Gremlin API)
    4. Blob
        1. Used for: Files, videos, docs
        2. Examples: Items' photos
        3. Options in Azure:
            1. Azure Blob Storage
2. Steps:
    1. Step 1: Identify the exact data type we are going to store
    2. Step 2: Select the database that we are going to use to store the data
        1. For relational database:
            1. Depends on taste and experience
            2. Azure offers the best SLA and best availability for Azure SQL, MySQL, PostgreSQL
                1. Azure SQL: Improved SLA and availability
                    1. Choose if we don't have any prior preference

Choosing Messaging Platform

1. Service
    1. Storage Queue
        1. Used for: Dead simple queueing
        2. Guarantees Order: Yes
        3. Max Message Size: 64KB
        4. And also:
            1. Extremely Simple
            2. No additional cost
    2. Event Grid
        1. Used for: Event driven architectures
        2. Guarantees Order: No
        3. Max Message Size: 1MB
        4. And also:
            1. Great integration with other services
    3. Service Bus
        1. Used for: Advanced queueing solutions
        2. Guarantees Order: Yes
        3. Max Message Size: 256KB
        4. And also:
            1. Advanced messaging features
            2. Durable
    4. Event Hubs
        1. Used for: Big data streaming
        2. Guarantees Order: Yes
        3. Max Message Size: 1MB

        4. And also:
            1. Low latency
            2. Designed for heavy load
2. Steps:
    1. Step 1: Define what we are exactly looking for
        1. Check "Used for"

Implementing Security

1. Extremely important in the cloud
2. Use the best practices discussed in the Security section
    1. Mainly:
        1. Restrict access to VMs and App Services
        2. Use NSG (whenever possible)
        3. Use encryption in data stores (in most cases is turned on by default)
        4. Use strong authentication

Implementing Logging and Monitoring

1. Azure offers various logging and monitoring tools
    1. Integrate the tools in the architecture
2. Utilize alerts to notify on any exceptional situation
    1. Such as:
        1. CPU
        2. Memory leaks
        3. Exception
        4. ...
3. Construct dashboards to visualize the system state
    1. To have a high level view of the system that we can always access
4. Use Application Insights to gain insights into the app
    1. Application insights offers great features especially with web-apps that can indicate the availability of the system and raise alerts when needed

Azure Architecture Center

1. **Central hub for all-things Azure architecture (built by Microsoft)**
    1. It has:
        1. How-tos
        2. Documents
        3. Design guidelines
        4. Case studies
    2. It has fresh content
    3. It is updated regularly
2. https://docs.microsoft.com/en-us/azure/architecture
    1. **Have a look into it at-least once a month to find out if there are new updates, new patterns, new services to use**

Migrating to the Cloud

Introduction

1. Many organizations want to migrate their apps to the cloud
2. Various motivations
3. Has to be planned ahead, and done methodically
4. Rushing to the cloud could end up badly
5. There's a well defined process that should be followed in order for the migration to complete successfully

The Migration Process

1. Process:
    1. Step 1: Motivation Assessment
    2. Step 2: System Assessment
    3. Step 3: Migration
    4. Step 4: Enhancements
    5. Step 5: Testing
    6. Step 6: Go Live

Motivation Assessment

1. Answer the grand question:
    1. Why? (do we want to migrate to the cloud)
        1. Possible answers:
            1. To save costs
                1. Not always the case
                    1. Sometimes cloud system can cost MORE than on-premises system
                        1. CapEx vs OpEx
                            1. Cloud: Only OpEx
                            2. On-Premise system: CapEx + OpEx
                        2. Moving to OpEx model will not necessarily save us money
                2. A very thorough cost estimation should be conducted before deciding that migrating to the cloud is going to save cost
            2. To take advantage of cloud capabilities (services, redundancy, scalability, etc.)
                1. Greate motivation! (probably the best...)
                    1. Make sure the migrated system can actually use cloud capabilities
                        1. Example: Legacy, single-session app that cannot be scaled will end up on a single VM, just like on-prem
                        2. Solution: Ensure that the capabilities can actually be used
            3. To modernize the system

1. Usually modernization = Rewrite
    1. Why?
        1. Possible answers:
            1. Current platform is not supported anymore
            2. Difficult to hire new employees because there is not enough knowledge in the market on the system and platform
            3. Licensing costs
4. To attract new employees
    1. Not a very good motivation, but it works
        1. The organization has to be part of the cloud trend in order to attract new employees
            1. Because new employees will usually look for upto date, bleeding edge technology
                1. Not being in cloud will not look good

Migration Strategies

1. Three strategies:
    1. Lift & Shift
        1. Install the app in the cloud "as-is" without any changes, usually on a VM
            1. Quickest method
            2. Less efficient one
                1. We do not enjoy the various services that the cloud can offer us
    2. Refactoring
        1. Modify the app a bit so it can run on managed services (example on App services)
    3. Rewrite
        1. Rewrite the app from scratch, usually in a modern platform, and take advantages of all cloud services
2. Lift & Shift:
    1. Two VMs On-Premise
        1. Java App
        2. Oracle DB
    2. We will have identical VMs in the cloud
        1. None of the other services are used
3. Refactor:
    1. Two VMs On-Premise
        1. NoteJS App
        2. MySQL DB
    2. Cloud:
        1. App service: NodeJS App
            1. Minor code changes might be required
        2. Azure MySQL: MySQL DB (Azure does not have a managed Oracle DB hence refactored to support MySQL)
            1. Use managed services instead of un-managed services whenever possible
                1. Backups
                2. Reliability
                3. Redundancy
                4. ...
4. Rewrite:
    1. Two VMs On-Premise
        1. C++ App (no equivalent in Azure)
        2. Files Storage (less effective than database)
    2. Cloud:
        1. No similarities
            1. Full rewrite is used and not a single line of code is used
5. Comparing Cloud Migration Strategies:
    1. Cloud Effectiveness
    2. Time to Market (TTM)
        1. The more cloud effective our system is, the longer TTM will take
    3. Comparison:
        1. Lift & Shift
            1. Least Cloud Effective (unmanaged services)
            2. Least TTM (almost immediately)
        2. Refactor:
            1. Some Cloud Effectiveness (some managed services)
            2. Longer TTM (need some changes to the code)
        3. Rewrite
            1. Most Cloud effective (we take the full advantage of all the services that cloud offers us)
            2. Longest TTM (Needs rewriting the entire code)
    4. The strategies are often combined:
        1. Step 1: Lift & Shift
            1. Take the VMs and put them in the cloud (to ensure that everything works correctly)
        2. Step 2: Starting Optimization
            1. Might require a re-write of some modules
            2. Might require some changes to the other modules
            3. Rest of the modules might remain the same

System Assessment

1. Goal: To find out what is the best way to migrate the system to the cloud
    1. Doing this by asking the right questions
        1. Questions we need to ask:
            1. What is the runtime platform of the system and on which OS? (.Net, Java, C++ etc.)
            2. How is it deployed today? (VM, Docker, ...)
            3. What database(s) is/are used?
            4. What is the authentication mechanism?
            5. Any interfaces to organizational systems?
            6. Any work with hardware? (dongles...)
        2. Answers ditate migration strategy, if possible at all:
            1. What is the runtime platform of the system and on which OS? (.Net, Java, C++, etc.)
                1. If C++, Refactor is not an option (cloud does not support C++ managed apps)

                1. Stay with VM Or
                2. Complete rewrite
            2. Any interfaces to organizational systems?
                1. Yes:
                    1. Migration is not an option
                        1. After migration, the organizational systems are not accessible from the cloud
                            1. How to overcome?
                                1. VPN
                                2. Express route
            3. What is the authentication mechanism?
                1. OAuth2 - Transition to Azure AD will be easy
                    1. There is built in support for OAuth2 with AD
                2. Completely different protocol or proprietary protocol:
                    1. Migration will be complicated
2. Azure Migrate:
    1. A central hub for assessing and migrating various resources
    2. Uses platform-specific agents to discover the resources that can be migrated
    3. Provides migration recommendations
    4. Free
    5. Search for: Azure Migrate
        1. Migration goals:
            1. Servers
            2. Databases
            3. VDI
            4. Web-Apps
            5. Data Box
    6. Good for low-level assessment (what platforms used etc.) but won't deal with other considerations
        1. Example:
            1. Interfaces with organizational systems (it does not look at our code but looks only at the metadata of platform (OS platform, ...))
        2. Can be used as a first-step, but must perform manual assessment afterwards

Migration

1. Migration strategy depends on the system assessment results
    1. Lift & Shift:
        1. The easiset strategy
        2. Simply construct VM images and build new VMs in the cloud
        3. Don't forget to put VMs in appropriate VNet and define NSG
            1. Never expose the VM to the internet with all the ports open
    2. Refactor:
        1. Should be done in two phases:
            1. Lift & Shift - Make sure the app works in the cloud as-is
            2. Refactor - gradually improve the system and integrate cloud services
        2. Don't try shortcuts
            1. Don't refactor before the lift & shift works
    3. Rewrite:
        1. Not really a migration
            1. Design the system from the ground up to be cloud native
            2. We use as many cloud services as possible
2. DB Migration:
    1. In general:
        1. Prefer the managed version of the DB (and not on a VM)
            1. Migrate to managed MySQL database from on-premise MySQL database
        2. Migration is better done using the DB's own tools and not via Azure migration services
            1. Every DB has its own migration tools (export, import, backup, ...)
                1. They give the best results of the migration

App Enhancements

1. After migration, enhance the system
    1. Not necessarily requires code Changes
        1. Main areas of improvements:
            1. Logging and Monitoring using Azure Monitor
            2. Network protection using Application Gateway
2. Examples:
    1. Lift & Shift:
        1. Add Application Gateway and Monitor
            1. App is well Protected
            2. App is well monitored (If there is a problem, we will be notified immediately)

Testing

1. Testing in cloud is similar to on-prem
    1. Put strong emphasis on logging and monitoring
        1. We want to makes sure we know what is happening to the system
    2. Make sure we have access to system's data
        1. If the logging and data are stored in a secure location, then we as system admin and as a system architect must be able to look at the data so that we can find out if there are any problems that we need to take care of
    3. If system is auto-scaled or DRed - test these scenarios
        1. Shut down systems in the primary region and ensure that the secondary region is immediately up and running
            1. Depending on the design
    4. Check performance - might vary from the on-prem figures
        1. We might find that the system is slower in the cloud than On-Prem
            1. There are various elements that function differently
        2. We might find exactly opposite
            1. System is much faster than on-prem

1. Consider scaling down

Go Live

1. Keep an eye on the costs
    1. Set up budget alerts
        1. To find out if the cost increases above a given threshold
    2. Define tags:
        1. For various Services
            1. To monitor them more easily
    3. Look at the data at least once a month
        1. To find out if there are places for optimization

Advanced Services

Introduction

1. Full List of Azure Services:
    1. https://azurecharts.com/overview
2. We have not covered even half of the services
3. To review 3 interesting services

IOT Hub

1. Central message hub for IoT devices

    1. IoT = Internet of Things (cameras, sensors, connected cars etc.)
        1. Booming industry
        2. New IoT devices are released almost on a daily basis
    2. IoT hub handles security, relaibility, enrichment, routing and more (of IoT communication)
    3. Integrates with other Azure services
    4. Pros:
        1. It can handle millions of devices easily

2. Use-cases:

    1. If we are planning to build an IoT system
        1. System that connects telemetry from connected cars
        2. System that receives photos from connected cameras

3. Architecture:

```
IoT Device ---> IoT Hub
                1. Validates the message
                2. Ensures that IoT device is secure and no-one tampered with it
                    |
                    v
                Message Router (Routes the message to various endpoints)
                    |
                    v
                Endpoints:
                    1. Built In endpoint (EventHub ...)
                    2. Custom Endpoint Connectors:
                        1. Event Hub (custom)
                        2. Service Bus Topics
                        3. Service Bus Queues
                        4. Storage Blob
```

    1. IoT Hub handles all the complications of connecting the IoT device to the cloud system and it does it really well

Notification Hub

1. It is a central hub for sending push notifications to any Platform

    1. Supports:
        1. iOS
        2. Android
        3. Windows
        4. Kindle
        5. Baidu

2. It can be used by any backend, including on-premise one

3. **Supports millions of mobile devices**

4. Architecture:

```
App backend (on-prem or cloud)
    |
    Notification sent to tag: user_Alice
    |
    v
Notification Hub <- Mobile, Tab: Register for tag: user_Bob
    |
    v
Mobile: Registered for tag: user_Alice
Tablet: Registered for tag: user_Alice
```

    1. It sends notifications to all the devices with the registered tag

Cognitive Services

1. It is not exactly a service but APIs
    1. Set of APIs running AI (Artificial Intelligence) algorithms
    2. It does not require machine learning knowledge (or AI knowledge)
        1. We just need to know how to access the APIs
    3. The API enables:
        1. Anomaly detection
        2. Language understanding
        3. Translator
        4. Speech to text
        5. Computer vision
        6. ...
    4. Extremely easy to use
2. Example:
    1. Upload a photo using the API
        1. It estimates age and gender (quite accurate)
        2. It can indicate that there is no adult content in the photo
        3. It categorizes it as people group
3. If we don't want to learn full AI but need AI capabilities, we can use the cognitive services

Summary

1. Azure has hundreds of services
    1. Before developing our own - we can check if it doesn't already exist
        1. It is easy to miss very useful and very relevant services
    2. We'll cover many of the services in future courses

Conclusion

Download the Azure Architecture Summary

1. Azure Architecture Summary:
    1. A great reminder of the various aspects of the cloud services
        1. Compute
        2. Networking
        3. Data
        4. Messaging
        5. ...
    2. Outlines the various alternatives of every aspect
        1. So that we will know various options in compute, networking, data, messaging, ...
    3. Document is extremely handy during the design process
        1. If we use it during the design process, it will ensure that the architecture is upto date
    4. Download the document from resources section
        1. Highly recommended to be used

Conclusion

1. What we've learned
    1. What is the cloud
    2. Using Azure services
        1. Compute
        2. Data
        3. Networking
        4. Messaging
        5. Identity
        6. ...
    3. Hands on!
        1. Best approach to learning Azure
    4. Helps us to become a great Azure Developer & Architect!
2. Began with 2 VMs and we ended up with the architecture
3. Agenda:
    1. Welcome
    2. Introduction to the Cloud
    3. Introduciton to Azure
    4. First Look at Azure
    5. Basic Concepts
        1. Regions
        2. Zones
        3. Resource Groups
        4. ...
    6. Introducing Our App (Readit website)
    7. Compute
        1. VMs
        2. App Services
        3. AKS
        4. Functions
    8. Networking
        1. VNets
        2. Subnets
        3. NSGs
        4. Load balancers
        5. Application Gateway
        6. ...
    9. Data

        1. Databases
    10. Messaging
        1. Event grid
        2. Event hub
        3. ...
    11. Azure Active Directory
        1. Authentication to readit website
    12. Monitoring
        1. Logging
        2. Application Insights
        3. Azure Monitor
    13. Security
        1. Techniques to secure cloud systems
    14. DR
        1. Ensure the app is running even in the case of a complete region failure
    15. Cost Management
        1. Techniques to make sure the cost is managed
    16. Azure Policy
        1. Helps us enforce organizational standards across Azure services
    17. Architecting Apps for Azure
        1. Best practices to make sure the apps are robust, reliable, and efficient
    18. Migrating to the Cloud
        1. A challenge to a lot of organizations
    19. Advanced Services
        1. IoT Hub
        2. Notification Hub
    20. Conclusion

BONUS: Next Steps

1. Join Facebook group: Software Architects Discussions
    1. Community for architects and developers
    2. To ask questions
    3. To share info
    4. To post thoughts about all-things architecture, software, and Azure
2. Software Architecture Newsletter
    1. Highlights of:
        1. Software Architecture
        2. Cloud
        3. Development
        4. DevOps latest developments
3. Discounted courses:
    1. https://www.udemy.com/course/microsoft-azure-from-zero-to-hero-the-complete-guide/learn/lecture/25410952#overview
4. Website